BestoClinic

Technical Project Report

Version 1.0.0

BestoClinic Engineering Team

May 2026

Comprehensive technical project report covering architecture, design, database schema, workflows, API surface, security, deployment and operations.

Contents

Part I: Overview **5**
1. Executive Summary . . . 6
1.1 Project Overview . . . 6
1.2 Key Highlights . . . 6
1.3 Problem Statement . . . 6
1.4 Solution . . . 7
2. Product Introduction . . . 8
2.1 Vision and Scope . . . 8
2.2 User Hierarchy and Personas . . . 8
2.2.1 Product Owner . . . 8
2.2.2 Super Admin . . . 8
2.2.3 Admin . . . 8
2.2.4 Manager . . . 9
2.2.5 Department Manager . . . 9
2.2.6 Doctor . . . 9
2.2.7 Receptionist . . . 9
2.2.8 Support Staff . . . 10
2.2.9 Billing Officer . . . 10
2.2.10 Accountant . . . 10
2.2.11 HR/Payroll Manager . . . 10
2.2.12 IT Support . . . 11
2.2.13 Patient . . . 11
2.2.14 Guardian . . . 11
2.3 Core Value Proposition per Persona . . . 11
3. Business Model . . . 13
3.1 Revenue Model Details . . . 13
3.2 Competitive Advantages . . . 13
3.3 Market Positioning . . . 13
4. Technology Stack . . . 14
4.1 Backend Technologies . . . 14
4.2 Frontend Technologies . . . 15
4.3 Integration Services . . . 16
4.4 DevOps & Infrastructure . . . 16

Part II: Architecture & Design **18**
5. System Architecture . . . 19
5.1 Architecture Overview . . . 19
5.2 Layered Architecture . . . 19
5.3 Component Interaction Diagram . . . 20
6. High-Level Design . . . 21
6.1 Request Lifecycle . . . 21
6.2 Authentication & Authorization Architecture . . . 22
6.3 Multi-Tenancy Architecture . . . 24
6.4 Scalability Architecture . . . 26
7. Low-Level Design . . . 27
7.1 Database Design . . . 27

7.2 Key Algorithms . . . 27
7.2.1 Availability Slot Generation . . . 27
7.2.2 Invoice Calculation with GST . . . 29
7.2.3 Cash Register Variance Detection . . . 29
7.2.4 Appointment Conflict Detection . . . 29
7.2.5 Subscription State Machine . . . 29
7.2.6 Tenant Connection Pool Management . . . 29
7.2.7 AI Dental Analysis JSON Repair . . . 31
7.2.8 Appointment Reminder Scheduling . . . 31
8. Database Schema Architecture . . . 32
8.1 Schema Specifications . . . 32
8.1.1 Platform Models . . . 32
8.1.2 Tenant Models — Clinical . . . 35
8.1.3 Tenant Models — Scheduling . . . 37
8.1.4 Tenant Models — Financial . . . 37
8.1.5 Tenant Models — Inventory . . . 38
8.1.6 Tenant Models — Organization . . . 38
8.1.7 Tenant Models — Communication, Integration, Security, Insurance 38
8.2 Schema Relationships Summary . . . 39

Part III — Core Features & User Journeys 40
Chapter 9: Core Module Deep Dive . . . 41
9.1 Patient Module . . . 41
9.2 Appointment Module . . . 41
9.3 Invoice Module . . . 43
9.4 Doctor Dashboard & Consultation Module . . . 46
9.5 Smart Queue Module . . . 48
9.6 Payment Module . . . 49
9.7 Cash Register Module . . . 50
9.8 Prescription Module . . . 51
9.9 Billing Module (Enhanced) . . . 51
Chapter 10: Feature Specifications . . . 53
10.1 AI Dental Scan System . . . 53
10.2 Telemedicine & Video Calling . . . 55
10.3 Emergency Module . . . 56
10.4 Subscription & Razorpay Integration . . . 57
10.5 Insurance Claims Management . . . 59
10.6 Patient Portal (Self-Service) . . . 61
10.7 Analytics Module . . . 62
10.8 Inventory & Purchase Orders . . . 63
10.9 Calendar Feeds (ICS Export) . . . 64
Chapter 11: User Workflows & Journeys . . . 66
11.1 Patient Booking Workflow . . . 66
11.2 In-Clinic Consultation Workflow . . . 67
11.3 Telemedicine Workflow . . . 67
11.4 Billing & Payment Workflow . . . 68
11.5 Emergency Triage Workflow . . . 69
11.6 Schedule Management Workflow . . . 70
11.7 Inventory & Procurement Workflow . . . 72

Part IV — Technical Implementation Details 73
Chapter 12: API Architecture . . . 74
12.1 API Design Principles . . . 74
12.2 Complete Route Registry . . . 74
12.3 API Endpoint Summary by Category . . . 75
12.4 Middleware Execution Chain . . . 76
Chapter 13: Multi-Tenancy Implementation . . . 78
13.1 Architecture Overview . . . 78
13.2 Tenant Connection Management . . . 78

13.3 Tenant Resolution Middleware . . . 79
13.4 Database URI Encryption . . . 79
13.5 Tenant Provisioning Pipeline . . . 79
13.6 Cross-Tenant Operations . . . 81
Chapter 14: Security & Authentication . . . 82
14.1 Authentication Architecture . . . 82
14.2 Authorization (RBAC) . . . 84
14.3 Role Hierarchy . . . 86
14.4 Subscription Enforcement . . . 86
14.5 Security Headers & Rate Limiting . . . 86
14.6 Data Encryption . . . 87
14.7 Audit & Compliance . . . 87
Chapter 15: Integration Points . . . 88
15.1 Razorpay (Payment Gateway) . . . 88
15.2 Google Gemini AI (Dental Analysis) . . . 88
15.3 Firebase Cloud Messaging (Push Notifications) . . . 89
15.4 Nodemailer (Email) . . . 89
15.5 AWS S3 (File Storage) . . . 89
15.6 External Dental Imaging Systems (RioView / Romexis) . . . 89
15.7 WebSocket (Socket.IO) . . . 90
15.8 BullMQ (Job Queues) . . . 91

Part V — DevOps & Infrastructure 92
Chapter 16: Deployment Architecture . . . 93
16.1 Container Architecture . . . 93
16.2 Service Topology . . . 93
16.3 Boot Sequence . . . 93
16.4 Graceful Shutdown . . . 95
16.5 Environment Configuration . . . 96
Chapter 17: System Monitoring & Observability . . . 97
17.1 Health Check Endpoint . . . 97
17.2 Logging . . . 97
17.3 Error Tracking . . . 97
17.4 Platform Monitoring . . . 97
17.5 Connection Pool Monitoring . . . 98
Chapter 18: Scalability Considerations . . . 99
18.1 Horizontal Scaling Strategy . . . 99
18.2 Caching Strategy . . . 99
18.3 Database Indexing Strategy . . . 99
18.4 Performance Patterns . . . 100
18.5 Connection Limits . . . 100

Part VI — Future Considerations 101
Chapter 19: Planned Enhancements . . . 102
19.1 Near-Term Roadmap . . . 102
19.2 Medium-Term Enhancements . . . 102
Chapter 20: Technical Debt & Recommendations . . . 103
20.1 Identified Technical Debt . . . 103
20.2 Architecture Recommendations . . . 103
20.3 Security Recommendations . . . 104

Appendices 105
Appendix A: Glossary . . . 106
Appendix B: Sample API Request/Response Payloads . . . 107
B.1 Create Appointment . . . 107
B.2 Complete Consultation . . . 107
B.3 Paginated Patient List . . . 109
Appendix C: Sample Webhook Payloads . . . 110
C.1 Razorpay Payment Captured . . . 110

C.2 Razorpay Subscription Charged 110
Appendix D: Configuration Reference 112
D.1 Subscription Plans 112
D.2 Feature Matrix 112
D.3 Role-Permission Matrix (Summary) 112

Part I: Overview

1. Executive Summary

1.1 Project Overview

BestoClinic is a multi-tenant Software-as-a-Service (SaaS) platform purpose-built for dental clinic management. It provides a unified system for appointment scheduling, clinical consultations, billing, inventory, HR/payroll, telemedicine, AI-powered dental scanning, and patient engagement — all isolated per clinic via dedicated tenant databases.

The platform serves three deployment surfaces: a React web application for clinic staff and administrators, a patient-facing web portal, and a React Native mobile application.

1.2 Key Highlights

Attribute	Detail
Platform Type	Multi-tenant SaaS
Target Market	Dental clinics (India-primary, IST timezone)
Architecture	Node.js/Express REST API + Socket.IO real-time + BullMQ async jobs
Database	MongoDB 7 (per-tenant isolation) + Redis 7 (cache/queue/sessions)
Frontend	React 19 SPA (Vite) + React Native mobile
AI Integration	Google Gemini 2.0 Flash for dental image analysis
Payment Gateway	Razorpay (subscriptions + patient payments)
Deployment	Docker (Node 18-alpine + MongoDB 7 + Redis 7)
Codebase	357 backend JS files, 347 frontend JSX files, 74 Mongoose models, 50+ modules

1.3 Problem Statement

Persona	Problem
Clinic Owner	Fragmented tools for scheduling, billing, inventory, HR. No single system covers the full operation.
Doctor	Paper-based or disconnected consultation records. No AI-assisted diagnostics. No telemedicine.
Receptionist	Manual queue management, phone-based appointment booking, no real-time visibility.
Billing Officer	Disconnected invoicing, manual GST calculations, no cash register reconciliation.
Patient	No self-service booking, no access to records, no prescription reminders.

Persona	Problem
Platform Operator	No centralized way to onboard clinics, manage subscriptions, or monitor tenants.

1.4 Solution

Persona	Solution
Clinic Owner	Single platform: appointments, billing, inventory, HR, payroll, analytics, compliance — all in one. Multi-branch support.
Doctor	Digital SOAP consultation form with auto-save, prescription builder, care task reminders, AI dental scan review, telemedicine video calls.
Receptionist	Real-time queue management, drag-and-drop scheduling, walk-in handling, patient check-in, billing integration.
Billing Officer	Invoice generation with GST, payment recording (cash/card/UPI), cash register with variance detection, insurance claims, credit notes.
Patient	Self-service portal: browse doctors, book appointments, receive reminders, view prescriptions, manage dependents, video consultations.
Platform Operator	Admin dashboard: tenant provisioning, subscription management, feature flags, revenue analytics, dental scan analytics.

2. Product Introduction

2.1 Vision and Scope

BestoClinic aims to be the operating system for dental clinics. The platform covers the full lifecycle: patient discovery → appointment booking → check-in → consultation → treatment → billing → follow-up care. It extends to back-office operations (inventory, procurement, HR, payroll) and platform-level concerns (subscription billing, tenant provisioning, feature gating).

The scope for v1.0 includes: - Multi-tenant architecture with per-clinic database isolation - 14 user roles with 80+ granular permissions - Appointment scheduling with conflict detection and availability calculation - Clinical consultation with SOAP notes, prescriptions, and care tasks - AI dental scanning via Google Gemini - Billing with GST, insurance claims, cash register, credit notes - Telemedicine with WebRTC video calls - Patient portal with self-service booking and record access - Inventory and purchase order management - HR, payroll, and leave management - Real-time notifications via Socket.IO and Firebase push - Platform admin with subscription management, feature flags, promotions - Job portal for clinic recruitment

2.2 User Hierarchy and Personas

2.2.1 Product Owner

- **Role slug:** `product_owner`
- **Capabilities:**
  - Manage all tenants (create, suspend, activate, cancel)
  - Manage subscription plans and pricing
  - Configure feature flags globally and per-tenant
  - View platform-wide analytics (revenue, MRR, churn, tenant growth)
  - Manage promotions and rebates
  - Monitor system health and performance
  - Manage job portal across all tenants
- **Count:** 1-3 per platform deployment
- **Data Isolation:** Full access to platform DB; can access any tenant DB

2.2.2 Super Admin

- **Role slug:** `super_admin`
- **Capabilities:**
  - Full clinic management (branches, departments, staff, doctors)
  - All billing and financial operations
  - All clinical operations
  - Subscription viewing (not management)
  - Integration configuration
  - Analytics and reporting
- **Count:** 1-2 per clinic
- **Data Isolation:** Own tenant DB only

2.2.3 Admin

- **Role slug:** `admin`
- **Capabilities:**

  - Branch, department, staff management
  - Appointment and queue management
  - Billing, invoicing, payments
  - Inventory and purchase orders
  - Analytics and reporting
  - Clinic settings
- **Count:** 1-5 per clinic
- **Data Isolation:** Own tenant DB; optionally filtered by branch

2.2.4 Manager

- **Role slug:** `manager`
- **Capabilities:**
  - Staff oversight
  - Appointment management
  - Queue management
  - Report viewing
  - Leave approval
- **Count:** 1-5 per clinic
- **Data Isolation:** Own tenant DB; branch-scoped

2.2.5 Department Manager

- **Role slug:** `department_manager`
- **Capabilities:**
  - Department-specific staff management
  - Department appointment oversight
  - Department reporting
- **Count:** 1 per department
- **Data Isolation:** Own tenant DB; department-scoped

2.2.6 Doctor

- **Role slug:** `doctor`
- **Capabilities:**
  - View own appointments and schedule
  - Conduct consultations (SOAP notes, prescriptions, care tasks)
  - Review AI dental scans
  - Manage own schedule, leaves, overrides
  - Conduct telemedicine video calls
  - Handle emergency sessions
  - View own patients and their compliance
  - Create consultation templates
- **Count:** 1-50 per clinic (plan-limited)
- **Data Isolation:** Own tenant DB; self-scoped for appointments/patients

2.2.7 Receptionist

- **Role slug:** `receptionist`
- **Capabilities:**
  - Manage appointments (create, confirm, cancel, reschedule)
  - Patient registration and check-in
  - Queue management (add, call, complete, reorder)
  - Basic billing (create invoices, record payments)

  - View doctor availability
- **Count:** 1-10 per clinic
- **Data Isolation:** Own tenant DB; branch-scoped

2.2.8 Support Staff

- **Role slug:** `support_staff`
- **Capabilities:**
  - Patient check-in
  - Queue assistance
  - Appointment viewing
  - Notification management
- **Count:** 1-10 per clinic
- **Data Isolation:** Own tenant DB; branch-scoped

2.2.9 Billing Officer

- **Role slug:** `billing_officer`
- **Capabilities:**
  - Full invoicing (create, send, void, write-off)
  - Payment recording and refunds
  - Cash register operations (open, close, reconcile)
  - Credit note management
  - Insurance claim processing
  - Revenue analytics and day-end reports
  - GST summary generation
- **Count:** 1-5 per clinic
- **Data Isolation:** Own tenant DB; branch-scoped

2.2.10 Accountant

- **Role slug:** `accountant`
- **Capabilities:**
  - View all invoices and payments
  - Financial reporting (P&L, GST, collection metrics)
  - Financial audit trail viewing
  - Day-end settlement reports
- **Count:** 1-3 per clinic
- **Data Isolation:** Own tenant DB; read-only financial data

2.2.11 HR/Payroll Manager

- **Role slug:** `hr_payroll_manager`
- **Capabilities:**
  - Staff management
  - Salary structure configuration
  - Payslip generation and approval
  - Leave management and approval
- **Count:** 1-2 per clinic
- **Data Isolation:** Own tenant DB

2.2.12 IT Support

- **Role slug:** `it_support`
- **Capabilities:**
  - Clinic settings configuration
  - Integration management (connect/disconnect external systems)
  - System status monitoring
  - Sync log viewing
- **Count:** 1-2 per clinic
- **Data Isolation:** Own tenant DB

2.2.13 Patient

- **Role slug:** `patient`
- **Capabilities:**
  - Browse doctors and book appointments
  - View own medical records and prescriptions
  - Receive and respond to care task reminders
  - Make payments
  - Share records via token
  - Submit reviews
  - Join video consultations
  - Manage notification preferences
- **Count:** Unlimited
- **Data Isolation:** Own records only (self-scoped)

2.2.14 Guardian

- **Role slug:** `guardian`
- **Capabilities:**
  - All patient capabilities
  - Manage dependents (add, update, detach)
  - View dependent medical records
  - Book appointments for dependents
  - Manage dependent care tasks
- **Count:** Unlimited
- **Data Isolation:** Own records + linked dependents

2.3 Core Value Proposition per Persona

Persona	Value
Clinic Owner	Single subscription replaces 5+ disconnected tools. Multi-branch visibility.
Doctor	60% less time on paperwork via digital consultations and templates. AI scan assistance.
Receptionist	Real-time queue eliminates shouting across the clinic. One-click check-in.
Billing Officer	Automated GST calculation. Cash register variance alerts catch discrepancies same-day.

Persona	Value
Patient	24/7 self-service booking. Prescription reminders reduce missed medications.
Platform Operator	Automated tenant provisioning. Feature flags enable gradual rollout. Revenue tracking per plan tier.

3. Business Model

3.1 Revenue Model Details

BestoClinic operates on a tiered subscription model with four plans:

Feature	Free	Starter	Professional	Enterprise
Max Doctors	1	5	20	Unlimited
Max Staff	2	10	50	Unlimited
Max Branches	1	2	5	Unlimited
Storage (MB)	100	500	2000	Unlimited
SMS Notifications	No	Yes	Yes	Yes
Telehealth	No	No	Yes	Yes
Custom Branding	No	No	Yes	Yes
API Access	No	No	No	Yes
Advanced Reports	No	No	Yes	Yes
White Labeling	No	No	No	Yes
Job Portal	No	No	Yes	Yes
Priority Support	No	No	No	Yes

Additional revenue streams: - **Job Portal Credits:** Clinics purchase credit packs to post job listings. Monthly credit grants for active subscribers. - **Promotion Codes:** Discount campaigns for new clinic acquisition. - **Add-on Features:** Individual priceable features (SMS, telehealth, etc.) can be enabled independently.

3.2 Competitive Advantages

- **Per-tenant database isolation:** Full data separation, not just row-level filtering. Each clinic gets its own MongoDB database.
- **AI dental scanning:** Google Gemini-powered analysis provides findings, severity, and specialist recommendations from patient photos.
- **Integrated telemedicine:** WebRTC video calls with recording consent, chat, and file sharing — no third-party app needed.
- **Granular RBAC:** 80+ permission keys across 14 roles. Clinics can define custom roles.
- **Real-time operations:** Socket.IO powers live queue updates, emergency broadcasts, doctor presence, and notification delivery.

3.3 Market Positioning

BestoClinic targets the Indian dental clinic market as the primary deployment region (IST timezone handling, INR currency, GST compliance with CGST/SGST/IGST, Razorpay payment gateway). The platform scales from single-doctor clinics (Free tier) to multi-branch dental chains (Enterprise tier).

---

4. Technology Stack

4.1 Backend Technologies

Technology	Version	Purpose	Rationale
Node.js	>= 18.0	Runtime	Non-blocking I/O for concurrent connections; JavaScript full-stack
Express	4.21	HTTP framework	Mature middleware ecosystem; straightforward routing
Mongoose	8.23	ODM	Schema validation, middleware hooks, population; tight MongoDB integration
MongoDB	7	Primary database	Document model fits clinical data; per-tenant DB isolation via separate connections
Redis	7	Cache / Queue / Sessions	In-memory speed for rate limits, JWT blacklist, response caching, BullMQ job queue
BullMQ	5.12	Job queue	Reliable delayed/repeating jobs for reminders, scans, invoices, subscription lifecycle
Socket.IO	4.7.5	WebSocket	Real-time queue, presence, emergency, notifications; room-based broadcasting
JWT (jsonwebtoken)	9.0.2	Authentication	Stateless tokens; three separate secrets for ac-cess/refresh/platform
bcryptjs	2.4.3	Password hashing	12 rounds; timing-safe comparison
Joi	17.13	Validation	Declarative schema validation with human-friendly error messages

Technology	Version	Purpose	Rationale
Helmet	7.1	Security headers	CSP, HSTS, X-Frame-Options out of the box
Winston	3.13	Logging	Structured JSON logging with daily file rotation
PDFKit	0.18	PDF generation	Invoice and scan report PDF rendering
Nodemailer	8.0	Email	SMTP-based email delivery for notifications and password resets

4.2 Frontend Technologies

Technology	Version	Purpose	Rationale
React	19.2	UI framework	Component model; concurrent features; hooks
Vite	7.3.1	Build tool	Sub-second HMR; native ESM; fast production builds
Tailwind CSS	4.2.1	Styling	Utility-first; design tokens; no runtime CSS-in-JS overhead
React Router	7.13.1	Routing	File-system conventions; nested layouts; role-based redirects
Framer Motion	12.34	Animations	Declarative animations; layout transitions; stagger effects
Recharts	3.7	Charts	Composable chart components for analytics dashboards
React Big Calendar	1.19.4	Calendar	Week/month/day views for appointment and schedule display
Lucide React	0.575	Icons	Tree-shakeable icon set; consistent design language
Socket.IO Client	4.8.3	WebSocket	Real-time event subscription matching server Socket.IO

Technology	Version	Purpose	Rationale
MediaPipe	0.10.34	Pose detection	Face/hand detection for guided dental scan capture
QRCode.react	4.2	QR codes	Clinic QR code generation for patient check-in
PapaParse	5.5.3	CSV parsing	Bulk upload processing (patients, staff, inventory)
react-signature-canvas	1.1.0	Signature pad	Doctor signature capture on consultations
date-fns	4.1	Date utilities	Lightweight date formatting and manipulation

4.3 Integration Services

Service	Purpose	Integration Pattern
Razorpay	Payment processing (subscriptions + patient payments)	Server-side SDK + webhook signature verification (HMAC SHA256)
Google Gemini AI	Dental image analysis	REST API via `@google/generative-ai` SDK; async via BullMQ worker
Firebase Cloud Messaging	Push notifications (mobile + web)	Firebase Admin SDK; per-device FCM tokens
Nodemailer / SMTP	Email delivery	SMTP transport (Gmail or custom); queued via BullMQ email worker
AWS S3	File storage (production)	AWS SDK; configurable via `STORAGE_PROVIDER` env var
Local filesystem	File storage (development)	`uploads/` directory with subdirectories per type
WebRTC / TURN	Video calls	Peer-to-peer with TURN relay for NAT traversal

4.4 DevOps & Infrastructure

Component	Technology	Detail
Version Control	Git	Main branch; conventional commit messages
Containerization	Docker	Multi-stage Node 18-alpine; non-root user (appuser:1001)

Component	Technology	Detail
Orchestration	Docker Compose	3 services: api, redis, mongo; bridge network; health checks
Process Management	Node.js native	Graceful shutdown (SIGTERM/SIGINT); 28s timeout
Testing	Jest 29.7 + Supertest 7.0	Unit + integration tests; mongodb-memory-server for isolation
Linting	ESLint 8.57	Enforced in Docker build stage
Code Formatting	Prettier	Consistent code style
Log Rotation	winston-daily-rotate-file	Daily log files with configurable retention

Part II: Architecture & Design

5. System Architecture

5.1 Architecture Overview

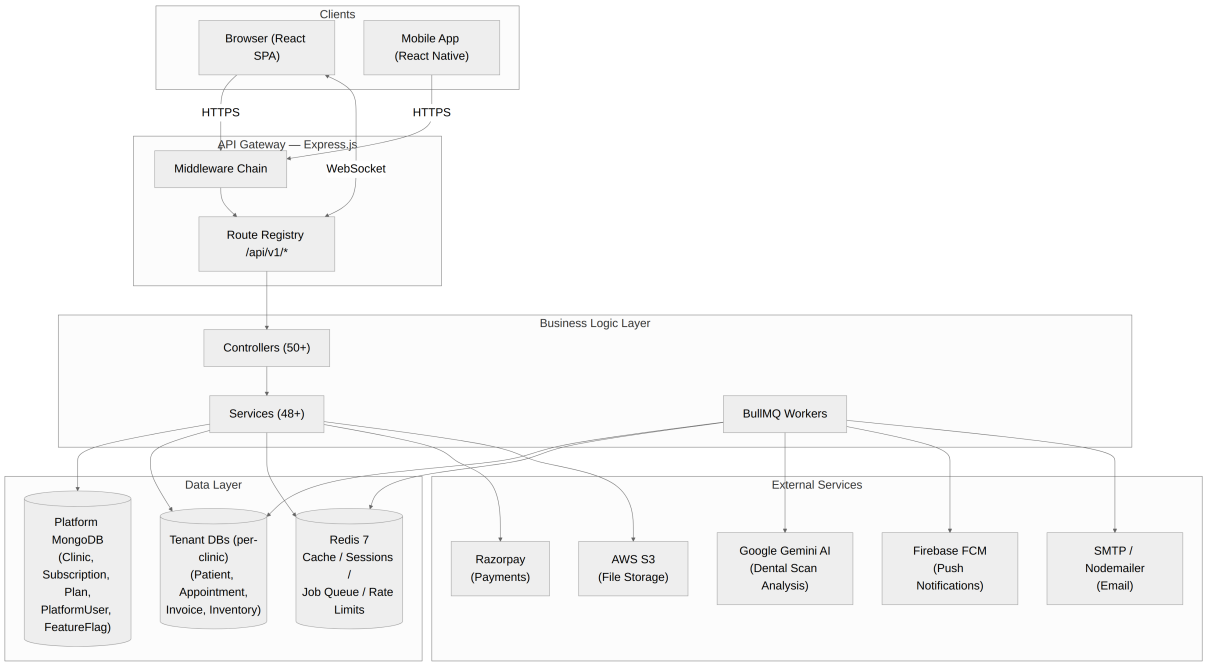


Figure 1: Diagram 1: System Architecture

BestoClinic follows a layered monolithic architecture deployed as a single Node.js process. The system comprises five layers:

1. **Presentation Layer:** React SPA + React Native mobile app, communicating over HTTPS and WebSocket.
2. **API Layer:** Express routes with a 16-stage middleware chain handling auth, tenancy, permissions, validation, caching, and rate limiting.
3. **Business Logic Layer:** 50+ feature modules each with controller/service/validator, plus 24 shared services.
4. **Data Layer:** Mongoose ODM with 74 schemas across platform and tenant databases, Redis for caching and session management.
5. **Integration Layer:** External service connectors for payments, AI, push notifications, email, and file storage.

5.2 Layered Architecture

Presentation Layer - React 19 SPA served by Vite (port 5173 dev, static in production) - React Native mobile app (Expo) - Socket.IO client for real-time events

API Layer - Express 4.21 with `/api/v1` route prefix - 43 route files mounted in `src/routes/index.js` - Middleware chain (16 middleware files in `src/middleware/`)

Business Logic Layer - 53 controllers handling HTTP request/response - 48 services encapsulating business rules - Joi validators for input sanitization - BullMQ workers for async processing

Data Layer - Platform MongoDB (shared): 21 models in `src/models/platform/` - Tenant MongoDB (per-clinic): 53 models in `src/models/tenant/` - Redis: JWT blacklist, response cache, rate limit counters, BullMQ queues, tenant URI cache

Integration Layer - Razorpay SDK for payment operations - Google Generative AI SDK for dental analysis - Firebase Admin SDK for push notifications - Nodemailer for SMTP email - AWS S3 SDK / local filesystem for storage

5.3 Component Interaction Diagram

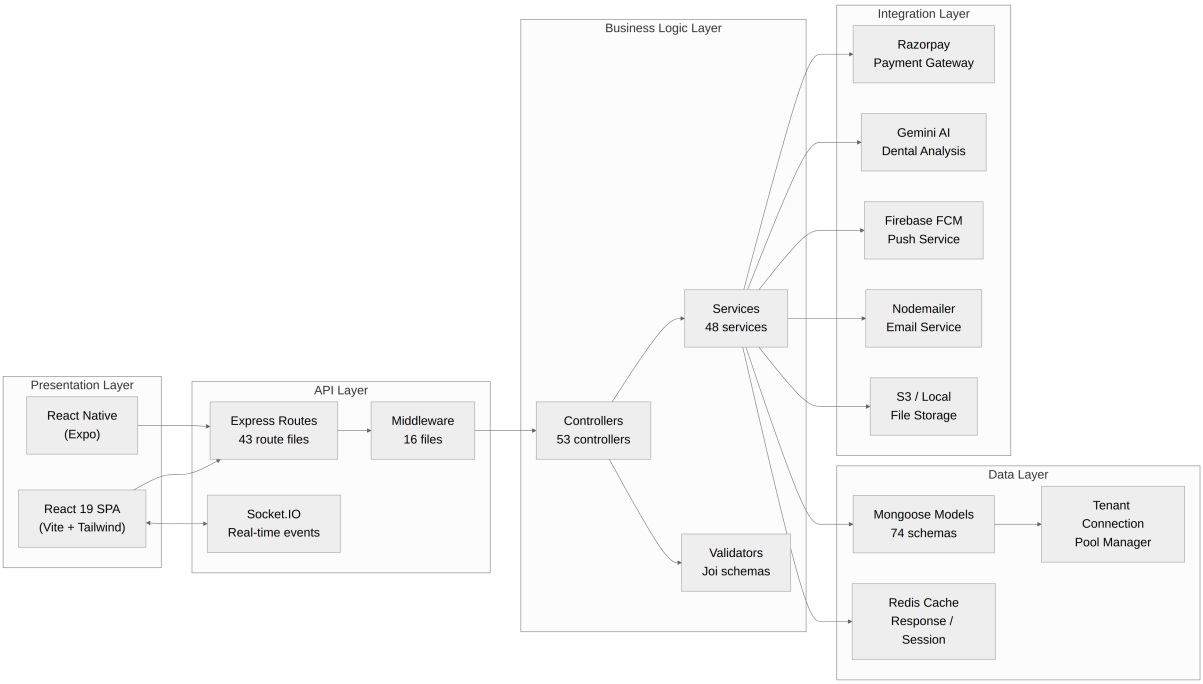


Figure 2: Diagram 2: Component Interaction

6. High-Level Design

6.1 Request Lifecycle

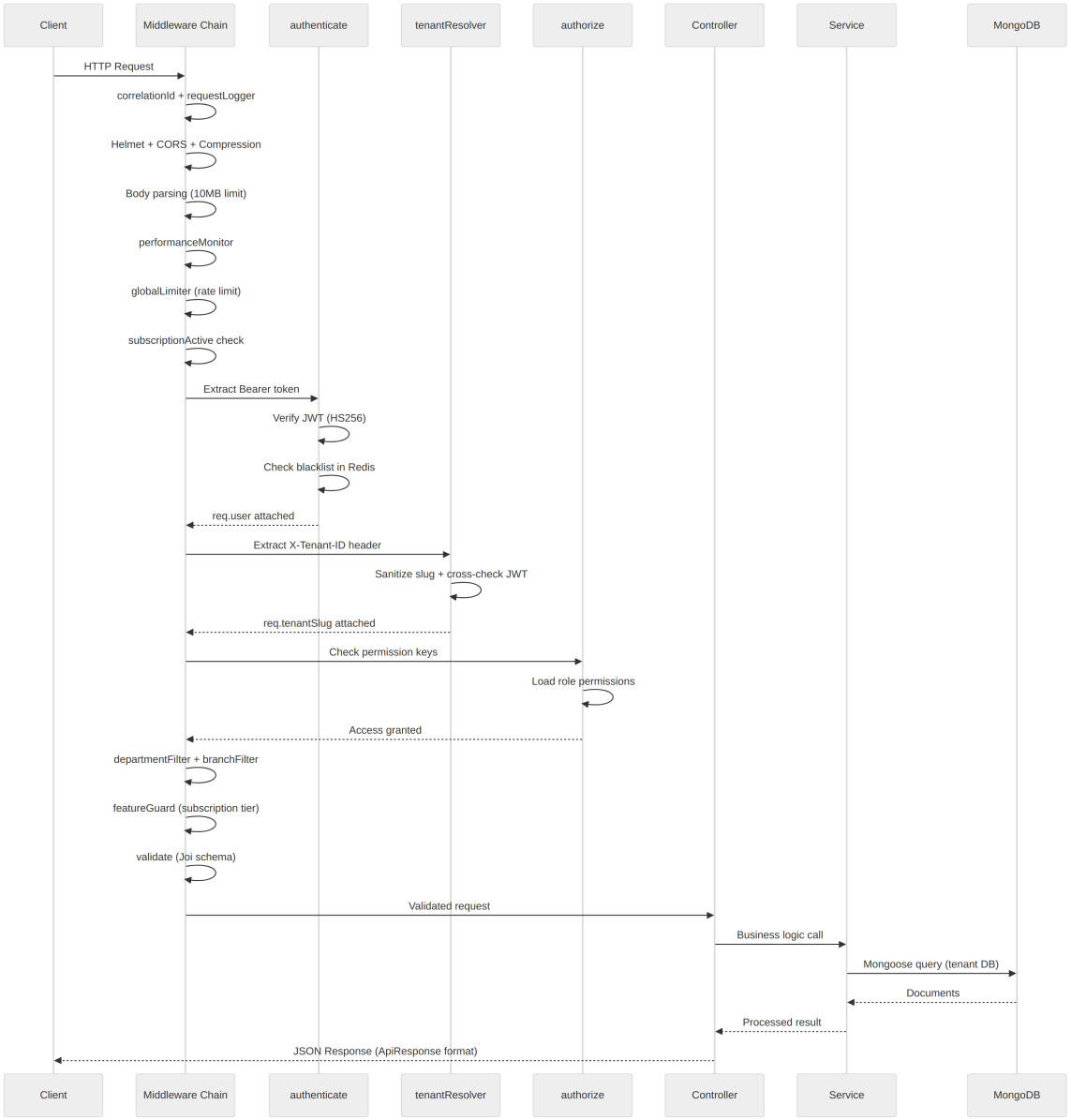


Figure 3: Diagram 3: Request Lifecycle

Every API request passes through the following middleware chain defined in `src/app.js`:

```
// Middleware execution order (src/app.js)
app.use(correlationId());          // Unique request ID
app.use(requestLogger());          // Morgan HTTP logging
app.use(helmet());                 // Security headers
app.use(cors(corsOptions));        // CORS policy
app.use(compression());            // Gzip responses
app.use(express.json({ limit: '10mb' }));  // Body parsing
app.use(performanceMonitor());     // Latency tracking
app.use(globalLimiter());          // Rate limiting (100 req/15min)
// Route-level: subscriptionActive, authenticate, tenantResolver,
//   authorize, departmentFilter, branchFilter, featureGuard, validate
```

```
app.use('/api/v1', routes);
app.use(errorHandler());           // Centralized error handling
```

6.2 Authentication & Authorization Architecture

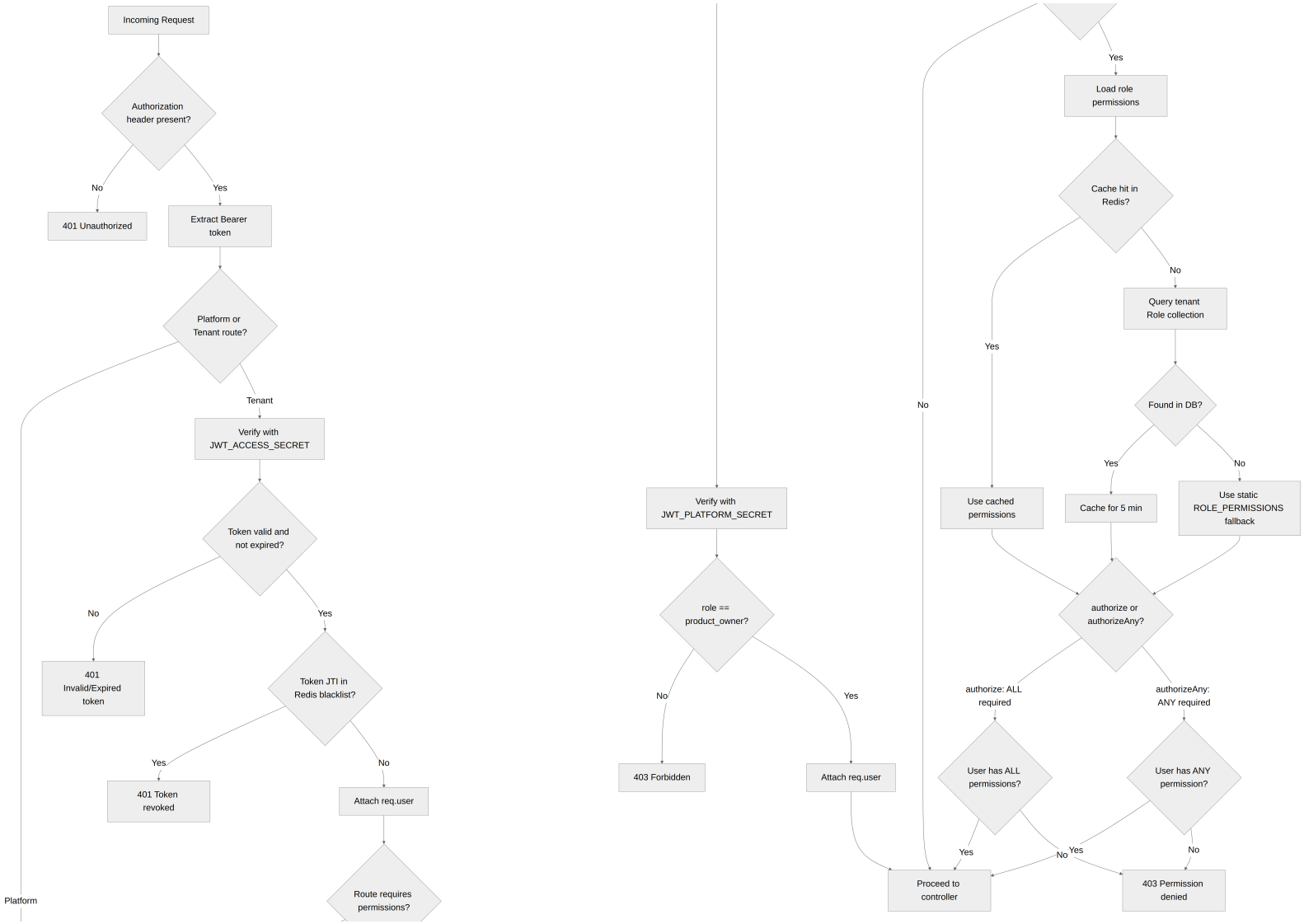


Diagram 4: Authentication and Authorization Flow

Token Structure:

Token	Secret Env Var	TTL	Audience	Use
Access	`JWT_ACCESS_SECRET`	15m	`bestoclinic-client`	API calls
Refresh	`JWT_REFRESH_SECRET`	7d	`bestoclinic-client`	Token renewal
Platform	`JWT_PLATFORM_SECRET`	1h	`bestoclinic-platform`	Product owner only

Access Token Payload:

```
{
  "userId": "ObjectId",
  "role": "doctor",
  "tenantSlug": "smilecare-clinic",
  "branch": "ObjectId",
  "department": "ObjectId",
  "jti": "uuid-v4",
  "iat": 1716620000,
  "exp": 1716620900,
  "aud": "bestoclinic-client",
  "iss": "bestoclinic"
}
```

Middleware chain for a protected route:

```
// Example: src/modules/patient/patient.routes.js
router.get('/',
  authenticate(),               // Verify JWT, check blacklist
  tenantResolver(),             // Extract X-Tenant-ID, cross-check JWT
  authorizeAny(
    PERMISSIONS.PATIENT_READ,   // Full access
    PERMISSIONS.PATIENT_READ_OWN // Self-scoped
  ),
  departmentFilter(),           // Set req.departmentFilter
  branchFilter(),               // Set req.branchFilter
  validate(listPatientsSchema, { source: 'query' }),
  patientController.listPatients
);
```

6.3 Multi-Tenancy Architecture

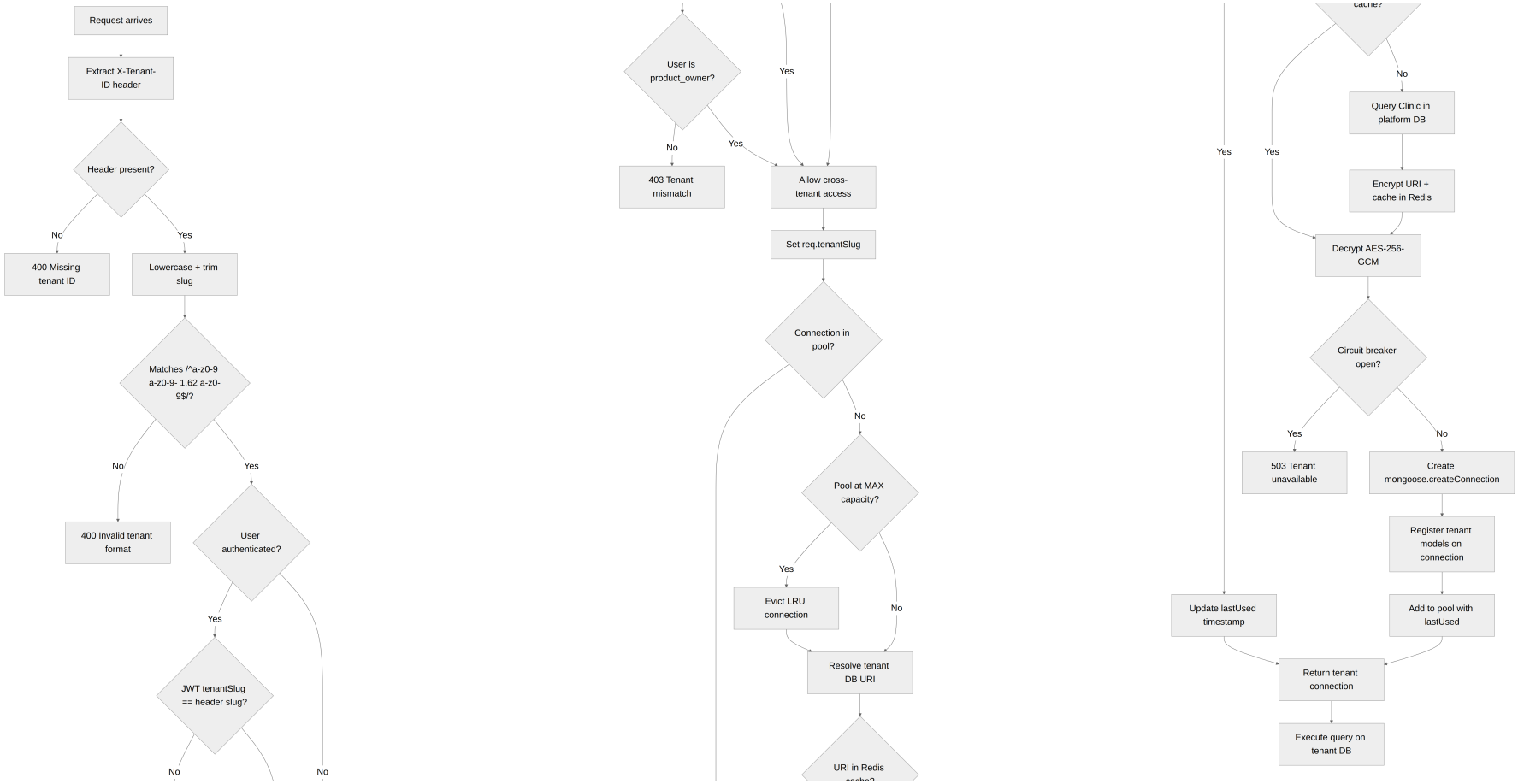


Diagram 5: Multi-Tenancy Request Flow

Isolation Strategy: Database-per-tenant. Each clinic gets its own MongoDB database. The platform database stores shared data (clinics, subscriptions, plans, platform users).

Tenant Resolution (src/middleware/tenantResolver.js): 1. Extract `X-Tenant-ID` header (case-insensitive) 2. Sanitize: lowercase, trim, validate against `/^[a-z0-9][a-z0-9-]{1,62}[a-z0-9]$/` 3. Cross-check: if user is authenticated, verify `JWT.tenantSlug === header slug` (product_owner exempt) 4. Attach `req.tenantSlug` for downstream use

Connection Pool (src/db/tenantConnectionManager.js):

```
Pool capacity: MAX_TOTAL_CONNECTIONS (configurable)
Idle timeout: 30 minutes
Pruning interval: every 10 minutes (via setInterval in server.js)
Circuit breaker: opens on connection error, prevents retry storms
URI resolution: Redis cache (encrypted AES-256-GCM) → Platform DB lookup
```

Before multi-tenancy (single DB):

```
const patients = await Patient.find({ clinicId: req.clinicId });
```

After multi-tenancy (isolated DB):

```
const conn = await getTenantConnection(req.tenantSlug);
const Patient = conn.model('Patient');
const patients = await Patient.find({}); // No clinicId filter needed
```

Security Measures: - JWT tenantSlug cross-checked against header on every request - Product owners can access any tenant; all other roles are locked to their own - Tenant DB URIs encrypted at rest in Redis with AES-256-GCM - Connection pool prevents resource exhaustion via hard cap and LRU eviction

6.4 Scalability Architecture

Horizontal Scaling: - Stateless API servers (JWT auth, no server-side sessions) - Redis shared across instances for cache, rate limits, job queue - Socket.IO can use Redis adapter for multi-process broadcasting - BullMQ workers can run on separate processes/machines

Caching Strategy: - Response caching via `cacheResponse(ttlSeconds)` middleware → Redis - Cached endpoints: queue current (15s), notifications unread count (30s), branch monitor (15-30s) - Feature flag resolution cached 5 minutes - Role permissions cached 5 minutes - Tenant DB URIs cached in Redis (encrypted)

Database Optimization: - Compound indexes on all high-frequency queries (defined per model) - Soft deletes (`deletedAt: null`) with pre-find hooks for automatic filtering - Lean queries (`.lean()`) for read-only operations - Aggregation pipelines for analytics (revenue, appointment trends, no-show analysis)

---

7. Low-Level Design

7.1 Database Design

The database is divided into two tiers:

Platform Database (21 collections): Shared across all tenants. - User Management: PlatformUser, UserDirectory, PlatformPatient - Tenant Management: Clinic, Subscription, Plan, ProvisioningStatus - Feature Management: FeatureFlag, Integration - Financial: CreditWallet, CreditTransaction, Promotion, Rebate, WebhookEvent - Content: JobPosting, JobApplication, FeaturedDoctor, GuestScan, PrescriptionScan - Cross-tenant: CalendarFeedToken, TenantAuditLog

Tenant Database (53 collections per clinic): Isolated per tenant. - Clinical: Patient, Appointment, TreatmentRecord, Prescription, PrescriptionTask, DentalScan, DentalScanResult, Consent, GuardianConsent, ConsultationRoom, ConsultationTemplate, EmergencySession, CareReminder - Scheduling: DoctorSchedule, DoctorScheduleOverride, DoctorLeave, QueueEntry, CalendarFeed, ClinicBookingPolicy, ClinicHoliday - Financial: Invoice, Payment, CreditNote, PatientLedger, FinancialAuditLog, CashRegister, ServiceItem - Inventory: InventoryItem, InventoryTransaction, PurchaseOrder - Organization: Branch, Department, Role, Staff, User, SalaryStructure, Payslip - Communication: Notification, ChatMessage, Review - Integration: IntegrationCredential, SyncLog, LegacyUpload - Security: AccessLog, ShareToken, ComplianceAlert, DependentArchive - Insurance: InsuranceProvider, InsuranceClaim

7.2 Key Algorithms

7.2.1 Availability Slot Generation

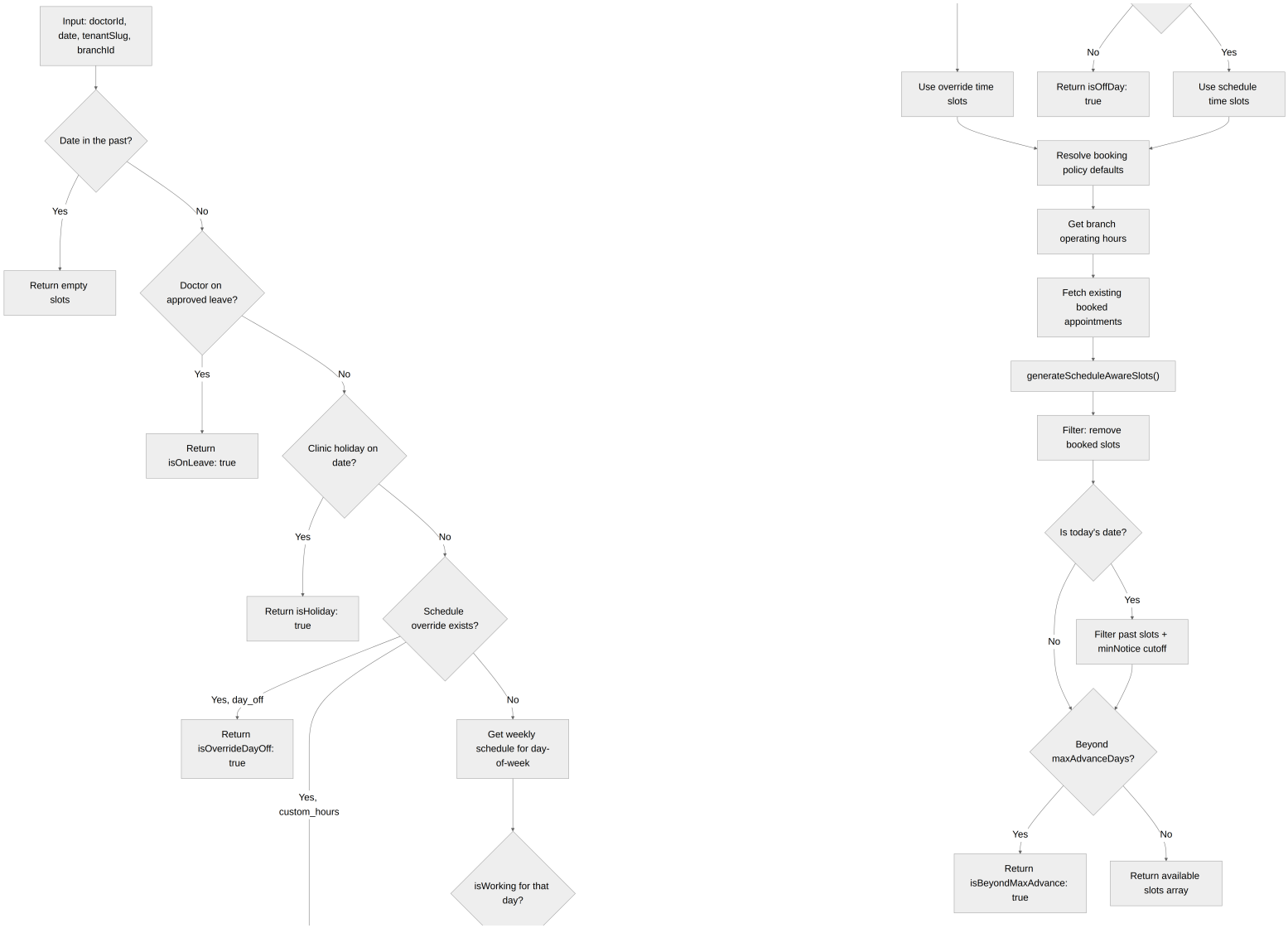


Diagram 16: Availability Calculation Algorithm

Pseudocode: 1. Input: `doctorId`, `date`, `tenantSlug`, `branchId` 2. If date is in the past → return empty 3. Query `DoctorLeave` for approved leave overlapping date → if found, return `isOnLeave` 4. Query `ClinicHoliday` for active holiday on date → if found, return `isHoliday` 5. Query `DoctorScheduleOverride` for date → if `day_off`, return unavailable; if `custom_hours`, use override slots 6. If no override, get `DoctorSchedule.weeklySchedule[dayOfWeek]` → if `!isWorking`, return `isOffDay` 7. Resolve booking policy: `DoctorSchedule` → `ClinicBookingPolicy` → system defaults (slotDuration, bufferMinutes, minNoticeMinutes, maxAdvanceDays) 8. Get branch operating hours → intersect with doctor hours 9. Query existing `Appointment` records for date (exclude cancelled/no_show) 10. Generate slots: iterate from working start to end at `slotDuration + bufferMinutes` step 11. Mark slots as unavailable where they overlap with booked appointments 12. If today: filter slots before `now + minNoticeMinutes` 13. If beyond `maxAdvanceDays`: return `isBeyondMaxAdvance` 14. Return available slots array

`Code Location: src/services/availability.service.js`

7.2.2 Invoice Calculation with GST

Pseudocode: 1. For each line item: `lineSubtotal = quantity × unitPrice` 2. Apply item-level discount: `lineDiscount = item.discount || 0` 3. Calculate item tax: `lineTax = item.tax || 0` 4. Line total: `lineTotal = lineSubtotal - lineDiscount + lineTax` 5. Aggregate: `subtotal = Σ(lineSubtotal)`, `discountAmount = Σ(lineDiscount)`, `taxAmount = Σ(lineTax)` 6. Invoice total: `totalAmount = subtotal - discountAmount + taxAmount` 7. GST split (intrastate): `cgst = lineTax / 2`, `sgst = lineTax - cgst` 8. GST split (interstate): `igst = lineTax`

`Code Location: src/modules/billing/billing.service.js`

7.2.3 Cash Register Variance Detection

Pseudocode: 1. On register close: receive `closingBalance` from user 2. Calculate `expectedBalance = openingBalance + totalCashIn - totalCashOut` 3. Calculate `variance = closingBalance - expectedBalance` 4. If `Math.abs(variance) > 100` (INR): set `managerApprovalRequired = true` 5. Atomic CAS update: `findOneAndUpdate({ _id, status: 'open' }, { status: 'closed', closingBalance, variance })` 6. Create `FinancialAuditLog` entry with before/after snapshots

`Code Location: src/modules/cashRegister/cashRegister.service.js`

7.2.4 Appointment Conflict Detection

Pseudocode: 1. Input: `doctorId`, `scheduledAt`, `duration` 2. Calculate `endTime = scheduledAt + duration` 3. Query: `Appointment.find({ doctor: doctorId, status: { $nin: ['cancelled', 'no_show'] }, scheduledAt: { $lt: endTime }, endTime: { $gt: scheduledAt } })` 4. If any result → reject with 409 Conflict

`Code Location: src/modules/appointment/appointment.service.js`

7.2.5 Subscription State Machine

Transitions: | Current State | Valid Next States | Trigger | |—|—|—| | trial | active, cancelled | Trial expires, user cancels | | active | grace_period, cancelled | Payment fails, user cancels | | grace_period | active, suspended, cancelled | Payment recovered, 7-day grace expires, user cancels | | suspended | active, cancelled | Reactivation with payment, admin cancels | | cancelled | (terminal) | — |

`Code Location: src/modules/subscription/subscription.service.js`

7.2.6 Tenant Connection Pool Management

Pseudocode: 1. `getTenantConnection(slug)`: a. If pool has entry for slug: update `lastUsed`, return connection b. If another call is already connecting (dedup): await that promise c. If

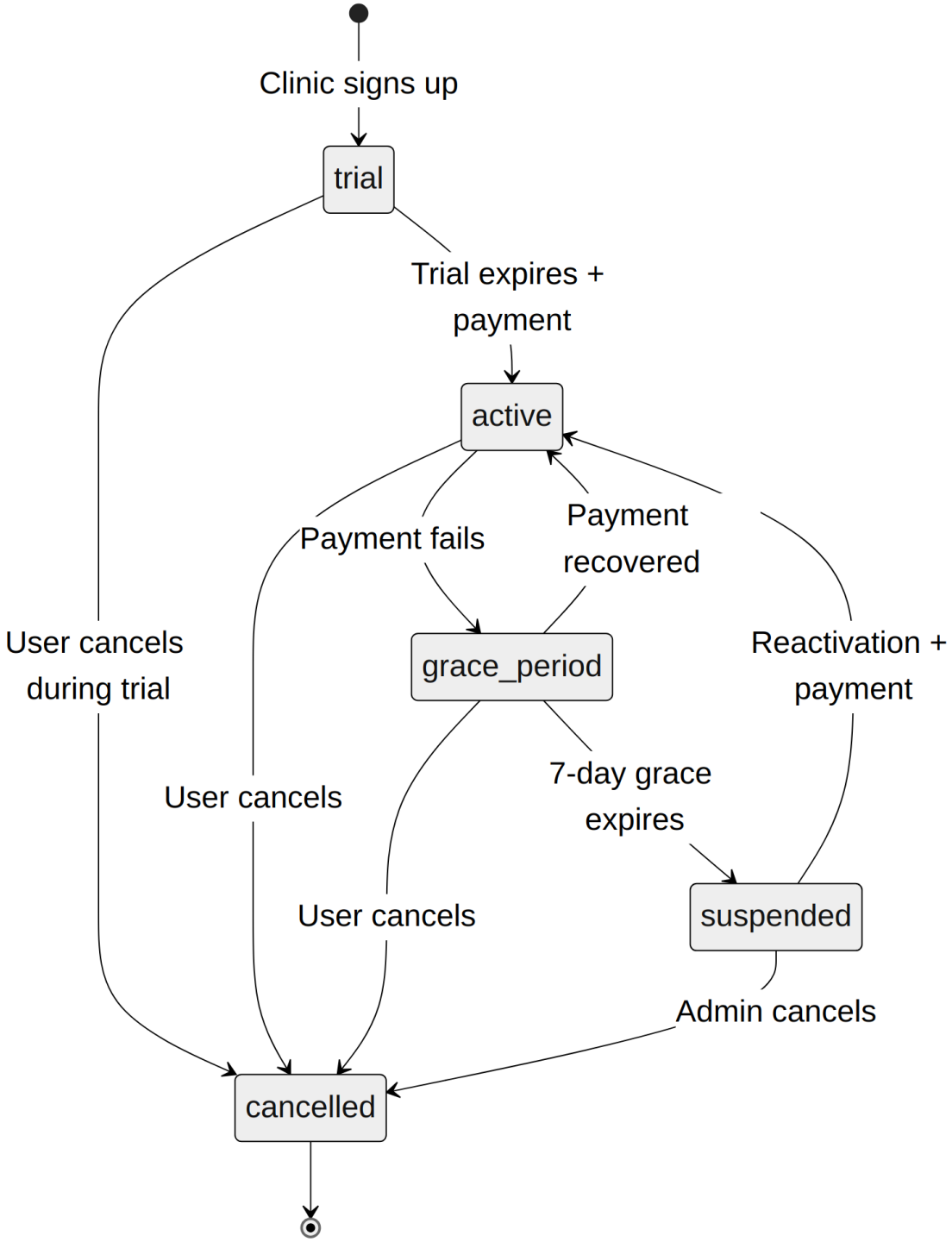


Figure 4: Diagram 8: Subscription Lifecycle

pool is at MAX_TOTAL_CONNECTIONS: evict LRU entry (oldest `lastUsed`) d. Resolve URI: check Redis cache `tenant:dbUri:{slug}` → decrypt AES-256-GCM; if miss → query `Clinic.findOne({ slug })` → encrypt → cache e. Create `mongoose.createConnection(uri, { maxPoolSize, serverSelectionTimeoutMS: 5000 })` f. Register all 53 tenant models on connection g. Add to pool with `lastUsed: Date.now()`, `circuitOpen: false` h. On error: open circuit breaker for slug 2. `pruneIdleConnections()` (every 10 min): close connections idle > 30 min

`Code Location: src/db/tenantConnectionManager.js`

7.2.7 AI Dental Analysis JSON Repair

Pseudocode: 1. Receive raw text from Gemini API 2. Try 1: `JSON.parse(text)` directly 3. Try 2: Extract content between ```` ```json ```` and ```` ``` ```` markers 4. Try 2b: Handle unclosed code block (truncation tolerance) 5. Try 3: Find first `{` ... last `}` and parse 6. Try 4: Repair truncated JSON — close open strings, remove trailing commas, build bracket closing sequence 7. Try 5: Strip to last comma, close brackets aggressively 8. If all fail: return `{ findings: [], summary: 'Analysis could not be parsed' }`

`Code Location: src/services/aiAnalysis.service.js`

7.2.8 Appointment Reminder Scheduling

Pseudocode: 1. On appointment creation: `scheduleReminders(appointmentId, tenantSlug, scheduledAt, timings)` 2. Default timings: `[1440, 60, 15]` (24h, 1h, 15min before) 3. For each timing: `fireAt = scheduledAt - (minutesBefore × 60 × 1000)` 4. If `fireAt` is past (delay < -60s): skip 5. Create BullMQ delayed job with ID `reminder-{appointmentId}-{minutesBefore}` 6. Worker processes: load appointment → check not cancelled → build message → create Notification → send FCM push

`Code Location: src/utils/reminderScheduler.js, src/jobs/workers/appointmentReminderWorker.js`

---

8. Database Schema Architecture

8.1 Schema Specifications

8.1.1 Platform Models

Collection: `clinics` Purpose: Master record for each tenant clinic. Key Fields: - `name` (String, required, trim) - `slug` (String, required, unique, lowercase) — tenant identifier - `email` (String, required) - `phone` (String) - `address` (Object: street, city, state, pincode, country) - `logo` (String) — URL - `status` (String, enum: pending/active/suspended/cancelled, default: pending) - `features` (Object) — plan-derived feature flags (maxDoctors, maxStaff, maxBranches, maxStorageMB, smsEnabled, teleHealthEnabled, customBranding, apiAccess, advancedReports, whiteLabeling, jobPortalEnabled, prioritySupport, externalIntegrations) - `usage` (Object) — current counters (doctorsCount, staffCount, branchCount, storageMB) - `owner` (Object: name, email, phone) - `dbUri` (String) — encrypted tenant database URI - `settings` (Object) — clinic-level preferences - `deletedAt` (Date, default: null) Indexes: `{ slug: 1 }` (unique), `{ status: 1 }`, `{ deletedAt: 1 }`

Collection: `subscriptions` Purpose: Subscription lifecycle per clinic. Key Fields: - `clinic` (ObjectId, ref: Clinic, required) - `clinicSlug` (String, required) - `plan` (ObjectId, ref: Plan, required) - `planSlug` (String, required) - `status` (String, enum: trial/active/grace_period/cancelled/suspended) - `currentPeriodStart` (Date) - `currentPeriodEnd` (Date) - `trialEnd` (Date) - `cancelledAt` (Date) - `suspendedAt` (Date) - `razorpaySubscriptionId` (String) - `pricing` (Object: basePrice, discount, finalPrice, currency) - `promotion` (Object: code, discountPercent, cyclesRemaining) Indexes: `{ clinic: 1 }`, `{ clinicSlug: 1, status: 1 }`, `{ status: 1 }`

Collection: `plans` Purpose: Subscription plan definitions with feature limits and pricing. Key Fields: - `name` (String, required) - `slug` (String, required, unique) - `description` (String) - `tier` (Number) — for plan comparison ordering - `pricing` (Object: monthly, yearly, currency) - `trialDays` (Number, default: 14) - `features` (Object) — same shape as Clinic.features (maxDoctors, maxStaff, etc.) - `isActive` (Boolean, default: true) - `isDefault` (Boolean, default: false) Indexes: `{ slug: 1 }` (unique), `{ isActive: 1, tier: 1 }`

Collection: `platformusers` Purpose: Platform-level admin accounts (product owners). Key Fields: - `name` (String, required) - `email` (String, required, unique) - `password` (String, required) — bcrypt hashed - `role` (String, default: product_owner) - `isActive` (Boolean, default: true) - `lastLoginAt` (Date)

Collection: `provisioningstatuses` Purpose: Tracks tenant provisioning pipeline state. Key Fields: - `clinic` (ObjectId, ref: Clinic, required) - `clinicSlug` (String, required) - `status` (String, enum: pending/creating_db/generating_clinic_code/seeding/creating_admin/applying_plan/completed/fail
- `stepDetails` (Object) — per-step metadata and timestamps - `error` (String) — failure reason - `startedAt` (Date) - `completedAt` (Date) Indexes: `{ clinic: 1 }` (unique), `{ status: 1 }`

Collection: `userdirectories` Purpose: Cross-tenant email-to-tenant lookup index. Key Fields: - `email` (String, required, lowercase) - `tenantSlug` (String, required) - `userId` (String,

required) - `role` (String) - `name` (String) Indexes: `{ email: 1, tenantSlug: 1 }` (unique), `{ email: 1 }`

Collection: **`featureflags`** Purpose: Dynamic feature toggles with tenant and plan overrides. Key Fields: - `key` (String, required, unique) - `description` (String) - `defaultValue` (Boolean, default: false) - `tenantOverrides` (Map of slug → Boolean) - `planDefaults` (Map of planSlug → Boolean) - `isActive` (Boolean, default: true) Method: `resolve(tenantSlug, planSlug)` — returns tenant override → plan default → global default Indexes: `{ key: 1 }` (unique)

Collection: **`integrations`** Purpose: Third-party integration configuration per clinic. Key Fields: - `clinic` (ObjectId, ref: Clinic, required) - `clinicSlug` (String, required) - `type` (String, enum: rioview/romexis, required) - `status` (String, enum: inactive/active/error/suspended) - `config` (Object: baseUrl, apiVersion, syncInterval, syncEnabled, lastSyncAt, lastSyncStatus, lastSyncRecordCount) - `enabledBy` (Object: userId, role) - `deletedAt` (Date, default: null) Indexes: `{ clinicSlug: 1, type: 1 }` (unique), `{ status: 1 }`, `{ deletedAt: 1 }`

Collection: **`promotions`** Purpose: Discount codes and campaigns for subscriptions. Key Fields: - `code` (String, required, unique, uppercase) - `description` (String) - `discountType` (String, enum: percentage/fixed) - `discountValue` (Number, required) - `maxUses` (Number) - `usedCount` (Number, default: 0) - `validFrom` (Date) - `validUntil` (Date) - `applicablePlans` (Array of plan slugs) - `isActive` (Boolean, default: true)

Collection: **`rebates`** Purpose: Rebate tracking for subscription payments. Key Fields: - `clinic` (ObjectId, ref: Clinic) - `amount` (Number, required) - `reason` (String) - `status` (String, enum: pending/applied/expired) - `appliedAt` (Date)

Collection: **`creditwallets`** Purpose: Platform credit balance per clinic (for job portal). Key Fields: - `clinic` (ObjectId, ref: Clinic, required, unique) - `balance` (Number, default: 0, min: 0) - `totalCreditsEarned` (Number, default: 0) - `totalCreditsSpent` (Number, default: 0) Indexes: `{ clinic: 1 }` (unique)

Collection: **`credittransactions`** Purpose: Immutable audit log of credit movements. Key Fields: - `wallet` (ObjectId, ref: CreditWallet, required) - `clinic` (ObjectId, ref: Clinic, required) - `type` (String, enum from CREDIT_TRANSACTION_TYPE) - `amount` (Number, required) — positive=in, negative=out - `balanceAfter` (Number, required) — running balance snapshot - `action` (String, enum from CREDIT_ACTION) - `reference` (Object: jobPosting, razorpayPaymentId, razorpayOrderId) - `description` (String, maxlength: 500) - `performedBy` (String) — userId or 'system' Indexes: `{ clinic: 1, createdAt: -1 }`, `{ wallet: 1, type: 1 }`, `{ reference.razorpayPaymentId: 1 }` (sparse)

Collection: **`webhookevents`** Purpose: Razorpay webhook event log for idempotency. Key Fields: - `eventId` (String, required, unique) - `eventType` (String, required) - `payload` (Object) - `processedAt` (Date) - `status` (String, enum: received/processed/failed)

Collection: **`jobpostings`** Purpose: Clinic job listings for recruitment. Key Fields: - `clinic` (ObjectId, ref: Clinic, required) - `clinicSlug` (String, required) - `title` (String, required, trim) - `department` (String) - `type` (String, enum: full_time/part_time/contract/internship) - `experience` (Object: min, max) - `compensation` (Object: salaryMin, salaryMax, currency, period) - `location` (Object with GeoJSON Point: type, coordinates) - `description` (String) - `requirements` (Array of String) - `status` (String, enum: draft/published/closed/expired) - `publishedAt` (Date) - `expiresAt` (Date) - `applicationCount` (Number, default: 0) - `deletedAt` (Date, default: null)

Indexes: `{ clinicSlug: 1, status: 1 }`, `{ status: 1, expiresAt: 1 }`, `{ location: '2dsphere' }`, `{ deletedAt: 1 }`

Collection: **`jobapplications`** Purpose: Applications submitted for job postings. Key Fields: - `job` (ObjectId, ref: JobPosting, required) - `applicantName` (String, required) - `email` (String, required) - `phone` (String) - `resumeUrl` (String) - `coverLetter` (String) - `status` (String, enum: received/shortlisted/interviewed/offered/rejected/withdrawn) - `notes` (String) Indexes: `{ job: 1, email: 1 }` (unique), `{ status: 1 }`

Collection: **`featureddoctors`** Purpose: Doctor profiles for public discovery/marketplace. Key Fields: - `clinicId` (ObjectId, ref: Clinic, required) - `clinicSlug` (String, required) - `clinicName` (String, required) - `tenantDoctorId` (String, required) - `name` (String, required, trim) - `specializations` (Array of String) - `bio` (String, maxlength: 1000) - `education` (Array of {degree, institution, year}) - `experienceYears` (Number) - `consultationFee` (Number) - `averageRating` (Number, 0-5, default: 0) - `totalReviews` (Number, default: 0) - `location` (GeoJSON Point) - `isActive` (Boolean, default: true) - `deletedAt` (Date, default: null) Indexes: `{ clinicSlug: 1, tenantDoctorId: 1 }` (unique), `{ specializations: 1, isActive: 1 }`, `{ isActive: 1, averageRating: -1 }`, `{ clinicCity: 1, isActive: 1 }`, `{ location: '2dsphere' }`, `{ deletedAt: 1 }`

Collection: **`guestscans`** Purpose: AI dental scans by unauthenticated public users. Key Fields: - `sessionToken` (String, unique, sparse) - `status` (String, enum: pending/completed/failed) - `images` (Array of {angle, fileUrl, mimeType}) - `aiAnalysis` (Object: findings, summary, confidence, modelVersion, processingMs) - `source` (String, enum: website/mobile_app) - `expiresAt` (Date) — TTL index, 30-day auto-cleanup Indexes: `{ sessionToken: 1 }`, `{ status: 1 }`, `{ expiresAt: 1 }` (TTL)

Collection: **`prescriptionscans`** Purpose: AI prescription OCR and transcription. Key Fields: - `sessionToken` (String, unique, sparse) - `userId` (ObjectId, sparse) - `status` (String, enum: pending/completed/failed) - `image` (Object: fileUrl, fileName, fileSize, mimeType) - `result` (Object: transcription, detectedLanguages, medications[{name, dosage, frequency, duration, notes}], patientName, doctorName, confidence, imageQuality) - `translation` (Object: translatedText, targetLanguage, sourceLanguages, translatedAt) - `expiresAt` (Date) — TTL index, 30-day auto-cleanup Indexes: `{ sessionToken: 1 }`, `{ userId: 1 }`, `{ status: 1 }`, `{ createdAt: -1 }`, `{ expiresAt: 1 }` (TTL)

Collection: **`platformpatients`** Purpose: Cross-clinic patient identity. Key Fields: - `name` (String, required) - `email` (String, required, unique, lowercase) - `phone` (String) - `password` (String) — bcrypt hashed (12 rounds) - `connectedClinics` (Array of {clinicId, clinicSlug, clinicName, joinedAt}) - `lastLoginAt` (Date) Indexes: `{ email: 1 }` (unique)

Collection: **`calendarfeedtokens`** Purpose: Cross-tenant token index for public ICS calendar feeds. Key Fields: - `token` (String, required, unique) — UUID in ICS URL - `tenantSlug` (String, required) - `feedType` (String, enum: doctor/patient) - `isEnabled` (Boolean, default: true) Indexes: `{ token: 1 }` (unique), `{ tenantSlug: 1 }`

Collection: **`tenantauditlogs`** Purpose: Platform-level audit log for tenant operations. Key Fields: - `clinicSlug` (String, required) - `action` (String, required) - `performedBy` (String) - `details` (Object) - `timestamp` (Date) Indexes: `{ clinicSlug: 1, timestamp: -1 }`

8.1.2 Tenant Models — Clinical

Collection: `users` Purpose: Clinic staff and patient user accounts. Key Fields: - `name` (String, required, trim) - `email` (String, required, lowercase) - `phone` (String) - `password` (String) — bcrypt hashed (12 rounds), pre-save hook - `role` (String, required, enum: all 14 role values) - `branch` (ObjectId, ref: Branch) - `department` (ObjectId, ref: Department) - `isActive` (Boolean, default: true) - `avatar` (String) - `doctorProfile` (Object: specializations[], bio, education[], experienceYears, qualifications[], languages[], consultationFee, acceptingNewPatients) - `lastLoginAt` (Date) - `passwordResetToken` (String) - `passwordResetExpires` (Date) - `fcmTokens` (Array of String) — push notification device tokens - `deletedAt` (Date, default: null) Indexes: `{ email: 1 }` (unique), `{ role: 1 }`, `{ branch: 1 }`, `{ department: 1 }`, `{ deletedAt: 1 }`

Collection: `patients` Purpose: Patient master records. Key Fields: - `user` (ObjectId, ref: User) — linked user account - `patientId` (String) — human-readable ID (auto-generated) - `name` (String, required) - `email` (String) - `phone` (String) - `dateOfBirth` (Date) - `gender` (String, enum: male/female/other) - `address` (Object: street, city, state, pincode) - `bloodGroup` (String) - `allergies` (Array of String) - `medicalHistory` (String) - `primaryDoctor` (ObjectId, ref: User) - `guardian` (ObjectId, ref: User) — guardian link - `insuranceProvider` (ObjectId, ref: InsuranceProvider) - `insurancePolicyNumber` (String) - `isActive` (Boolean, default: true) - `deletedAt` (Date, default: null) Indexes: `{ user: 1 }`, `{ patientId: 1 }` (unique, sparse), `{ email: 1 }`, `{ phone: 1 }`, `{ primaryDoctor: 1 }`, `{ deletedAt: 1 }`

Collection: `appointments` Purpose: Appointment scheduling records. Key Fields: - `patient` (ObjectId, ref: Patient, required) - `doctor` (ObjectId, ref: User, required) - `branch` (ObjectId, ref: Branch) - `department` (ObjectId, ref: Department) - `scheduledAt` (Date, required) - `duration` (Number, default: 30) — minutes - `type` (String, enum: consultation/follow_up/review/check_up/emergency) - `status` (String, enum: requested/confirmed/checked_in/in_progress/completed/cancelled/no_show) - `cancellationReason` (String) - `notes` (String) - `reminders` (Array of {minutesBefore, jobId, sentAt}) - `checkedInAt` (Date) - `source` (String, enum: portal/receptionist/doctor/admin) - `payment` (Object: required, amount, status, razorpayOrderId) - `deletedAt` (Date, default: null) Indexes: `{ doctor: 1, scheduledAt: 1 }`, `{ patient: 1, scheduledAt: -1 }`, `{ status: 1 }`, `{ branch: 1, scheduledAt: 1 }`, `{ deletedAt: 1 }`

Collection: `treatmentrecords` Purpose: Consultation/treatment documentation (SOAP notes). Key Fields: - `appointment` (ObjectId, ref: Appointment, required) - `patient` (ObjectId, ref: Patient, required) - `doctor` (ObjectId, ref: User, required) - `visitType` (String) - `chiefComplaint` (String) - `diagnosis` (String) - `treatmentPlan` (String) - `subjective` (String) - `objective` (String) - `assessment` (String) - `plan` (String) - `vitalSigns` (Object: bloodPressure, pulse, temperature, weight) - `teethInvolved` (Array of Number) — FDI notation - `attachments` (Array of {fileName, fileUrl, mimeType}) - `signature` (String) — base64 doctor signature - `followUpDate` (Date) - `followUpInstructions` (String) - `status` (String, enum: in_progress/completed/draft) - `materialsUsed` (Array of {item: ObjectId ref InventoryItem, itemName, quantity, costPerUnit}) Indexes: `{ appointment: 1 }` (unique), `{ patient: 1, createdAt: -1 }`, `{ doctor: 1, createdAt: -1 }`

Collection: `prescriptions` Purpose: Medication prescriptions linked to consultations. Key Fields: - `patient` (ObjectId, ref: Patient, required) - `doctor` (ObjectId, ref: User, required) - `treatmentRecord` (ObjectId, ref: TreatmentRecord) - `medications` (Array of {name, dosage, frequency, duration, instructions, quantity}) - `status` (String, enum: active/completed/cancelled) - `notes` (String) Indexes: `{ patient: 1, createdAt: -1 }`, `{ doctor: 1 }`, `{ treatmentRecord: 1 }`

Collection: `prescriptiontasks` Purpose: Prescription reminder/task tracking for patients. Key Fields: - `prescription` (ObjectId, ref: Prescription, required) - `patient` (ObjectId, ref: Patient, required) - `medication` (Object: name, dosage, frequency) - `remindersEnabled` (Boolean, default: false) - `nextReminderAt` (Date) - `status` (String, enum: active/paused/completed)

Collection: `dentalscans` Purpose: Dental scan sessions with image captures. Key Fields: - `patient` (ObjectId, ref: Patient) - `user` (ObjectId, ref: User) — who initiated - `scanMode` (String) — e.g., guided_3pos, panoramic - `status` (String, enum: capturing/processing/completed/failed) - `images` (Array of {angle, fileUrl, mimeType, uploadedAt}) - `consent` (Object: givenAt, ipAddress) - `aiAnalysis` (Object: findings[], summary, confidence, modelVersion, processingMs) - `doctorReview` (Object: reviewedBy, reviewedAt, notes, approved) Indexes: `{ patient: 1, createdAt: -1 }`, `{ user: 1 }`, `{ status: 1 }`

Collection: `dentalscanresults` Purpose: Detailed AI analysis results for dental scans. Key Fields: - `scan` (ObjectId, ref: DentalScan, required) - `findings` (Array of {type, location, severity, description, specialization, recommendations[]}) - `summary` (String) - `confidence` (Number, 0-1) - `modelVersion` (String) - `processingMs` (Number) Indexes: `{ scan: 1 }` (unique)

Collection: `consents` Purpose: Patient consent records for procedures. Key Fields: - `patient` (ObjectId, ref: Patient, required) - `type` (String, required) — consent category - `description` (String) - `status` (String, enum: pending/approved/revoked) - `approvedAt` (Date) - `revokedAt` (Date) - `signature` (String) — base64

Collection: `guardianconsents` Purpose: Parental/guardian consent for minor patients. Key Fields: - `patient` (ObjectId, ref: Patient, required) - `guardian` (ObjectId, ref: User, required) - `consentType` (String) - `status` (String, enum: pending/approved/revoked) - `approvedAt` (Date)

Collection: `consultationrooms` Purpose: Physical consultation room definitions. Key Fields: - `name` (String, required) - `branch` (ObjectId, ref: Branch) - `capacity` (Number, default: 1) - `isActive` (Boolean, default: true) - `equipment` (Array of String)

Collection: `consultationtemplates` Purpose: Reusable consultation form templates. Key Fields: - `name` (String, required) - `code` (String, required) - `category` (String, enum: surgery/orthodontic/prosthetic/endodontic/periodontal/cosmetic/diagnostic/other) - `chiefComplaint` (String) - `diagnosis` (String) - `treatment` (String) - `followUpInstructions` (String) - `defaultProcedure` (String) - `defaultPriority` (String) - `defaultPrescriptions` (Array of medication objects) - `defaultCareTasks` (Array of care task objects) - `defaultMaterials` (Array of {item, itemName, quantity, costPerUnit}) - `createdBy` (ObjectId, ref: User) - `isActive` (Boolean, default: true) Indexes: `{ code: 1 }` (unique), `{ category: 1 }`, `{ createdBy: 1 }`

Collection: `emergencysessions` Purpose: Emergency consultation sessions. Key Fields: - `patient` (ObjectId, ref: Patient, required) - `doctor` (ObjectId, ref: User) - `status` (String, enum: requested/accepted/in_progress/ended/cancelled/declined) - `chiefComplaint` (String, required) - `severity` (String, enum: low/medium/high/critical) - `acceptedAt` (Date) - `endedAt` (Date) - `notes` (String) Indexes: `{ patient: 1, status: 1 }`, `{ doctor: 1, status: 1 }`

Collection: `carereminders` Purpose: Post-treatment care task reminders for patients. Key Fields: - `patient` (ObjectId, ref: Patient, required) - `treatmentRecord` (ObjectId, ref: TreatmentRecord) - `taskType` (String, enum: medication/hygiene/post_care/follow_up/dietary/custom) - `title` (String, required) - `description` (String) - `frequency` (String, enum: once/once_daily/twice_daily/etc.) - `startDate` (Date) - `endDate` (Date) - `enabled` (Boolean, default: true) - `reminderEnabled`

(Boolean, default: true) - `completedCount` (Number, default: 0) - `lastCompletedAt` (Date) Indexes: `{ patient: 1, enabled: 1 }`, `{ treatmentRecord: 1 }`

8.1.3 Tenant Models — Scheduling

Collection: **`doctorschedules`** — Weekly schedule templates for doctors. Key Fields: `doctor` (ObjectId, ref: User), `branch` (ObjectId, ref: Branch), `weeklySchedule` (Array of {day, isWorking, slots[{startTime, endTime}], startTime, endTime, breakStart, breakEnd}), `slotDuration`, `bufferMinutes`, `isActive`

Collection: **`doctorscheduleoverrides`** — Per-date schedule exceptions. Key Fields: `doctor`, `date`, `branch`, `type` (enum: day_off/custom_hours/extra_day), `slots`, `startTime`, `endTime`, `reason`

Collection: **`doctorleaves`** — Leave requests. Key Fields: `doctor`, `startDate`, `endDate`, `status` (enum: pending/approved/rejected), `reason`, `branch`

Collection: **`queueentries`** — Patient queue management. Key Fields: `patient`, `doctor`, `appointment`, `branch`, `tokenNumber`, `priority` (enum: emergency/high/normal), `status` (enum: waiting/called/in_consultation/completed/cancelled/no_show), `calledAt`, `completedAt`, `room`, `sortOrder`

Collection: **`calendarfeeds`** — ICS calendar feed configuration. Key Fields: `user`, `token` (UUID), `feedType`, `filters`, `isEnabled`

Collection: **`clinicbookingpolicies`** — Clinic-level booking defaults. Key Fields: `branch`, `slotDuration` (default: 30), `bufferMinutes` (default: 10), `minNoticeMinutes` (default: 60), `maxAdvanceDays` (default: 90), `autoConfirm`, `defaultReminderTimings`, `appointmentTypeDurations`

Collection: **`clinicholidays`** — Holiday calendar. Key Fields: `name`, `startDate`, `endDate`, `branches` (Array of ObjectId), `isActive`, `deletedAt`

8.1.4 Tenant Models — Financial

Collection: **`invoices`** — Billing invoices. Key Fields: `invoiceNumber`, `patient`, `doctor`, `appointment`, `branch`, `lineItems` (Array of {description, quantity, unitPrice, discount, tax, cgst, sgst, igst, total, serviceItem}), `subtotal`, `discountAmount`, `taxAmount`, `totalAmount`, `paidAmount`, `balanceDue`, `status` (enum: draft/sent/paid/partially_paid/overdue/cancelled/refunded), `dueDate`, `gstNumber`, `deletedAt`

Collection: **`payments`** — Payment records. Key Fields: `invoice`, `patient`, `amount`, `method` (enum: cash/card/upi/bank_transfer/cheque/online), `status` (enum: pending/completed/failed/refunded), `razorpayPaymentId`, `razorpayOrderId`, `receiptNumber`, `refundAmount`, `refundedAt`, `cashRegister`

Collection: **`creditnotes`** — Credit memos. Key Fields: `creditNoteNumber`, `patient`, `invoice`, `amount`, `reason`, `status` (enum: draft/issued/applied/cancelled), `issuedAt`, `appliedAt`, `appliedToInvoice`

Collection: **`patientledgers`** — Patient account ledger. Key Fields: `patient`, `type` (enum: invoice/payment/credit_note/advance/adjustment/write_off), `amount`, `runningBalance`, `reference` (Object: invoice, payment, creditNote), `description`, `performedBy`

Collection: **`financialauditlogs`** — Financial operation audit trail. Key Fields: `action`, `entityType`, `entityId`, `before` (Object), `after` (Object), `performedBy`, `reason`, `branch`

Collection: **`cashregisters`** — Daily cash drawer management. Key Fields: `branch`, `openedBy`, `closedBy`, `openingBalance`, `closingBalance`, `totalCashIn`, `totalCashOut`, `expectedBalance`, `variance`, `status` (enum: open/closed/reconciled), `managerApprovalRequired`, `openedAt`, `closedAt`, `notes`

Collection: **`serviceitems`** — Billable service catalog. Key Fields: `name`, `code`, `category`, `unitPrice`, `taxRate`, `hsnCode`, `isActive`, `description`

8.1.5 Tenant Models — Inventory

Collection: `inventoryitems` — Stock items. Key Fields: `name`, `sku`, `category`, `currentStock`, `minimumThreshold`, `unit`, `costPerUnit`, `sellingPrice`, `supplier`, `expiryDate`, `branch`, `isActive`, `deletedAt`

Collection: `inventorytransactions` — Stock movement log. Key Fields: `item` (ObjectId, ref: InventoryItem), `type` (enum: stock_in/stock_out/adjustment/damage/expiry), `quantity`, `previousStock`, `newStock`, `reason`, `performedBy`, `reference` (Object: purchaseOrder, treatmentRecord)

Collection: `purchaseorders` — Procurement orders. Key Fields: `poNumber`, `vendor` (Object: name, contact, email), `items` (Array of {inventoryItem, name, quantity, unitPrice, total}), `subtotal`, `taxAmount`, `totalAmount`, `status` (enum: draft/submitted/approved/received/cancelled), `expectedDelivery`, `receivedAt`, `notes`, `createdBy`

8.1.6 Tenant Models — Organization

Collection: `branches` — Clinic locations. Key Fields: `name`, `code`, `address`, `phone`, `email`, `operatingHours` (Array of {day, openTime, closeTime, isClosed}), `isActive`, `deletedAt`

Collection: `departments` — Organizational departments. Key Fields: `name`, `code`, `branch`, `description`, `isActive`, `deletedAt`

Collection: `roles` — Custom role definitions (tenant-level). Key Fields: `name`, `slug`, `description`, `permissions` (Array of String), `isSystem` (Boolean), `isActive`

Collection: `staff` — Staff records (HR data). Key Fields: `user`, `employeeId`, `designation`, `joiningDate`, `branch`, `department`, `salary`, `emergencyContact`

Collection: `salarystructures` — Payroll templates. Key Fields: `name`, `components` (Array of {name, type: earning/deduction, amount, isPercentage, percentageOf}), `isActive`

Collection: `payslips` — Generated payroll slips. Key Fields: `staff`, `month`, `year`, `salaryStructure`, `earnings`, `deductions`, `grossPay`, `netPay`, `status` (enum: draft/approved/paid), `approvedBy`, `paidAt`

8.1.7 Tenant Models — Communication, Integration, Security, Insurance

Collection: `notifications` — In-app notification records. Key Fields: `recipient` (ObjectId, ref: User), `type`, `title`, `message`, `data` (Object), `isRead`, `readAt`

Collection: `chatmessages` — Telemedicine chat messages. Key Fields: `session` (ObjectId), `sender`, `senderRole`, `content`, `type` (enum: text/file/system), `fileUrl`, `readBy`

Collection: `reviews` — Patient reviews of doctors/clinic. Key Fields: `patient`, `doctor`, `rating` (Number, 1-5), `comment`, `isApproved`

Collection: `integrationcredentials` — Encrypted third-party API credentials. Key Fields: `type`, `credentials` (Object, encrypted), `isActive`, `lastTestedAt`

Collection: `synclogs` — Integration sync operation logs. Key Fields: `integrationType`, `direction` (enum: inbound/outbound), `status`, `recordCount`, `errors`, `startedAt`, `completedAt`

Collection: `legacyuploads` — Data migration/import history. Key Fields: `type`, `fileName`, `fileUrl`, `status`, `recordCount`, `processedCount`, `errors`, `uploadedBy`

Collection: `accesslogs` — Patient record access audit. Key Fields: `patient`, `accessedBy`, `action`, `ipAddress`, `userAgent`

Collection: `sharetokens` — Shareable record access tokens. Key Fields: `patient`, `token` (UUID), `expiresAt`, `accessedCount`, `maxAccess`

Collection: `compliancealerts` — Compliance notification records. Key Fields: `type`, `severity`, `message`, `patient`, `resolvedAt`, `resolvedBy`

Collection: `dependentarchives` — Guardian-dependent relationship archives. Key Fields: `guardian`, `dependent`, `relationship`, `detachedAt`, `reason`

Collection: `insuranceproviders` — Insurance company master data. Key Fields: `name`, `code`, `contactPerson`, `phone`, `email`, `address`, `plans` (Array), `isActive`

Collection: `insuranceclaims` — Insurance claim records. Key Fields: `patient`, `provider` (ObjectId, ref: InsuranceProvider), `invoice`, `claimNumber`, `amount`, `status` (enum: pending/submitted/under_review/approved/rejected/settled/appealed), `preAuthStatus`, `preAuthAmount`, `coPayAmount`, `settlementAmount`, `submittedAt`, `settledAt`

8.2 Schema Relationships Summary

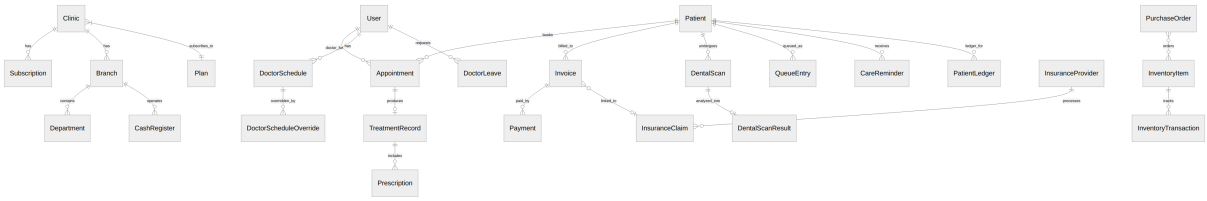


Figure 5: Diagram 6: Database Schema Relationships (ER Diagram)

Relationship Types:

Type	Examples
One-to-One	Appointment → TreatmentRecord, DentalScan → DentalScanResult
One-to-Many	Clinic → Branches, Patient → Appointments, Invoice → Payments, Doctor → DoctorSchedules
Many-to-Many	PurchaseOrder ↔ InventoryItems (via line items), Patient ↔ InsuranceProviders (via claims)
Self-referential	User (guardian) → User (dependent patient)
Cross-database	PlatformPatient ↔ User (via email matching), CalendarFeedToken (platform) → CalendarFeed (tenant)

Part III — Core Features & User Journeys

Chapter 9: Core Module Deep Dive

9.1 Patient Module

Location: `src/modules/patient/`

The Patient module provides full lifecycle management for patient records, from creation and bulk import through soft-deletion with referential cascade cleanup.

API Endpoints:

Method	Path	Permission	Description
GET	`/patients`	PATIENT_READ / PATIENT_READ_OWN	Paginated patient list with search
GET	`/patients/:id`	PATIENT_READ / PATIENT_READ_OWN	Patient detail with populated references
POST	`/patients`	PATIENT_CREATE	Create new patient record
POST	`/patients/bulk`	PATIENT_CREATE	Bulk CSV import
PUT	`/patients/:id`	PATIENT_UPDATE / PATIENT_UPDATE_OWN	Update patient record
DELETE	`/patients/:id`	PATIENT_DELETE	Soft-delete with cascade

Key Service Methods:

1. **`listPatients(query, user, tenantSlug)`** — Paginated listing with search across name, email, phone, and patientId. Filters by gender and active status. Self-scope enforcement ensures non-admin users see only their own or related records.
2. **`getPatientById(id, user, tenantSlug)`** — Retrieves patient with populated `primaryDoctor` and `guardian` references. Creates an access log entry for compliance audit. Enforces self-scope checks.
3. **`createPatient(data, user, tenantSlug)`** — Creates a new patient record with access logging. Auto-generates `patientId` if not provided.
4. **`deletePatient(id, user, tenantSlug)`** — Soft-deletes (sets `deletedAt`) and performs a referential cascade:
   - Cancels all non-terminal appointments (sets status → CANCELLED)
   - Cancels unpaid invoices (DRAFT/SENT status)
   - Marks pending payments as FAILED with reason `patient_deleted`
   - Cancels active VideoSessions (REQUESTED, SCHEDULED, WAITING)
   - Cancels waiting QueueEntries
   - Deactivates PrescriptionTasks and cancels CareReminders
5. **`bulkCreatePatients(rows, user, tenantSlug)`** — Batch creation with per-row error tracking, allowing partial success imports.

9.2 Appointment Module

Location: `src/modules/appointment/`

The Appointment module is the scheduling backbone of BestoClinic, handling creation, conflict detection, state transitions, availability computation, and multi-channel notifications.

API Endpoints:

Method	Path	Permission	Description
GET	`/appointments`	APPOINTMENT_READ / READ_OWN	Filtered appointment list
GET	`/appointments/availability-overview`	APPOINTMENT_READ	All doctors' availability overview
GET	`/appointments/availability/:doctorId/summary`	APPOINTMENT_READ	Monthly availability summary
GET	`/appointments/availability/:doctorId`	APPOINTMENT_READ	Daily slot availability
GET	`/appointments/:id`	APPOINTMENT_READ / READ_OWN	Appointment detail
POST	`/appointments`	APPOINTMENT_CREATE	Create appointment
PUT	`/appointments/:id`	APPOINTMENT_UPDATE	Update appointment
DELETE	`/appointments/:id`	APPOINTMENT_CANCEL / CANCEL_OWN	Cancel appointment
POST	`/appointments/:id/admin-reschedule`	APPOINTMENT_UPDATE	Admin-initiated reschedule
POST	`/appointments/:id/cancel-refund`	APPOINTMENT_CANCEL	Cancel with payment refund
POST	`/appointments/:id/cancel-credit`	APPOINTMENT_CANCEL	Cancel with credit note
POST	`/appointments/:id/cancel-unpaid`	APPOINTMENT_CANCEL	Cancel unpaid appointment
POST	`/appointments/:id/mark-contacted`	APPOINTMENT_UPDATE	Mark no-show patient as contacted
POST	`/appointments/bulk-cancel-refund`	APPOINTMENT_CANCEL	Bulk cancel with refund

Appointment Creation Logic (`createAppointment`):

The creation flow involves multiple validation and resolution steps:

```
Step 1: Resolve clinic booking policy (auto-confirm, type durations)
Step 2: Validate scheduling time against:
        → Doctor weekly schedule (working days/hours)
        → Doctor scheduled leaves
        → Branch operating hours
Step 3: Check for scheduling conflicts
        → Existing appointments for same doctor at requested time
        → Duration window overlap check
        → Excludes CANCELLED and NO_SHOW statuses
Step 4: Resolve reminder timings
        → Doctor defaults → Clinic policy → System fallback
Step 5: Set initial status
        → CONFIRMED (if clinic autoConfirm enabled)
        → REQUESTED (if manual confirmation required)
Step 6: Auto-create VideoSession for teleconsult type
Step 7: Schedule BullMQ reminder jobs
Step 8: Send notifications (doctor + front-desk)
Step 9: Broadcast WebSocket (appointment:update, availability:invalidated)
```

(Algorithm location: `src/modules/appointment/appointment.service.js:createAppointment`)

State Transition Model:

```
REQUESTED → CONFIRMED, CANCELLED
CONFIRMED → CHECKED_IN, CANCELLED, IN_PROGRESS
CHECKED_IN → IN_PROGRESS, CANCELLED, NO_SHOW
IN_PROGRESS → COMPLETED, CANCELLED
COMPLETED → (terminal)
CANCELLED → (terminal)
NO_SHOW → (terminal)
```

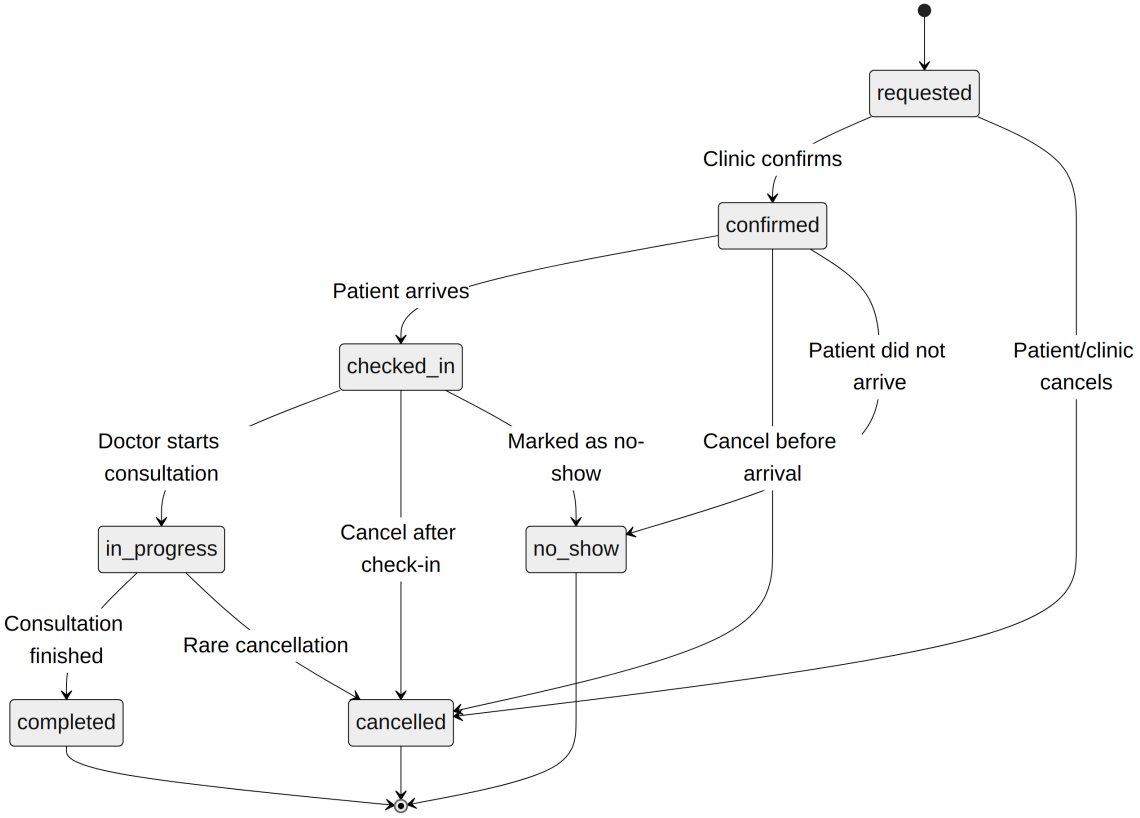


Figure 6: Diagram 7: Appointment State Machine

Update Cascade Operations:

When an appointment status changes, the service triggers cascading updates:

- **CANCELLED / NO_SHOW** → Cancels linked invoices (DRAFT/SENT) → Fails linked pending payments → Cancels linked VideoSession → Cancels linked QueueEntry → Removes scheduled reminders
- **CHECKED_IN** → Auto-queues patient in SmartQueue (creates QueueEntry with WAITING status)
- **NO_SHOW** → Sets `noShowAt` timestamp, increments `patient.noShowCount`
- All transitions → WebSocket broadcast to tenant room

9.3 Invoice Module

Location: `src/modules/invoice/`

The Invoice module manages the complete invoice lifecycle from draft creation through payment reconciliation and PDF generation.

API Endpoints:

Method	Path	Permission	Description
GET	`/invoices`	INVOICE_READ / INVOICE_READ_OWN	Paginated invoice list
GET	`/invoices/:id`	INVOICE_READ / INVOICE_READ_OWN	Invoice detail
GET	`/invoices/:id/pdf`	INVOICE_READ / INVOICE_READ_OWN	Download invoice PDF
POST	`/invoices`	INVOICE_CREATE	Create invoice
PUT	`/invoices/:id`	INVOICE_UPDATE	Update invoice
DELETE	`/invoices/:id`	INVOICE_VOID	Void/soft-delete invoice

Invoice Status Transition Model:

```
DRAFT → SENT, PAID, CANCELLED
SENT → PAID, PARTIALLY_PAID, OVERDUE, CANCELLED
PARTIALLY_PAID → PAID, OVERDUE, CANCELLED, REFUNDED
OVERDUE → PAID, PARTIALLY_PAID, CANCELLED
PAID → REFUNDED
CANCELLED → (terminal)
REFUNDED → (terminal)
```

GST Calculation Logic (Enhanced Billing Service):

The enhanced billing service (`src/modules/billing/billing.service.js`) performs full GST-compliant invoice calculation:

```
1. Resolve service items:
   → If lineItem has serviceItem ID, fetch ServiceItem for defaults
   → Populate description, unitPrice, hsnCode, taxRate from service item

2. Determine GST supply type:
   → 'interstate' → IGST only
   → 'intrastate' → 50/50 CGST + SGST

3. Per-item calculation:
   lineSubtotal = quantity × unitPrice
   lineDiscount = item.discount || 0
   lineTax = (taxRate / 100) × (lineSubtotal - lineDiscount)
   lineTotal = lineSubtotal - lineDiscount + lineTax

4. Invoice totals:
   subtotal = Σ(lineSubtotal)
   discountAmount = Σ(lineDiscount)
   taxAmount = Σ(lineTax)
   totalAmount = subtotal - discountAmount + taxAmount

5. Rounding:
   Math.round((n + Number.EPSILON) * 100) / 100
```

(Algorithm location: `src/modules/billing/billing.service.js:createEnhancedInvoice`)

PDF Generation (`generateInvoicePdf`): - Generates PDF buffer with clinic branding (name, address, phone, email, GSTIN, logo) - Includes line items with HSN codes and tax breakdowns - Returns binary buffer for download

Protection Rules: - Line items and financial fields cannot be modified once any payment exists (`paidAmount > 0`) - Deletion rejected if status is PAID or PARTIALLY_PAID - Cascades pending payment failure on cancellation

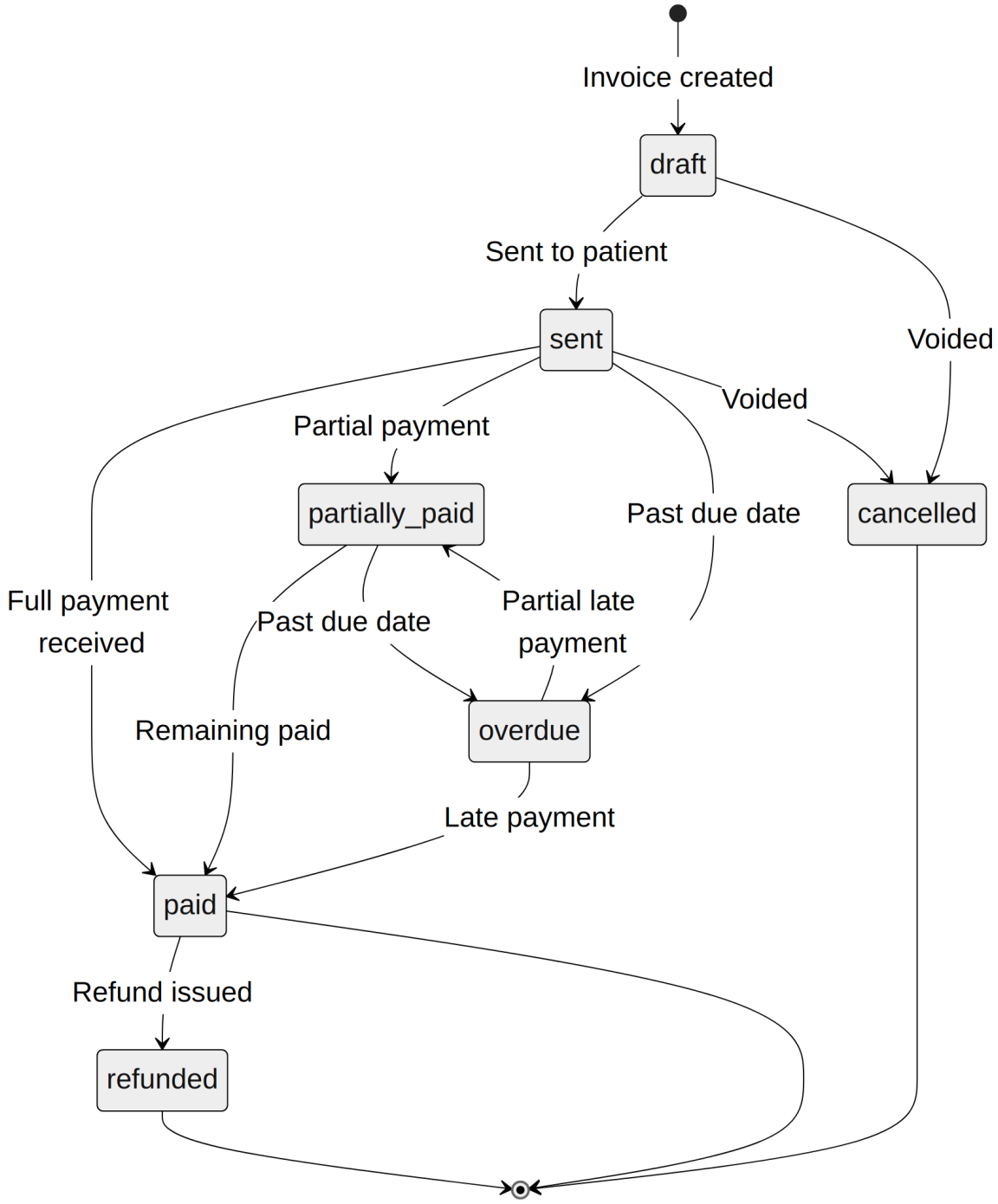


Figure 7: Diagram 9: Invoice Lifecycle

9.4 Doctor Dashboard & Consultation Module

Location: `src/modules/doctorDashboard/`

This is the most complex module in the system, serving as the primary interface for doctor workflows including consultations, prescriptions, care management, scheduling, and patient monitoring.

API Endpoints (33 endpoints):

Category	Method	Path	Description
Dashboard	GET	`/doctor/stats`	Daily/weekly stats, revenue, next appointment
Dashboard	GET	`/doctor/current-room`	Current room assignment
Consultation	GET	`/doctor/consultations`	List consultations
Consultation	GET	`/doctor/consultation/by-appointment/:id`	Get by appointment
Consultation	POST	`/doctor/consultation`	Start consultation
Consultation	PUT	`/doctor/consultation/:id`	Update consultation
Consultation	PUT	`/doctor/consultation/:id/complete`	Complete consultation
Attachments	POST	`/doctor/attachments`	Upload attachment
Attachments	GET	`/doctor/attachments/:consultationId`	List attachments
Follow-up	POST	`/doctor/follow-ups`	Create follow-up
Compliance	GET	`/doctor/patients/:patientId/compliance`	Patient compliance
Care Tasks	PUT	`/doctor/patients/:patientId/care-tasks/:taskId/toggle`	Toggle reminder
Care Tasks	POST	`/doctor/patients/:patientId/prescriptions/:id/enable-reminders`	Enable reminders
Care Tasks	DELETE	`/doctor/patients/:patientId/prescriptions/:id/disable-reminders`	Disable reminders
Care Tasks	GET	`/doctor/compliance-summary`	Doctor compliance summary
Care Tasks	GET	`/doctor/care-tasks`	List care tasks
Care Tasks	POST	`/doctor/patients/:patientId/care-summary`	Send care summary push
Patients	GET	`/doctor/patients`	Doctor’s patient list
Schedule	GET	`/doctor/schedule`	Self-service schedule
Schedule	GET	`/doctor/leaves`	Self-service leaves
Schedule	POST	`/doctor/leaves`	Request leave
Schedule	DELETE	`/doctor/leaves/:leaveId`	Delete leave
Schedule	GET	`/doctor/overrides`	Schedule overrides
Schedule	POST	`/doctor/overrides`	Create override
Schedule	DELETE	`/doctor/overrides/:overrideId`	Delete override
Holidays	GET	`/doctor/holidays`	View holidays
Profile	GET	`/doctor/profile`	Get doctor profile

Category	Method	Path	Description
Profile	PUT	`/doctor/profile`	Update profile
Templates	GET	`/doctor/templates`	List consultation templates
Templates	GET	`/doctor/templates/:id`	Get template
Templates	POST	`/doctor/templates`	Create template
Templates	PUT	`/doctor/templates/:id`	Update template
Templates	DELETE	`/doctor/templates/:id`	Delete template

Consultation Lifecycle:

The consultation workflow follows these key stages:

Stage 1 — Start Consultation (`startConsultation`): - Validates appointment exists and is not deleted - Checks authorization (doctor-only for own appointments, admin for any) - Returns existing TreatmentRecord if already IN_PROGRESS/COMPLETED - Creates new TreatmentRecord for valid appointment statuses - Updates appointment status → IN_PROGRESS - Auto-assigns consultation room (explicit selection or auto-first-available from branch) - Syncs QueueEntry status → IN_CONSULTATION - Sends notifications (patient, front-desk, admins) - Broadcasts WebSocket events (appointment:update, consultation:started)

Stage 2 — Update Consultation (`updateConsultation`): - Accepts SOAP fields: diagnosis, treatment, prescription, notes, followUpDate, chiefComplaint, medicalHistory, allergies, vitals, examination, oralFindings, severity, assessmentNotes, treatmentPlan, followUpInstructions, procedure, priority, estimatedCost, visitType, patientConsent, careTasks, teeth, materialsUsed - Saves doctorSignature as draft (dataUrl only) - Enforces doctor-only access (non-admin) - Frontend auto-saves with 30-second debounce

Stage 3 — Complete Consultation (`completeConsultation`):

```
1. Validate diagnosis and treatment are not empty/"In progress"
2. Require digital signature (dataUrl, signedAt, signedBy, ipAddress)
3. Set record.status = 'completed', completedAt timestamp

4. INVENTORY DEDUCTION:
   For each materialsUsed item:
     → Decrement InventoryItem.currentStock
     → Create InventoryTransaction (reason: "Used during consultation")

5. AUTO-CREATE PRESCRIPTION:
   If record.prescription.length > 0:
     → Map medications from prescription array
     → Set status = 'active'

6. CARE TASK GENERATION:
   For each careTask with enabled + reminderEnabled:
     → Create PrescriptionTask entry
     → Generate CareReminder entries via generateReminders(task)
     → Schedule reminders (bulk or EOD checklist by reminderMode)
     → Default 7-day duration

7. Send care summary FCM push to patient
8. Update linked appointment status → COMPLETED
9. Auto-complete linked QueueEntry, release consultation room
10. Notify patient, front-desk, and admins
11. Enqueue auto-invoice generation job (BullMQ)
12. Broadcast WebSocket events
```

(Algorithm location: `src/modules/doctorDashboard/doctorDashboard.service.js:completeConsultation`)

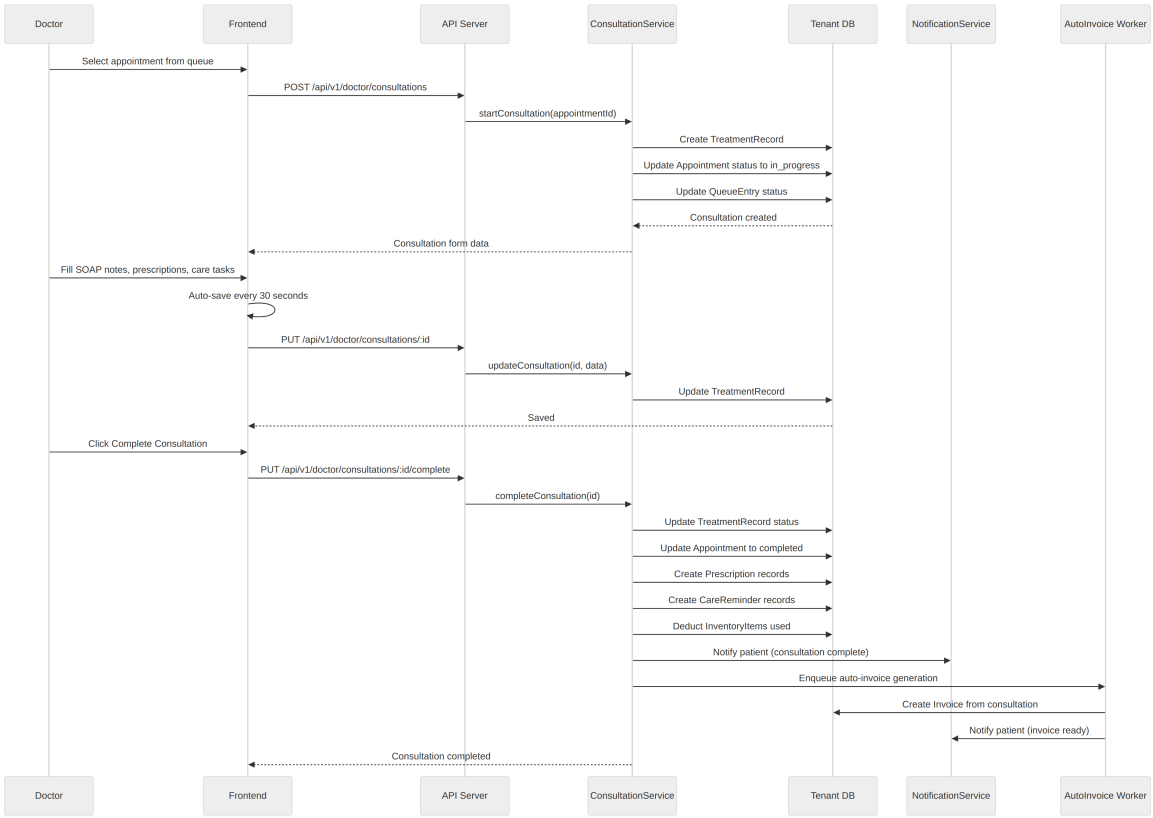


Figure 8: Diagram 12: Doctor Consultation Workflow

Care Compliance Tracking:

The module tracks patient medication adherence and care task completion:

- **`getDoctorComplianceSummary`** — Aggregates completedCount/totalReminders from PrescriptionTasks. Identifies low-compliance patients (< 50% for tasks with ≥ 3 reminders). Returns avgCompliance percentage.
- **`listDoctorCareTasks`** — Returns enriched care tasks with progress metrics (completed, skipped, missed, complianceRate), last response status, next pending reminder date, and patient details.

9.5 Smart Queue Module

Location: `src/modules/queue/`

The Smart Queue implements a priority-weighted, real-time patient queue management system with room assignment.

API Endpoints:

Method	Path	Permission	Description
GET	`/queue`	QUEUE_READ	Paginated queue entries
GET	`/queue/current`	QUEUE_READ	Current branch queue (15s cache)
GET	`/queue/:id`	QUEUE_READ	Queue entry detail
POST	`/queue`	QUEUE_MANAGE	Add patient to queue

Method	Path	Permission	Description
PUT	`/queue/:id/priority`	QUEUE_MANAGE	Change entry priority
PUT	`/queue/:id/move`	QUEUE_MANAGE	Move entry up/down
PUT	`/queue/:id/call`	QUEUE_MANAGE	Call patient
PUT	`/queue/:id/complete`	QUEUE_MANAGE	Complete consultation
PUT	`/queue/:id/cancel`	QUEUE_MANAGE	Cancel/mark no-show
PUT	`/queue/swap`	QUEUE_MANAGE	Swap two entry positions

Queue Algorithm:

```
Priority Weights:
  EMERGENCY = 0 (highest)
  HIGH      = 1
  NORMAL    = 2 (lowest)

Sort Order:
  1. PRIORITY_WEIGHT ascending (emergency first)
  2. sortOrder ascending (manual reorder support)
  3. tokenNumber ascending (FIFO fallback)

Token Generation:
  tokenNumber = MAX(tokenNumber for today's branch) + 1
  Retry loop (3 attempts) for duplicate token race conditions
```

(Algorithm location: `src/modules/queue/queue.service.js:addToQueue`)

Room Assignment (`callPatient`):

When a patient is called: 1. Explicit room selection OR auto-first-available from branch (sorted by type ASC, code ASC) 2. Updates room status → `occupied` with currentDoctor, currentPatient, currentAppointment, occupiedSince 3. Updates linked appointment status → IN_PROGRESS 4. Sends in-app notification and SMS to patient 5. Broadcasts `queue:update` WebSocket event

Room Release (`completeConsultation`):

When consultation completes: 1. Room status → `available`, clears currentDoctor/currentPatient/currentAppointment 2. Linked appointment status → COMPLETED 3. QueueEntry status → COMPLETED

9.6 Payment Module

Location: `src/modules/payment/`

The Payment module handles all payment operations with atomic financial operations and comprehensive audit logging.

API Endpoints:

Method	Path	Permission	Description
POST	`/payments/gateway/webhook`	None (HMAC verified)	Razorpay webhook handler
POST	`/payments`	PAYMENT_CREATE	Record manual payment
POST	`/payments/gateway/order`	PAYMENT_CREATE	Create Razorpay order
POST	`/payments/gateway/verify`	PAYMENT_CREATE	Verify gateway payment

Method	Path	Permission	Description
POST	`/payments/:id/refund`	PAYMENT_REFUND	Process refund
GET	`/payments`	PAYMENT_READ / PAYMENT_READ_OWN	List payments
GET	`/payments/invoice/:invoiceId`	PAYMENT_READ / PAYMENT_READ_OWN	Payments for invoice
GET	`/payments/:id/receipt`	PAYMENT_READ / PAYMENT_READ_OWN	Download receipt
GET	`/payments/:id`	PAYMENT_READ / PAYMENT_READ_OWN	Payment detail

Atomic Payment Processing:

The payment service uses atomic MongoDB operations to prevent race conditions:

```
Invoice Update (atomic):
  → $inc: { paidAmount: payment.amount }
  → $push: { payments: payment._id, paidVia: { mode, amount, reference } }
  → Auto-determine status:
    IF paidAmount >= totalAmount → PAID (set paidAt)
    ELSE IF paidAmount > 0 → PARTIALLY_PAID

Ledger Entry (transactional):
  → Read previous balance within MongoDB transaction
  → Calculate new balance: CREDIT reduces, DEBIT increases
  → Write new entry within same transaction
  → Prevents concurrent ledger race conditions

Audit Log:
  → FinancialAuditLog with before/after snapshots
  → Tracks actor, actorRole, action type, metadata
```

(Algorithm location: `src/modules/payment/payment.service.js:_updateInvoicePayment`)

9.7 Cash Register Module

Location: `src/modules/cashRegister/`

The Cash Register module manages daily cash drawer operations with variance detection and manager approval workflows.

API Endpoints:

Method	Path	Permission	Description
POST	`/cash-register/open`	CASH_REGISTER_OPEN	Open register
PUT	`/cash-register/:id/close`	CASH_REGISTER_CLOSE	Close register
PUT	`/cash-register/:id/reconcile`	CASH_REGISTER_RECONCILE	Reconcile register
GET	`/cash-register/current`	CASH_REGISTER_READ	Current open register

Method	Path	Permission	Description
GET	`/cash-register`	CASH_REGISTER_READ	List registers
GET	`/cash-register/:id`	CASH_REGISTER_READ	Register detail
GET	`/cash-register/:id/transactions`	CASH_REGISTER_READ	Register transactions

Variance Detection Algorithm:

```
On register close:
  expectedBalance = openingBalance + totalCashIn - totalCashOut
  variance = closingBalance - expectedBalance

  IF |variance| > 100 (Rs.100 threshold):
    managerApproval.required = true
    → Register requires manager reconciliation before closure is final

Close uses CAS (Compare-And-Set) via findOneAndUpdate
  → Prevents double-close race conditions
```

(Algorithm location: `src/modules/cashRegister/cashRegister.service.js:closeRegister`)

Status Model:

```
OPEN → CLOSED → RECONCILED (terminal)
```

9.8 Prescription Module

Location: `src/modules/prescription/`

API Endpoints:

Method	Path	Permission	Description
GET	`/prescriptions`	PRESCRIPTION_READ / READ_OWN	List prescriptions
GET	`/prescriptions/:id`	PRESCRIPTION_READ / READ_OWN	Prescription detail
POST	`/prescriptions`	PRESCRIPTION_CREATE	Create prescription
PUT	`/prescriptions/:id`	PRESCRIPTION_UPDATE	Update prescription
PUT	`/prescriptions/:id/cancel`	PRESCRIPTION_UPDATE	Cancel prescription

Cancellation Cascade: When a prescription is cancelled: - Sets status → CANCELLED, `followUpRemindersEnabled = false` - Deactivates linked PrescriptionTasks - Cancels all pending CareReminders - Notifies patient of cancellation

---

9.9 Billing Module (Enhanced)

Location: `src/modules/billing/`

The enhanced billing module provides advanced invoicing, service item management, financial analytics, and audit capabilities.

API Endpoints:

Method	Path	Permission	Description
POST	`/billing/invoices`	INVOICE_CREATE	Create enhanced invoice
POST	`/billing/invoices/:id/payment`	PAYMENT_CREATE	Record payment on invoice
GET	`/billing/service-items`	SERVICE_ITEM_READ	List service items
POST	`/billing/service-items`	SERVICE_ITEM_CREATE	Create service item
PUT	`/billing/service-items/:id`	SERVICE_ITEM_UPDATE	Update service item
GET	`/billing/analytics/revenue`	REVENUE_REPORT_VIEW	Revenue analytics
GET	`/billing/analytics/outstanding`	REVENUE_REPORT_VIEW	Outstanding report
GET	`/billing/analytics/doctor-revenue`	REVENUE_REPORT_VIEW	Doctor revenue
GET	`/billing/reports/day-end`	REVENUE_REPORT_VIEW	Day-end report
GET	`/billing/reports/gst-summary`	REVENUE_REPORT_VIEW	GST summary
GET	`/billing/reports/collection-metrics`	REVENUE_REPORT_VIEW	Collection metrics
GET	`/billing/audit-trail`	FINANCIAL_AUDIT_READ	Financial audit trail
POST	`/billing/write-off`	INVOICE_WRITE_OFF	Write-off invoice amount
GET	`/billing/ledger/:patientId`	LEDGER_READ / LEDGER_READ_OWN	Patient ledger
POST	`/billing/ledger/advance`	PAYMENT_CREATE	Record advance payment
POST	`/billing/ledger/adjustment`	LEDGER_ADJUST	Adjust ledger

Chapter 10: Feature Specifications

10.1 AI Dental Scan System

Location: `src/modules/dentalScan/`

The dental scan module provides an AI-powered dental analysis pipeline using Google Gemini 2.5 Flash for image analysis with multi-angle capture support.

API Endpoints:

Method	Path	Permission	Description
POST	`/dental-scans`	—	Initiate guided scan session
PUT	`/dental-scans/:id/upload`	—	Upload image (10MB limit)
POST	`/dental-scans/:id/analyze`	—	Trigger AI analysis
POST	`/dental-scans/:id/validate-frame`	—	Real-time positioning check
POST	`/dental-scans/:id/detect-teeth`	—	Tooth number detection
GET	`/dental-scans/:id/result`	—	Retrieve AI analysis
GET	`/dental-scans/:id/recommendations`	—	Doctor recommendations
GET	`/dental-scans/history`	—	Patient scan history
GET	`/dental-scans/:id`	—	Scan detail
PUT	`/dental-scans/:id/review`	—	Doctor review (verdict)

Scan Modes: - `guided_3pos` — Requires angles: left, front, right, tongue - `classic_5angle` — Requires angles: front, left, right, upper, lower

AI Analysis Pipeline:

```
1. INITIATE SCAN:
   → Create DentalScan record, status = CAPTURING
   → Link to Patient, record consent (timestamp + IP)

2. IMAGE UPLOAD (per angle):
   → Base64 → S3 upload via uploadFile() service
   → Validate angle allowed for scan mode
   → Prevent duplicate angles

3. FRAME VALIDATION (real-time, ~0.5-1.5s):
   → Call validateDentalFrame() from aiAnalysis service
   → Returns positioning feedback for guidance overlay
```

```
4. TOOTH DETECTION (post-capture):
   → Analyze frame to detect visible tooth numbers
   → Store detectedTeeth, quadrants, positionConfidence

5. TRIGGER ANALYSIS:
   → Validate ALL required angles captured
   → Enqueue BullMQ job: 'analyze-dental-scan' on dentalScanQueue
   → Status → PROCESSING

6. AI WORKER (BullMQ):
   → Call analyzeDentalImage() → Gemini 2.5 Flash API
   → Multi-image prompt construction (_buildMultiImagePrompt)
   → Robust JSON extraction: handles markdown code blocks + token truncation repair
   → On success: status → COMPLETED, store aiAnalysis
   → On failure: status → FAILED, store errorMessage
   → Notify patient + all doctors (WebSocket + DB notification)

7. DOCTOR REVIEW:
   → Verdict: "confirmed" | "modified" | "rejected"
   → Stores clinicalNotes, modifiedFindings, recommendations, urgency
   → Notifies patient via WebSocket + DB notification
   → Idempotent (prevents double-review)
```

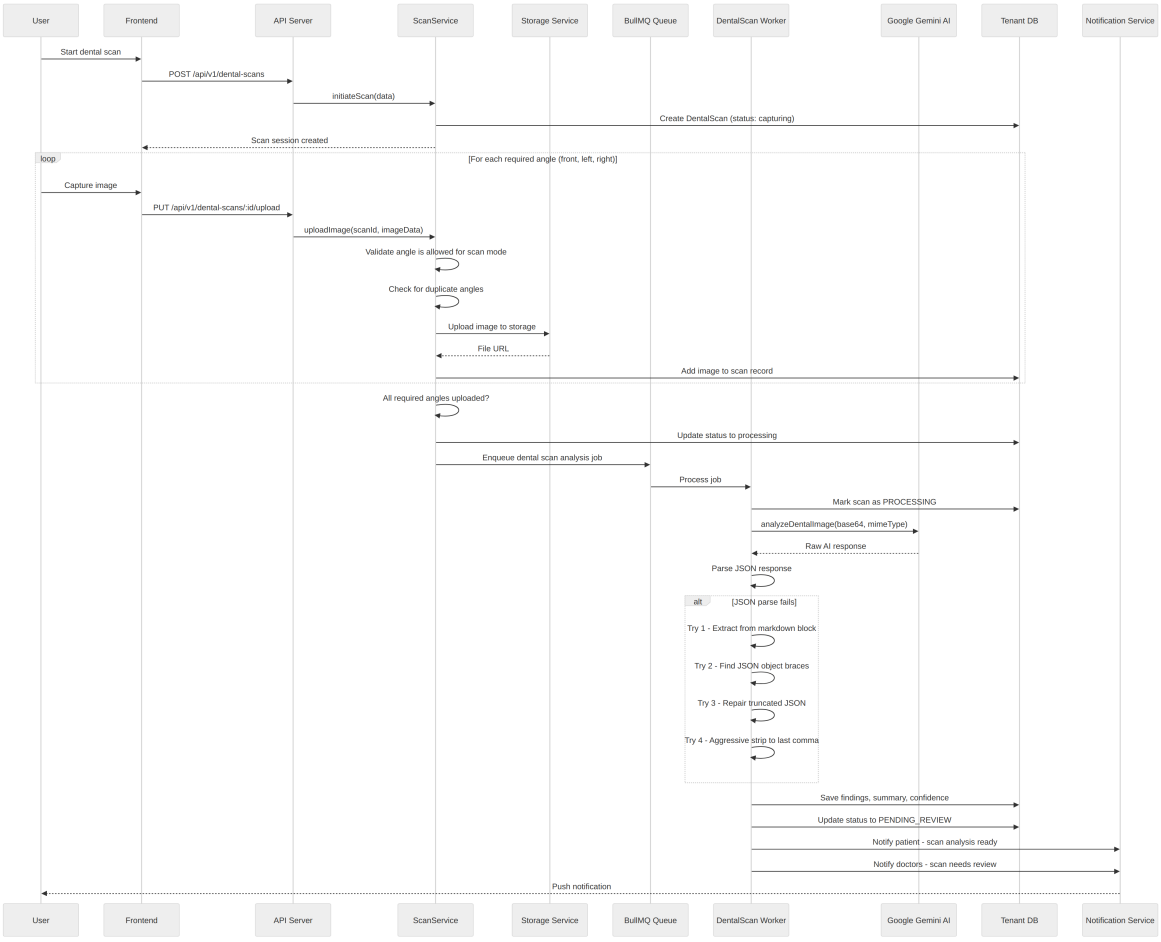


Figure 9: Diagram 17: Dental Scan AI Pipeline

AI Observation Types: `plaque`, `inflammation`, `discoloration`, `misalignment`, `cavity`, `gum_recession`, `other`

Doctor Recommendation Algorithm:

```
1. Map AI observations → medical specializations:
   plaque, cavity          → General Dentistry
   inflammation, gum_recession → Periodontics
   misalignment            → Orthodontics
   discoloration           → Cosmetic Dentistry
   cavity (high confidence) → Endodontics

2. Score doctors:
   relevanceScore   = (matchedObservations / totalObservations) × 0.5
   confidenceBoost  = (avgConfidenceWeights) × 0.3
   ratingScore      = (doctor.rating / 5) × 0.2
   totalScore       = relevanceScore + confidenceBoost + ratingScore

3. Return top 5 doctors by totalScore
```

(Algorithm location: `src/modules/dentalScan/dentalScan.service.js:getRecommendations`)

10.2 Telemedicine & Video Calling

Location: `src/modules/telemedicine/`

The telemedicine module provides WebRTC-based video consultations with end-to-end encrypted chat, room token management, and session lifecycle management.

API Endpoints (17 endpoints):

Method	Path	Permission	Description
GET	`/telemedicine`	—	List video sessions
POST	`/telemedicine`	—	Create session
POST	`/telemedicine/presence`		Set doctor presence status
GET	`/telemedicine/presence`		Get presence snapshot
GET	`/telemedicine/:id`		Session detail
PUT	`/telemedicine/:id/accept`		Doctor accepts call
POST	`/telemedicine/:id/join`		Join session (get room token)
PUT	`/telemedicine/:id/end`		End session
PUT	`/telemedicine/:id/cancel`		Cancel session
PUT	`/telemedicine/:id/decline`		Doctor declines
PUT	`/telemedicine/:id/extend`		Extend duration (+15min, max 2×)
PUT	`/telemedicine/:id/consent`		Update recording consent
POST	`/telemedicine/:id/messages`		Send encrypted message
GET	`/telemedicine/:id/messages`		Get decrypted messages
POST	`/telemedicine/:id/messages/:msgId/read`		Mark message read
POST	`/telemedicine/:id/files`		Share file in session
POST	`/telemedicine/:id/events`		Log connection diagnostics

Room Token Security:

```
Token Generation (roomToken.service.js):
```

```
  roomId format: tm-{tenantPrefix}-{uuid}
  Payload: { roomId, participantId, role, exp, iat, jti }
  Encryption: AES-256-GCM using TELEMEDICINE_SECRET
  Returns: { token, iv, expiresAt }

Token Verification:
  Decrypts with same secret + IV
  Validates expiry + authenticity
  Throws on tampering or timeout

Message Encryption:
  encryptMessage(plaintext) → { ciphertext, iv }
  decryptMessage(ciphertext, ivHex) → plaintext
  Same AES-256-GCM scheme
```

(Algorithm location: `src/modules/telemedicine/roomToken.service.js`)

Session State Machine:

```
REQUESTED → SCHEDULED (if scheduledAt provided)
REQUESTED → WAITING (doctor accepts)
SCHEDULED → WAITING (session time reached)
WAITING → ACTIVE (both parties connected via WebRTC)
WAITING → CANCELLED (initiator cancels)
ACTIVE → PAUSED (temporary hold)
ACTIVE/PAUSED → ENDED (session complete)
→ EXPIRED (no activity beyond max duration)
→ NO_SHOW (neither party joined)
```

Presence Management: - Doctors set status: `online`, `busy`, `dnd` (do-not-disturb), `offline` - Real-time snapshot for UI availability indicators - Used for auto-routing and incoming call decisions

---

10.3 Emergency Module

Location: `src/modules/emergency/`

The emergency module implements priority-based triage with automatic severity computation and real-time doctor broadcast.

API Endpoints:

Method	Path	Permission	Description
GET	`/emergency`	—	List sessions (role-filtered)
GET	`/emergency/:id`	—	Session detail
POST	`/emergency`	—	Create emergency (patient SOS)
PUT	`/emergency/:id/accept`	—	Doctor claims emergency
PUT	`/emergency/:id/decline`	—	Doctor dismisses from queue
PUT	`/emergency/:id/end`	—	End emergency
PUT	`/emergency/:id/cancel`	—	Patient cancels

Priority Auto-Calculation:

```
function computePriority({ pain, symptoms }):
  criticalTriggers = ['chest pain', 'unconscious', 'bleeding heavy',
                      'stroke', 'not breathing']
```

```
highTriggers = ['bleeding', 'severe pain', 'broken', 'fainting']

IF criticalTriggers match OR pain >= 9  → 'critical'
IF highTriggers match OR pain >= 7      → 'high'
IF pain >= 4                            → 'medium'
ELSE                                     → 'low'
```

(Algorithm location: `src/modules/emergency/emergency.service.js:_computePriority`)

First-Come-First-Serve Acceptance: - Emergency broadcasts to ALL on-duty doctors via `emitToRole('doctor')` + FCM push - First doctor to accept claims the emergency (status → ACTIVE) - Other doctors' pickup notifications auto-expire - Declining stores in doctor's `dismissedEmergencies` cache (prevents repeat alerts)

10.4 Subscription & Razorpay Integration

Location: `src/modules/subscription/`

API Endpoints:

Method	Path	Permission	Description
GET	`/subscription`	—	Get current subscription
POST	`/subscription`	—	Create subscription
PUT	`/subscription/change-plan`	—	Upgrade/downgrade plan
POST	`/subscription/cancel`	—	Cancel subscription
POST	`/subscription/webhook`	None (HMAC verified)	Razorpay webhook

Subscription Status Flow:

```
TRIAL (if trial enabled, default 14 days)
  ↓ trial expires
ACTIVE (billing active)
  ↓ payment fails
GRACE_PERIOD (7-day recovery window)
  ↓ still fails
SUSPENDED (features locked, read-only)
  ↓ manual reactivation with payment
ACTIVE

ACTIVE → CANCELLED (manual cancel)
  ↓ downgrade to free tier
Features: maxDoctors=1, maxStaff=2, maxBranches=1,
          maxStorageMB=100, smsEnabled=false,
          teleHealthEnabled=false, apiAccess=false
```

Webhook Processing: - Signature validation: `crypto.createHmac('sha256', RAZORPAY_WEBHOOK_SECRET).update(body).digest('hex')` - Idempotent event processing via `eventIdempotencyKey` - Handles: `subscription.charged`, `subscription.updated`, `subscription.ended`, payment failures

---

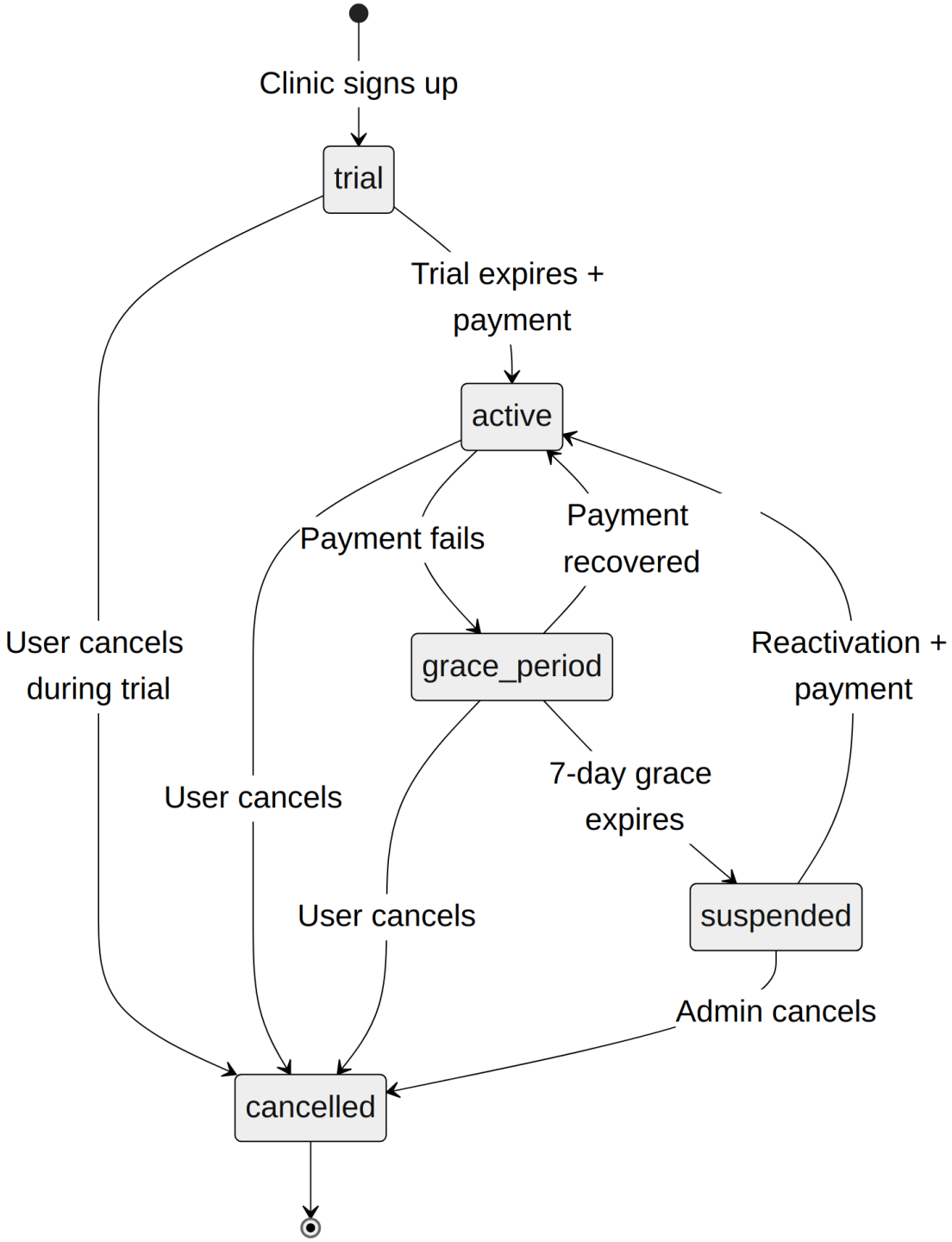


Figure 10: Diagram 8: Subscription Lifecycle

10.5 Insurance Claims Management

Location: `src/modules/insurance/`

API Endpoints:

Method	Path	Permission	Description
GET	`/insurance/providers`		List insurance providers
GET	`/insurance/providers/:id`		Provider detail
POST	`/insurance/providers`		Create provider
PUT	`/insurance/providers/:id`		Update provider
DELETE	`/insurance/providers/:id`		Delete provider
GET	`/insurance/claims`		List claims
GET	`/insurance/claims/:id`		Claim detail
POST	`/insurance/claims`		Create claim
PUT	`/insurance/claims/:id/status`		Update claim status
DELETE	`/insurance/claims/:id`		Delete claim
PUT	`/insurance/claims/:id/pre-auth`		Record pre-authorization
GET	`/insurance/claims/:id/co-pay`		Calculate co-payment
PUT	`/insurance/claims/:id/reconcile`		Reconcile settlement

Insurance Claim State Machine:

```
SUBMITTED → ACKNOWLEDGED, REJECTED
ACKNOWLEDGED → PRE_AUTH_PENDING, PROCESSED
PRE_AUTH_PENDING → PROCESSING, REJECTED
PROCESSING → PARTIAL_SETTLED, SETTLED, REJECTED
PARTIAL_SETTLED → SETTLED, REVERSED
SETTLED, REJECTED, REVERSED → (terminal)
```

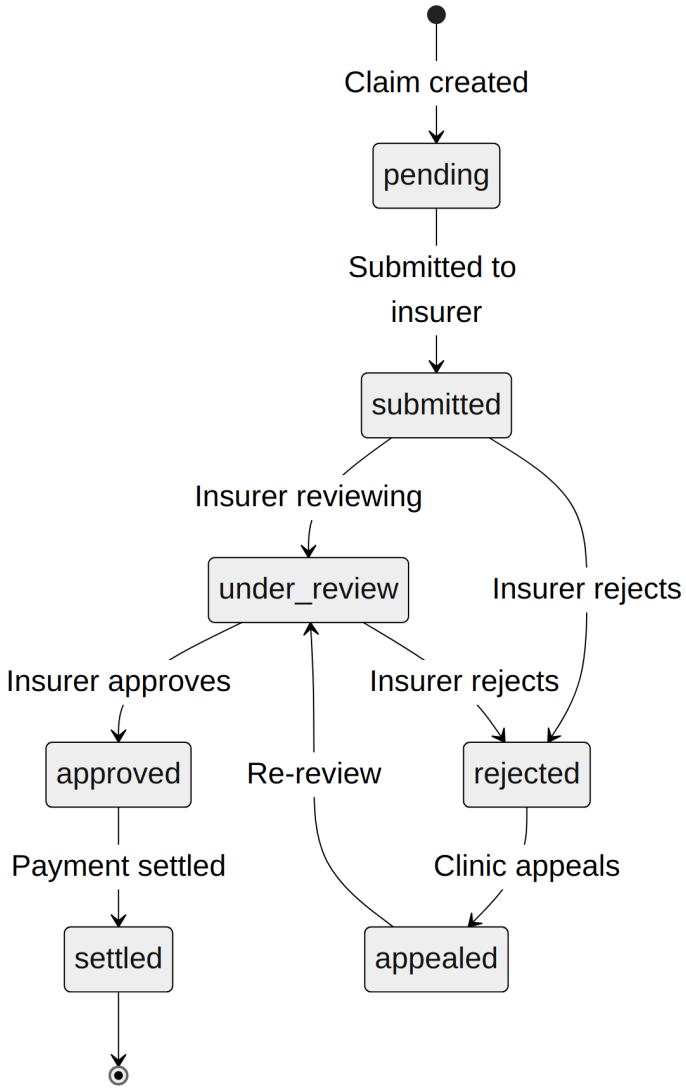


Diagram 10: Insurance Claim Lifecycle

Co-Payment Formula:

```
patientShare = (invoiceAmount × copayPercentage) - creditApplied
insurerShare = invoiceAmount - patientShare
```

10.6 Patient Portal (Self-Service)

Location: `src/modules/patientPortal/`

The patient portal is the largest service file in the system (~82KB), providing comprehensive self-service capabilities.

API Endpoints (30+ endpoints):

Category	Method	Path	Description
Profile	GET	`/portal/profiles`	All linked patients (guardian support)
Profile	GET	`/portal/profiles/:profileId`	Single profile
Profile	PUT	`/portal/profiles/:profileId`	Update profile
Dashboard	GET	`/portal/dashboard`	Summary cards
Discovery	GET	`/portal/departments`	List departments
Discovery	GET	`/portal/doctors`	List doctors with filters
Discovery	GET	`/portal/doctors/:id`	Doctor profile + availability
Booking	GET	`/portal/appointments`	List appointments
Booking	GET	`/portal/appointments/:id`	Appointment detail
Booking	POST	`/portal/appointments`	Book appointment
Booking	PUT	`/portal/appointments/:id/cancel`	Cancel appointment
Booking	PUT	`/portal/appointments/:id/reschedule`	Reschedule
Booking	POST	`/portal/appointments/:id/confirm-attendance`	Confirm attendance
Emergency	POST	`/portal/emergency`	Create emergency
Health	GET	`/portal/prescriptions`	List prescriptions
Health	GET	`/portal/medical-history`	Paginated timeline
Health	GET	`/portal/consultations/:appointmentId`	Consultation detail
Payments	GET	`/portal/payments`	List payments
Payments	POST	`/portal/payments/:invoiceId/pay`	Initiate Razorpay payment
Payments	POST	`/portal/payments/verify-payment`	Verify payment
Family	GET	`/portal/dependents-profiles`	List dependents
Family	POST	`/portal/dependents`	Create dependent
Family	PUT	`/portal/dependents/:depId`	Update dependent
Family	DELETE	`/portal/dependents/:depId`	Delete dependent
Insurance	POST	`/portal/insurance`	Upload insurance docs
Insurance	GET	`/portal/insurance`	Get insurance info
Reviews	GET	`/portal/reviews/:doctorId`	List reviews
Reviews	POST	`/portal/reviews/:doctorId`	Submit review
Sharing	GET	`/portal/share/view/:token`	View shared record (public)

Category	Method	Path	Description
Sharing	GET	`/portal/share/request-access`	Request guardian access
Audit	GET	`/portal/access-logs`	Access audit trail

Key Features: - **Guardian Support:** Patients with `guardian` role can manage multiple dependents' appointments and records - **Doctor Discovery:** Browse doctors by specialty, department, availability; view ratings and reviews - **Payment Integration:** Razorpay order creation and client-side payment verification - **Shared Records:** Time-limited, token-based public URLs for sharing medical records with specialists or insurance companies - **Review System:** 1-5 rating with `isVerified` flag (only after completed appointment)

10.7 Analytics Module

Location: `src/modules/analytics/`

API Endpoints:

Method	Path	Feature Guard	Description
GET	`/analytics/summary`		Quick overview (no guard)
GET	`/analytics/revenue`	advancedReports	Revenue trends by period
GET	`/analytics/appointments`	advancedReports	Appointment trends
GET	`/analytics/doctors`	advancedReports	Doctor performance
GET	`/analytics/no-shows`	advancedReports	No-show analysis
GET	`/analytics/insurance`	advancedReports	Insurance claim trends
GET	`/analytics/telemedicine`	advancedReports	Telemedicine usage
GET	`/analytics/patients`	advancedReports	Patient growth
GET	`/analytics/integrations`	advancedReports	Integration usage
GET	`/analytics/compliance-overview`		Compliance stats
GET	`/analytics/compliance-alerts`		Compliance alerts
GET	`/analytics/compliance-trends`		Compliance trends
PUT	`/analytics/compliance-alerts/:id/resolve`		Resolve alert

Caching Strategy: - In-memory cache with 5-minute TTL per tenant - Cache eviction when entries exceed 200 per tenant - `_dateGroupExpr()` formats dates as YYYY-MM-DD (daily), YYYY-Wxx (weekly), or YYYY-MM (monthly) - Feature guard prevents `advancedReports` access for plans without the feature enabled

Analytics Summary Response:

```
{
  "revenue": {
    "today": 45000,
    "thisMonth": 1250000,
```

```
    "lastMonth": 1180000,
    "growthPercent": 5.9
  },
  "appointments": {
    "today": 24,
    "thisWeek": 156,
    "completionRate": 0.87,
    "noShowRate": 0.04
  },
  "patients": {
    "total": 3420,
    "newThisMonth": 87,
    "retentionRate": 0.72
  },
  "telemedicine": {
    "activeSessions": 3,
    "totalThisMonth": 89
  },
  "insurance": {
    "pendingClaims": 12,
    "approvalRate": 0.85
  },
  "outstandingInvoices": 34
}
```

10.8 Inventory & Purchase Orders

Location: `src/modules/inventory/`, `src/modules/purchaseOrder/`

Inventory Endpoints:

Method	Path	Permission	Description
GET	`/inventory`	INVENTORY_READ	List items with filters
GET	`/inventory/low-stock`	INVENTORY_READ	Low-stock alerts
GET	`/inventory/expiring`	INVENTORY_READ	Expiring items (30-day window)
GET	`/inventory/reports`	INVENTORY_READ	Usage/wastage reports
GET	`/inventory/:id`	INVENTORY_READ	Item + recent transactions
POST	`/inventory`	INVENTORY_CREATE	Create item (+ initial stock_in)
POST	`/inventory/bulk`	INVENTORY_CREATE	Bulk CSV import
PUT	`/inventory/:id`	INVENTORY_UPDATE	Update item (prevents stock/code edit)
PUT	`/inventory/:id/restock`	INVENTORY_RESTOCK	Restock with audit trail
DELETE	`/inventory/:id`	INVENTORY_DELETE	Soft delete

Purchase Order Endpoints:

Method	Path	Permission	Description
GET	`/purchase-orders/vendors`	PO_READ	List vendors
GET	`/purchase-orders/vendors/:id`	PO_READ	Vendor detail
POST	`/purchase-orders/vendors`	PO_CREATE	Create vendor
PUT	`/purchase-orders/vendors/:id`	PO_UPDATE	Update vendor
DELETE	`/purchase-orders/vendors/:id`	PO_DELETE	Delete vendor
GET	`/purchase-orders/orders`	PO_READ	List purchase orders
GET	`/purchase-orders/orders/:id`	PO_READ	PO detail
POST	`/purchase-orders/orders`	PO_CREATE	Create PO
PUT	`/purchase-orders/orders/:id`	PO_UPDATE	Update PO (REQUESTED only)
PUT	`/purchase-orders/orders/:id/status`	PO_UPDATE	PO status transition
PUT	`/purchase-orders/orders/:id/cancel`	PO_UPDATE	Cancel PO

Purchase Order State Machine:

```
REQUESTED → APPROVED → DELIVERED → RECEIVED (terminal)
REQUESTED → CANCELLED (terminal)
APPROVED → CANCELLED (terminal)
```

Auto-Restock on Delivery: When a PO status transitions to DELIVERED, the system automatically: 1. Creates `stock_in` InventoryTransactions for all PO items 2. Increments `currentStock` on each InventoryItem 3. Uses `receivedQty` if specified, otherwise ordered quantity 4. Generates audit trail entries

---

10.9 Calendar Feeds (ICS Export)

Location: `src/modules/calendarFeed/`

Enables doctors to subscribe to their appointment calendar in Outlook, Google Calendar, or Apple Calendar via ICS feeds.

API Endpoints:

Method	Path	Permission	Description
POST	`/calendar-feeds/my-feed`	—	Generate personal feed
GET	`/calendar-feeds/my-feeds`	—	List personal feeds

Method	Path	Permission	Description
PUT	`/calendar-feeds/:feedId`	—	Update feed settings
DELETE	`/calendar-feeds/:feedId`	—	Revoke feed
POST	`/calendar-feeds/:feedId/regenerate`	—	Regenerate token
POST	`/calendar-feeds/doctor/:doctorId`	—	Generate feed (admin)
GET	`/calendar-feeds/doctor/:doctorId`	—	List feeds (admin)

Feed URL Format: `/api/v1/public/cal/{token}.ics`

Token Security: - UUID token (128-bit entropy) — cryptographically unguessable - One token = one feed (revoke prevents replay) - Platform token registry enables cross-tenant lookup - Old URLs stop working immediately on revoke/regenerate

ICS Output (RFC 5545):

```
BEGIN:VEVENT
UID:appointment-{appointmentId}@{tenantSlug}
DTSTART:{scheduledAt ISO}
DTEND:{scheduledAt + duration ISO}
SUMMARY:Dr. {doctorName} - {reason}
DESCRIPTION:[patient name if showPatientNames]
LOCATION:{branch.name}
STATUS:CONFIRMED|TENTATIVE
END:VEVENT
```

Feed Settings: - `showPatientNames` — Toggle patient visibility for privacy - `includeLeaves` — Include doctor leave periods as blocked events - `branch` — Filter to specific branch only

---

Chapter 11: User Workflows & Journeys

11.1 Patient Booking Workflow

Actors: Patient, System, Doctor

```
1. Patient browses /browse → clicks clinic profile
2. Navigates to doctor profile page (public)
3. Clicks "Book Appointment"
4. IF not logged in → redirects to /patient/register
5. Patient selects date → API: getDoctorAvailability(doctorId, date, branch)
6. System returns available time slots
7. Patient selects slot, enters reason
8. API: createAppointment(data)
9. System validates:
   → Doctor schedule + leaves + branch hours
   → Conflict detection (overlapping appointments)
   → Booking policy (min notice, max advance days)
10. IF clinic autoConfirm → status = CONFIRMED
    ELSE → status = REQUESTED (awaits staff confirmation)
11. System schedules BullMQ reminder jobs (24h, 1h before)
12. System notifies doctor + front-desk via WebSocket + push
13. Patient sees confirmation on /patient/appointments
```

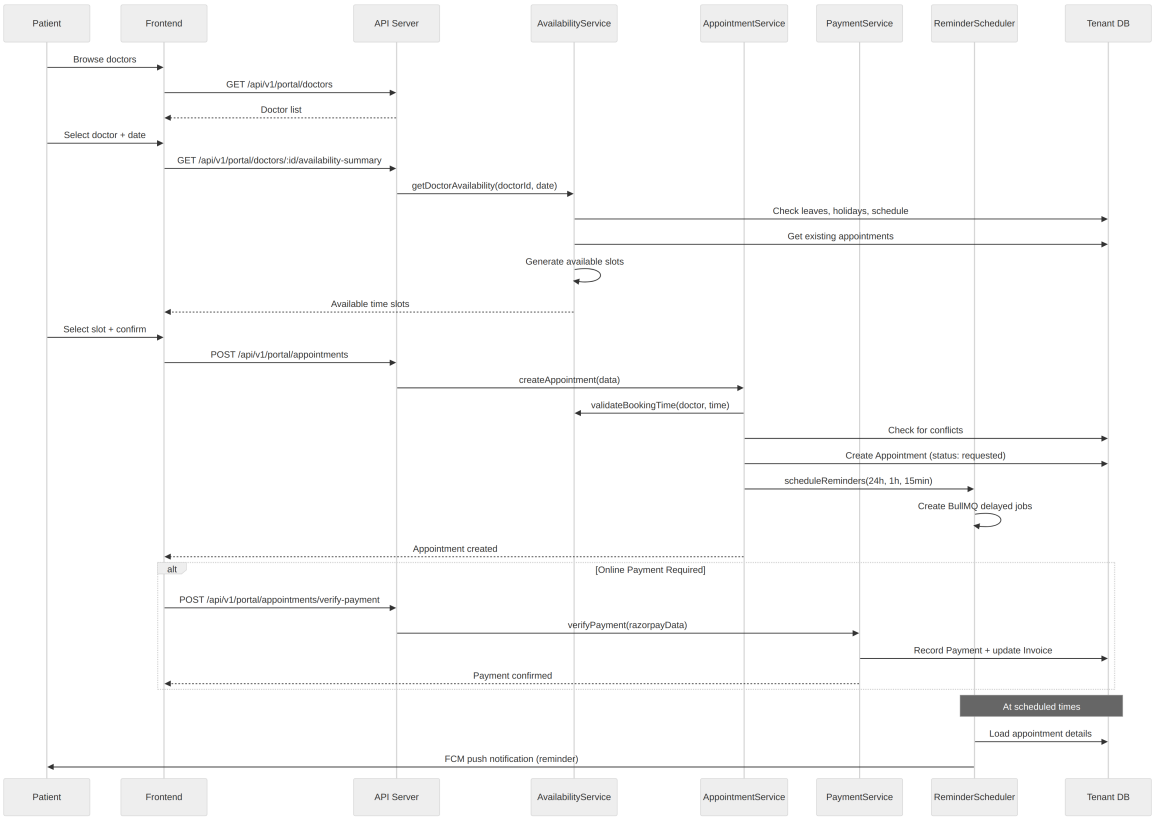


Figure 11: Diagram 13: Patient Booking Workflow

11.2 In-Clinic Consultation Workflow

Actors: Receptionist, Doctor, Patient, System

```
PHASE 1 — CHECK-IN (Receptionist):
  1. Patient arrives at clinic
  2. Receptionist opens /clinic/receptionist-queue
  3. Checks in patient → appointment status → CHECKED_IN
  4. System auto-queues patient (QueueEntry: WAITING)
  5. Live queue display updates via WebSocket

PHASE 2 — QUEUE & CALL:
  6. Doctor views /clinic/doctor-dashboard → sees waiting queue
  7. Receptionist or doctor calls patient → callQueuePatient(queueId, roomId)
  8. System assigns room (auto or manual selection)
  9. Room status → occupied
  10. Patient receives SMS/push notification: "Please proceed to Room 3"
  11. QueueEntry status → IN_CONSULTATION

PHASE 3 — CONSULTATION (Doctor):
  12. Doctor opens /clinic/doctor-consultation/:appointmentId
  13. Loads consultation form (SOAP template + dental chart)
  14. API: startConsultation(data)
  15. System creates TreatmentRecord, status → in_progress
  16. Doctor fills in:
      - Chief complaint + medical history
      - Examination (vitals, oral findings)
      - Assessment (diagnosis, severity)
      - Plan (prescriptions, care tasks, follow-up)
      - Materials used (from inventory)
  17. Auto-save every 30 seconds → updateConsultation(id, data)
  18. Doctor signs digitally and clicks "Complete"
  19. API: completeConsultation(id, data)

PHASE 4 — POST-CONSULTATION (System):
  20. System deducts inventory (materialsUsed)
  21. System auto-creates Prescription (if medications exist)
  22. System generates CareReminders (medication adherence)
  23. System releases consultation room
  24. QueueEntry status → COMPLETED
  25. Appointment status → COMPLETED
  26. System enqueues auto-invoice generation job
  27. Patient receives care summary push notification
  28. All staff receive WebSocket updates
```

11.3 Telemedicine Workflow

Actors: Patient, Doctor, System

```
PHASE 1 — SESSION CREATION:
  1. Patient or doctor initiates video session
  2. API: createVideoSession({ type: SCHEDULED|EMERGENCY, patientId, doctorId })
  3. System generates roomId (tm-{tenant}-{uuid})
  4. Status → REQUESTED or SCHEDULED
  5. Doctor receives WebSocket + FCM push notification
```

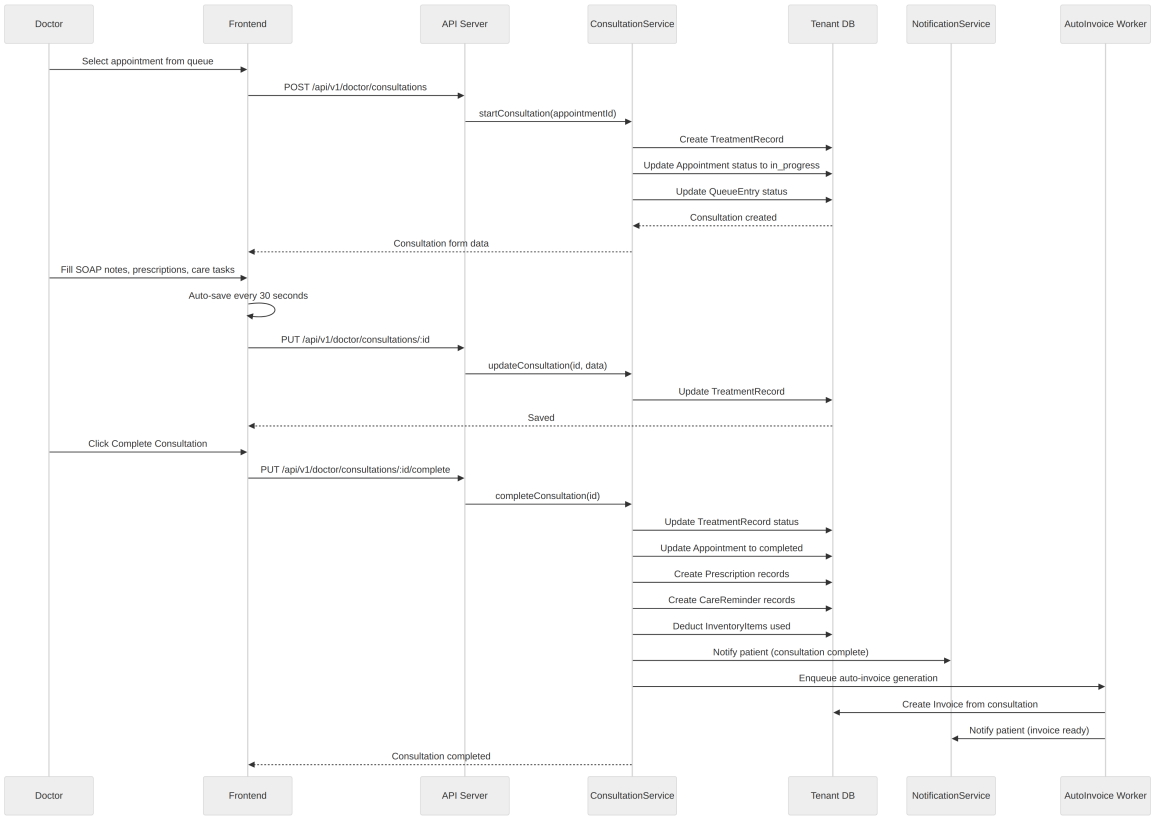


Figure 12: Diagram 12: Doctor Consultation Workflow

```
PHASE 2 — CONNECTION:
  6. Doctor accepts → acceptVideoSession(id) → status → WAITING
  7. Both parties join → joinVideoSession(id)
  8. System generates AES-256-GCM encrypted room tokens
  9. Client establishes WebRTC peer connection via Socket.IO signaling
  10. Socket events: video:offer → video:answer → video:ice-candidate
  11. Status → ACTIVE when both peers connected

PHASE 3 — IN-SESSION:
  12. Video/audio streaming via WebRTC (peer-to-peer)
  13. Encrypted chat via sendVideoMessage() (AES-256-GCM)
  14. File sharing via shareFile()
  15. Connection diagnostics logged via logConnectionEvent()
  16. Session extendable (+15min, max 2 extensions)
  17. Recording consent toggleable

PHASE 4 — COMPLETION:
  18. Doctor ends session → endVideoSession(id, data)
  19. Status → ENDED, records totalDurationSeconds
  20. Billing calculation triggered if appointment linked
  21. Both parties notified
```

11.4 Billing & Payment Workflow

Actors: Receptionist, Billing Officer, Accountant, Patient, System

```
PHASE 1 — INVOICE CREATION:
```

```
  1. Auto-generated after consultation completes (BullMQ job)
     OR manually created by billing officer
  2. Billing officer opens /clinic/billing-officer-invoices
  3. Selects patient, adds line items from service catalog
  4. System resolves ServiceItem defaults (price, HSN code, tax rate)
  5. System calculates GST (CGST+SGST or IGST based on supply type)
  6. API: createEnhancedInvoice(data)
  7. Invoice status = DRAFT

PHASE 2 — PAYMENT COLLECTION:
  8. Send invoice to patient (status → SENT)
  9. Patient pays via:
     a. Cash (receptionist records) → recordPaymentForInvoice()
     b. Card/UPI (Razorpay gateway) → createGatewayOrder() → verifyGatewayPayment()
     c. Insurance (claim submitted) → createInsuranceClaim()
  10. System atomically updates invoice:
      → $inc paidAmount
      → Auto-determines status (PARTIALLY_PAID or PAID)
  11. PatientLedger entry created within MongoDB transaction
  12. FinancialAuditLog recorded with before/after snapshots

PHASE 3 — CASH REGISTER:
  13. Billing officer opens register → openCashRegister()
  14. Cash payments linked to register throughout the day
  15. End of day: closeCashRegister(id, closingBalance)
  16. System calculates variance:
      expected = opening + cashIn - cashOut
      variance = closing - expected
  17. IF |variance| > Rs.100 → managerApproval.required = true
  18. Manager reconciles → reconcileCashRegister()

PHASE 4 — REPORTING:
  19. Day-end report generated
  20. GST summary available for filing
  21. Doctor revenue breakdown
  22. Collection metrics and outstanding reports
  23. Financial audit trail for compliance
```

11.5 Emergency Triage Workflow

Actors: Patient/Receptionist, All On-Duty Doctors, Accepting Doctor, System

```
1. Emergency created (patient SOS or receptionist escalation)
2. System auto-calculates priority:
   → pain scale (1-10) + symptom keyword matching
   → Result: critical / high / medium / low
3. BROADCAST: All on-duty doctors receive alert
   → WebSocket: emergency:new event
   → FCM push: high-priority notification
   → Doctor dashboard shows animated red pulse
4. First doctor to accept claims emergency
   → acceptEmergencySession(id) → status = ACTIVE
   → Other doctors' alerts auto-expire
5. Doctor can decline (removed from their view, others still see)
6. Consultation proceeds with priority handling
7. endEmergencySession(id, data) → records outcome
```

```
8. Patient notified of resolution
```

11.6 Schedule Management Workflow

Actors: Admin, Doctor, System

```
ADMIN WORKFLOW:
  1. Navigate to /clinic/doctor-availability
  2. View all doctors' availability matrix (month view)
  3. Edit doctor schedule: weekly template + slot duration + buffer
  4. Detect conflicts: detectScheduleConflicts(data)
  5. Bulk operations: reschedule/cancel affected appointments

DOCTOR SELF-SERVICE:
  1. Navigate to /clinic/doctor-schedule
  2. View personal weekly schedule
  3. Request leave → goes to manager approval queue
  4. Create one-off override (extra hours, early leave)
  5. Generate ICS calendar feed for external calendar sync

SYSTEM AVAILABILITY COMPUTATION:
  1. Fetch doctor's weekly schedule template
  2. Apply schedule overrides for specific dates
  3. Subtract approved leaves
  4. Subtract clinic holidays (branch-specific + clinic-wide)
  5. Subtract existing appointments (confirmed/checked_in/in_progress)
  6. Apply booking policy (min notice, max advance days)
  7. Generate available slot array
```

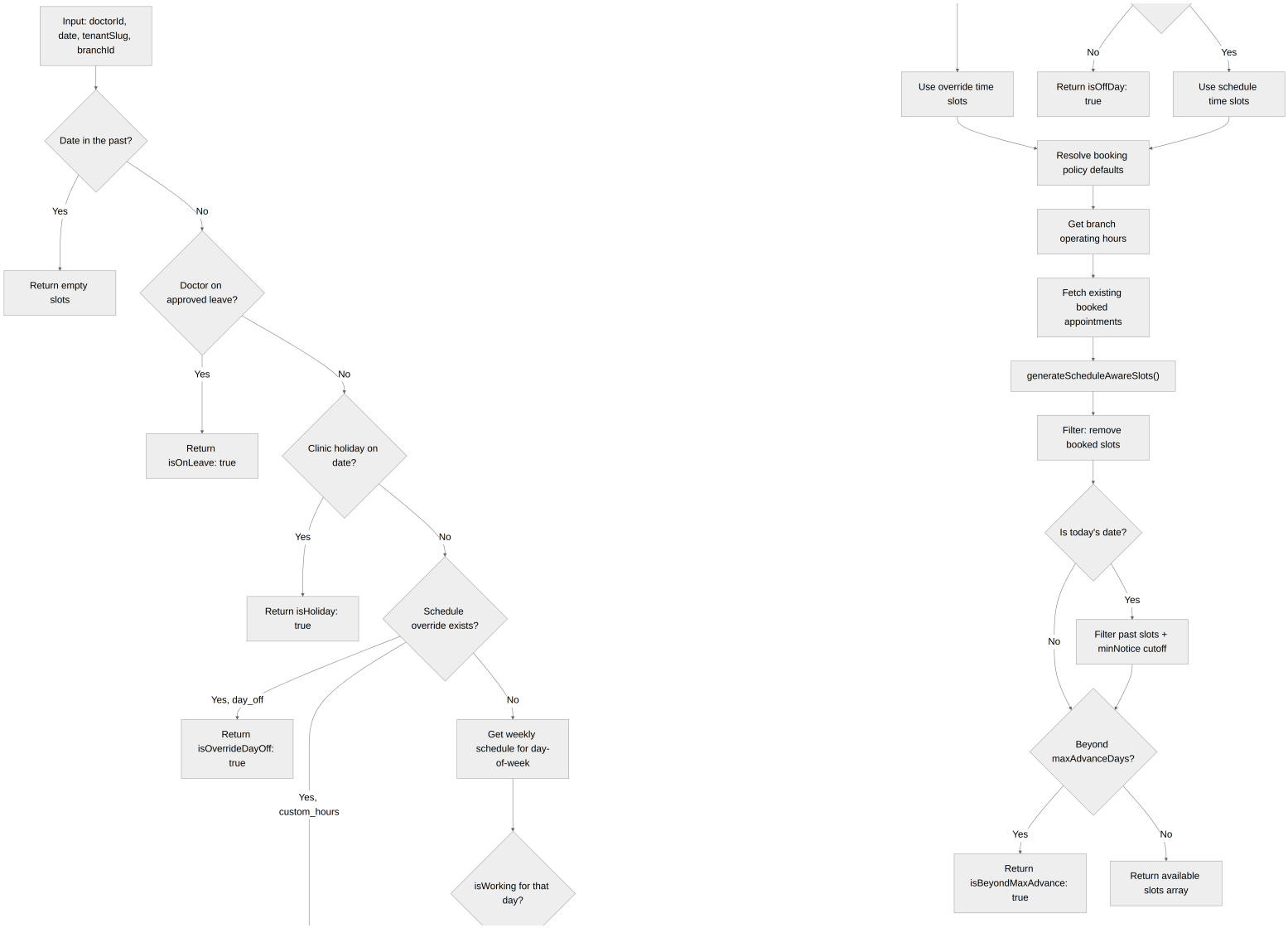


Diagram 16: Availability Calculation Algorithm

11.7 Inventory & Procurement Workflow

Actors: Admin, Doctor, System

```
RESTOCK WORKFLOW:
  1. System flags low-stock items (currentStock ≤ minimumThreshold)
  2. Admin views /clinic/inventory → "Low Stock" filter
  3. Admin creates Purchase Order (vendor + items + quantities)
  4. PO status: REQUESTED → APPROVED (by manager)
  5. Vendor delivers → PO status: DELIVERED
  6. System AUTO-RESTOCKS:
     → Creates stock_in InventoryTransaction for each item
     → Increments currentStock on InventoryItem
     → Audit trail with PO reference
  7. PO status: RECEIVED (final verification)

CONSUMPTION WORKFLOW:
  1. Doctor selects materials during consultation
  2. On completeConsultation():
     → Decrements InventoryItem.currentStock
     → Creates stock_out InventoryTransaction
     → Links to TreatmentRecord for traceability
  3. Low-stock alerts trigger automatically
```

Part IV — Technical Implementation Details

Chapter 12: API Architecture

12.1 API Design Principles

BestoClinic follows a RESTful API design with consistent patterns across all 43 route modules:

- **Base URL:** `/api/v1` — All routes mounted under versioned prefix
- **Response Format:** Standardized via `ApiResponse` utility class
- **Authentication:** JWT Bearer tokens in Authorization header
- **Tenant Isolation:** `X-Tenant-ID` header for tenant-scoped requests
- **Pagination:** Query params `page` (default: 1), `limit` (default: 20)
- **Soft Deletes:** All DELETE operations set `deletedAt` timestamp
- **Validation:** Joi schemas via `validate()` middleware
- **Error Handling:** `asyncHandler` wrapper + `ApiError` custom class

Standard Response Shapes:

```
// Success
{
  "success": true,
  "message": "Patients fetched",
  "data": [...],
  "meta": { "page": 1, "limit": 20, "total": 150, "totalPages": 8, "hasNextPage": true, "hasPrevPage": false }
}

// Error
{
  "success": false,
  "message": "Validation failed",
  "errors": [{ "field": "email", "message": "Email is required" }]
}
```

12.2 Complete Route Registry

The application mounts 43 route modules under `/api/v1`. The global subscription enforcement middleware blocks suspended/cancelled tenants from all routes except auth, public, platform, monitoring, and health.

Route Mounting Order (from `src/routes/index.js`):

Mount Path	Module	Description
`/health`	—	Health check (no auth)
`/auth`	auth	Authentication (login, register, refresh, logout)
`/patients`	patient	Patient CRUD + bulk import
`/appointments`	appointment	Scheduling + availability
`/invoices`	invoice	Invoice CRUD + PDF
`/clinic`	clinic	Clinic settings + logo
`/users`	user	User management
`/subscriptions`	subscription	Subscription lifecycle + Razorpay
`/dashboard`	dashboard	Aggregated clinic dashboard
`/branches`	branch	Branch CRUD + monitoring + rooms
`/departments`	department	Department CRUD
`/holidays`	holiday	Holiday calendar

Mount Path	Module	Description
`/booking-policy`	bookingPolicy	Booking policy settings
`/doctors`	doctor	Doctor CRUD + schedule + availability
`/staff`	staff	Staff CRUD + role management
`/insurance`	insurance	Providers + claims
`/payroll`	payroll	Salary structures + payslips
`/notifications`	notification	In-app notifications
`/legacy-uploads`	legacyUpload	CSV data import
`/integrations`	integration	External system sync
`/roles`	role	Role/permission viewer
`/queue`	queue	Smart queue management
`/prescriptions`	prescription	Prescription CRUD
`/consents`	consent	Patient consent management
`/emergency`	emergency	Emergency sessions
`/doctor`	doctorDashboard	Doctor dashboard + consultations
`/integration-data`	integrationViewer	Integration data viewer
`/portal`	patientPortal	Patient self-service portal
`/telemedicine`	telemedicine	Video sessions + presence
`/payments`	payment	Payment processing + gateway
`/billing`	billing	Enhanced invoicing + analytics
`/cash-register`	cashRegister	Cash drawer management
`/credit-notes`	creditNote	Credit memo management
`/dental-scans`	dentalScan	AI dental analysis
`/inventory`	inventory	Stock management
`/purchase-orders`	purchaseOrder	Procurement + vendors
`/analytics`	analytics	Clinic analytics + compliance
`/jobs`	jobPortal	Job postings + applications
`/credits`	credit	Credit wallet + purchases
`/calendar-feeds`	calendarFeed	ICS export
`/global-search`	globalSearch	Cross-entity search
`/public`	public	Public routes (no auth)
`/platform`	productOwner	Platform admin routes

12.3 API Endpoint Summary by Category

Total API Endpoints: ~250+

Category	Endpoint Count	Modules
Authentication & Users	~15	auth, user
Patient Management	~10	patient
Appointment & Scheduling	~20	appointment, doctor (schedule), bookingPolicy, holiday
Doctor Dashboard & Consultation	~35	doctorDashboard
Queue Management	~10	queue

Category	Endpoint Count	Modules
Billing & Payments	~30	invoice, payment, billing, cashRegister, creditNote
Insurance	~13	insurance
Prescriptions & Care	~15	prescription, doctorDashboard (care tasks)
Telemedicine	~17	telemedicine
Emergency	~7	emergency
Dental Scans (AI)	~10	dentalScan
Inventory & Procurement	~20	inventory, purchaseOrder
Clinic Administration	~25	clinic, branch, department, staff, role
Analytics & Reports	~15	analytics, dashboard, billing (reports)
Patient Portal	~30	patientPortal
Notifications	~5	notification
Subscription	~5	subscription
Calendar & Feeds	~7	calendarFeed
Search	~3	globalSearch
Job Portal & Credits	~15	jobPortal, credit
Platform Admin	~15	productOwner
Integration	~8	integration, integrationViewer
Other	~5	legacyUpload, consent, public

12.4 Middleware Execution Chain

Every API request passes through a layered middleware chain:

```
1. Express built-in:
   → cors() — CORS headers
   → express.json({ limit: '50mb' }) — Body parsing
   → express.urlencoded({ extended: true })
   → helmet() — Security headers

2. Request logging:
   → morgan('combined') — HTTP access logging

3. Rate limiting:
   → express-rate-limit — Per-IP rate limiting

4. Static files:
   → express.static('uploads') — Local file serving

5. Route-level middleware (per route):
   → subscriptionActive() — Blocks suspended tenants
   → authenticate() — JWT verification
   → tenantResolver() — Resolves tenant DB connection
```

```
   → authorize(...permissions) — RBAC permission check
   → featureGuard(feature) — Plan feature check
   → validate(schema) — Joi request validation
   → upload(config) — Multer file upload (where needed)

6. Error handling:
   → errorHandler() — Centralized error response
```

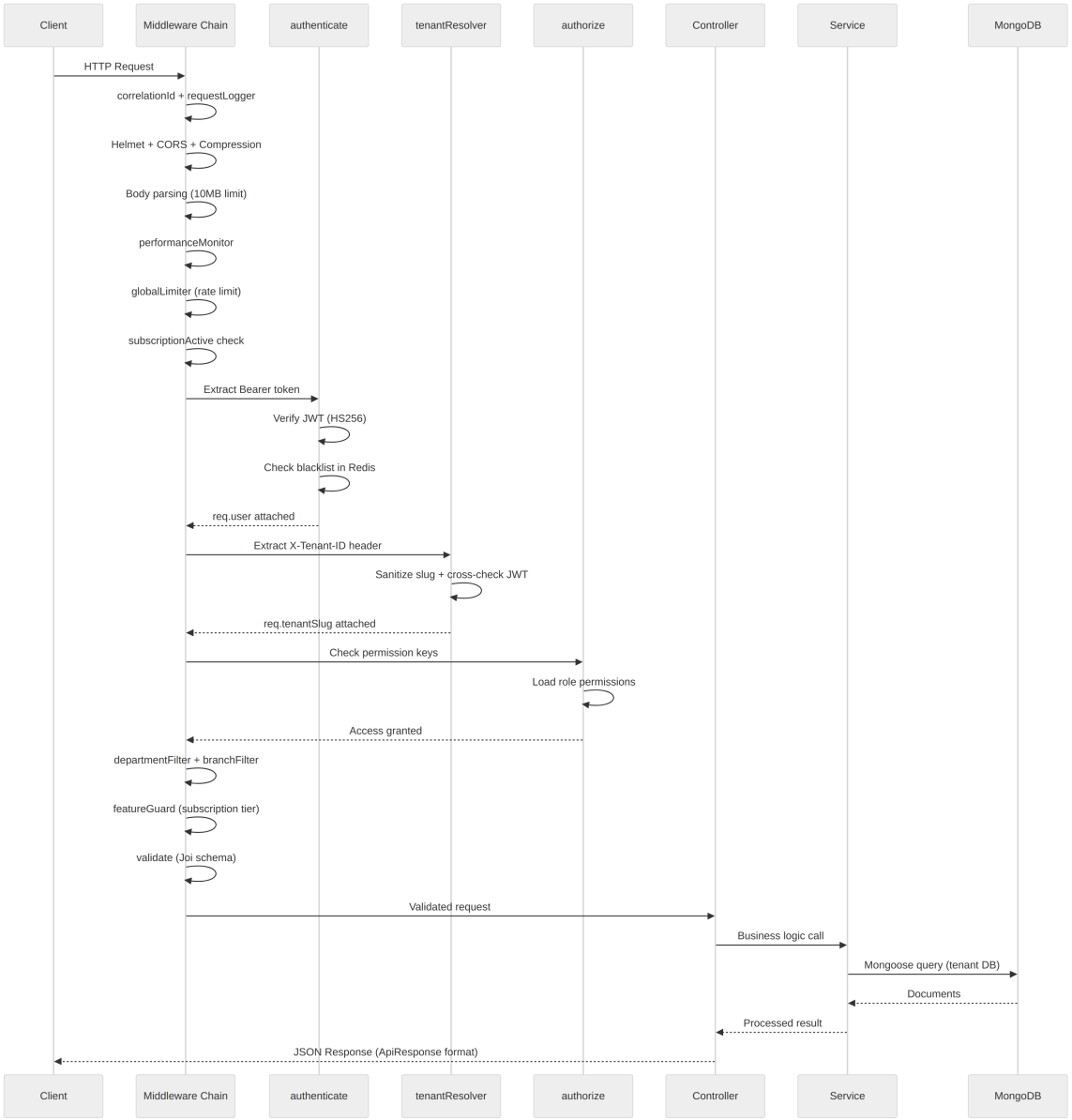


Figure 13: Diagram 3: Request Lifecycle

Chapter 13: Multi-Tenancy Implementation

13.1 Architecture Overview

BestoClinic implements a **database-per-tenant** isolation model. Each clinic (tenant) has its own MongoDB database, while a shared platform database stores cross-tenant entities (clinics, subscriptions, plans, platform users).

Database Topology:

```
Platform Database (shared):
  → clinics, subscriptions, plans, platformusers
  → provisioningstatuses, userdirectories
  → featureflags, integrations, promotions
  → jobpostings, jobapplications, creditwallets
  → guestscans, prescriptionscans, calendarfeedtokens

Tenant Database (per clinic):
  → users, patients, appointments, treatmentrecords
  → invoices, payments, prescriptions, inventoryitems
  → ... (53 collections per tenant)
```

13.2 Tenant Connection Management

Location: `src/db/tenantConnectionManager.js`

The tenant connection manager implements a connection pool with LRU eviction and encrypted URI storage.

Connection Pool Algorithm:

```
getTenantConnection(tenantSlug):
  1. Check in-memory pool:
     IF connection exists AND is connected:
       → Update lastUsed timestamp (LRU touch)
       → Return cached connection

  2. Resolve tenant database URI:
     → Fetch Clinic document from platform DB
     → Decrypt dbUri using AES-256-GCM
     → Cache decrypted URI in Redis (TTL: 1 hour)

  3. Create new Mongoose connection:
     → mongoose.createConnection(decryptedUri, options)
     → Register all 53 tenant models on new connection
     → Store in pool with { connection, lastUsed, slug }

  4. Pool size management:
     IF pool.size > MAX_POOL_SIZE:
       → Evict least-recently-used connections
       → Close evicted connections gracefully

  RETURN connection
```

(Algorithm location: `src/db/tenantConnectionManager.js:getTenantConnection`)

Configuration: - `MAX_POOL_SIZE` — Maximum concurrent tenant connections (configurable, default: 50) - LRU eviction when pool is full - Idle connection pruning every 10 minutes (via `pruneIdleConnections`)

13.3 Tenant Resolution Middleware

Location: `src/middleware/tenantResolver.js`

The `tenantResolver()` middleware runs after authentication to establish the tenant context:

```
tenantResolver(req, res, next):
  1. Extract tenantSlug:
     → From req.user.tenantSlug (JWT payload) — primary source
     → From X-Tenant-ID header — fallback
  2. Validate tenantSlug is present
  3. Call getTenantConnection(tenantSlug)
  4. Attach to request: req.tenantConnection = connection
  5. Attach to request: req.tenantSlug = tenantSlug
  6. Call next()
```

All downstream service calls use `req.tenantConnection` to create Mongoose models scoped to the correct tenant database.

13.4 Database URI Encryption

Tenant database URIs are stored encrypted in the platform database to prevent URI exposure in case of platform DB compromise:

- **Algorithm:** AES-256-GCM
- **Key:** `ENCRYPTION_KEY` environment variable
- **Encryption:** On tenant provisioning (createClinic)
- **Decryption:** On tenant connection establishment
- **Caching:** Decrypted URIs cached in Redis with 1-hour TTL to avoid repeated decryption

13.5 Tenant Provisioning Pipeline

Location: `src/modules/productOwner/`

When a new clinic registers, the provisioning pipeline executes:

```
Step 1: PENDING — Validate registration data
Step 2: CREATING_DB — Create new MongoDB database for tenant
Step 3: GENERATING_CLINIC_CODE — Generate unique clinic slug/code
Step 4: SEEDING — Seed default data (roles, booking policy, service items)
Step 5: CREATING_ADMIN — Create super_admin user for clinic
Step 6: APPLYING_PLAN — Apply subscription plan features
Step 7: COMPLETED — Tenant ready for use

On failure at any step:
Step 8: FAILED — Record error details
Step 9: ROLLED_BACK — Cleanup partial provisioning
```

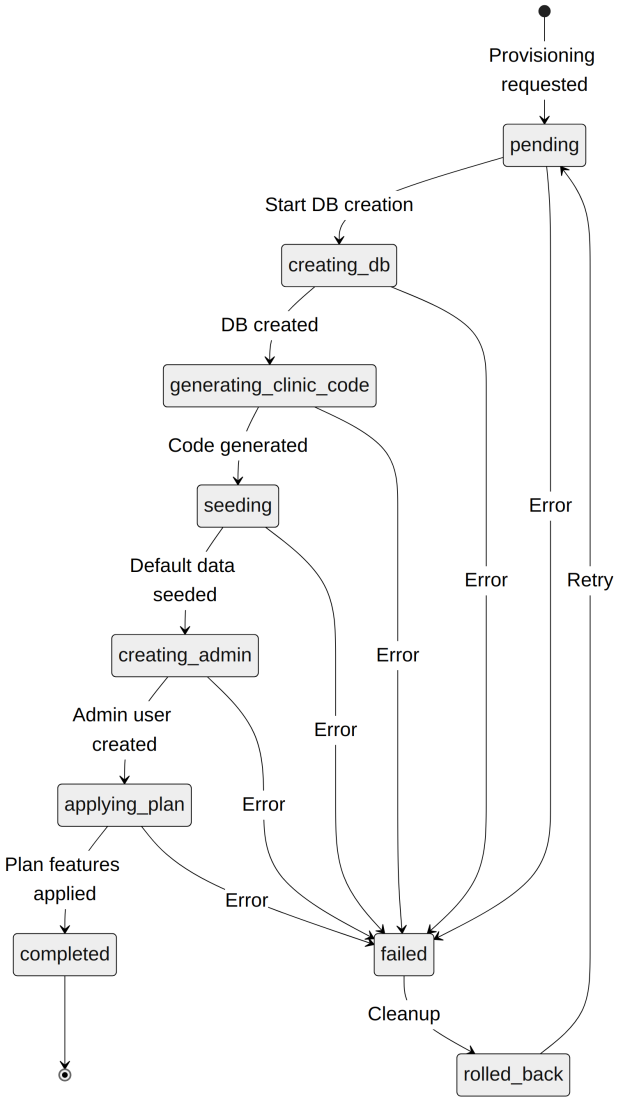


Diagram 11: Tenant Provisioning Flow

Each step updates the `ProvisioningStatus` document with timestamps and step details, enabling progress tracking and failure diagnostics.

13.6 Cross-Tenant Operations

Some operations require cross-tenant data access:

Operation	Mechanism
Email-first login	UserDirectory (platform) maps email → tenantSlug
Calendar feeds (public)	CalendarFeedToken (platform) maps token → tenantSlug
Patient portal login	PlatformPatient stores connectedClinics array
Global search (platform)	Queries platform DB directly
Job portal (public)	JobPostings stored in platform DB
Guest dental scans	GuestScan stored in platform DB
Feature flags	FeatureFlag (platform) with per-tenant overrides

Chapter 14: Security & Authentication

14.1 Authentication Architecture

Location: `src/modules/auth/`, `src/middleware/authenticate.js`

BestoClinic implements a JWT-based authentication system with dual-token rotation and per-device tracking.

Token Configuration:

Token Type	Secret	Expiry	Purpose
Access Token	`JWT_SECRET`	15 minutes	API request authentication
Refresh Token	`JWT_REFRESH_SECRET`	7 days	Access token renewal
Platform Token	`JWT_PLATFORM_SECRET`	1 hour	Platform admin authentication

Authentication Flow:

```
LOGIN:
  1. Validate email + password (bcrypt compare, 12 rounds)
  2. Generate access token (15m) with payload:
     { userId, role, tenantSlug, email, permissions[] }
  3. Generate refresh token (7d) with payload:
     { userId, tenantSlug, deviceId }
  4. Store refresh token in Redis:
     Key: refresh:{tenantSlug}:{userId}:{deviceId}
     Value: { token, createdAt, userAgent }
     TTL: 7 days
  5. Return both tokens to client

TOKEN REFRESH:
  1. Verify refresh token signature
  2. Check token exists in Redis (not revoked)
  3. TOKEN REUSE DETECTION:
     IF token already used (rotated):
       → Revoke ALL tokens for this user (security breach)
       → Force re-login on all devices
  4. Generate new access + refresh token pair
  5. Delete old refresh token from Redis
  6. Store new refresh token in Redis
  7. Return new token pair

LOGOUT:
  1. Add access token to Redis blacklist (TTL: remaining token life)
  2. Delete refresh token from Redis
  3. Clear client-side tokens
```

(Algorithm location: `src/modules/auth/auth.service.js`)

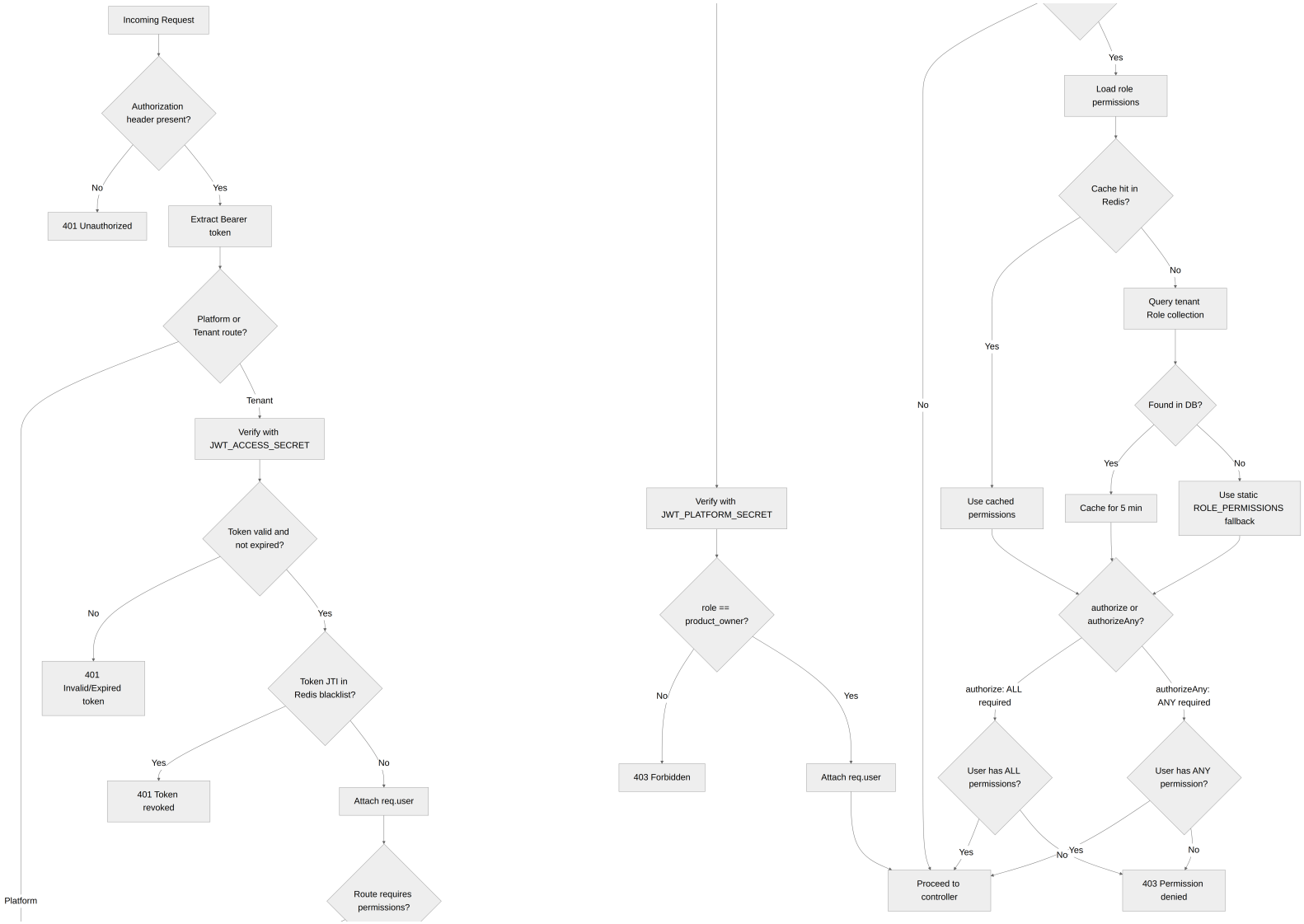


Diagram 4: Authentication and Authorization Flow

14.2 Authorization (RBAC)

Location: `src/middleware/authorize.js`, `src/config/permissions.js`

The authorization system implements Role-Based Access Control with 80+ permission keys across 14 roles.

Permission Resolution:

```
authorize(...requiredPermissions)(req, res, next):
  1. Extract user role from JWT payload
  2. Resolve role permissions:
     → Check tenant DB for custom role overrides (cached 5min)
     → Fallback to static ROLE_PERMISSIONS[role] from config
  3. Check if user has ANY required permission:
     → Direct match: 'patient:read'
     → OWN variant: 'patient:read_own' (scoped access)
  4. IF permission found → next()
     ELSE → 403 Forbidden
```

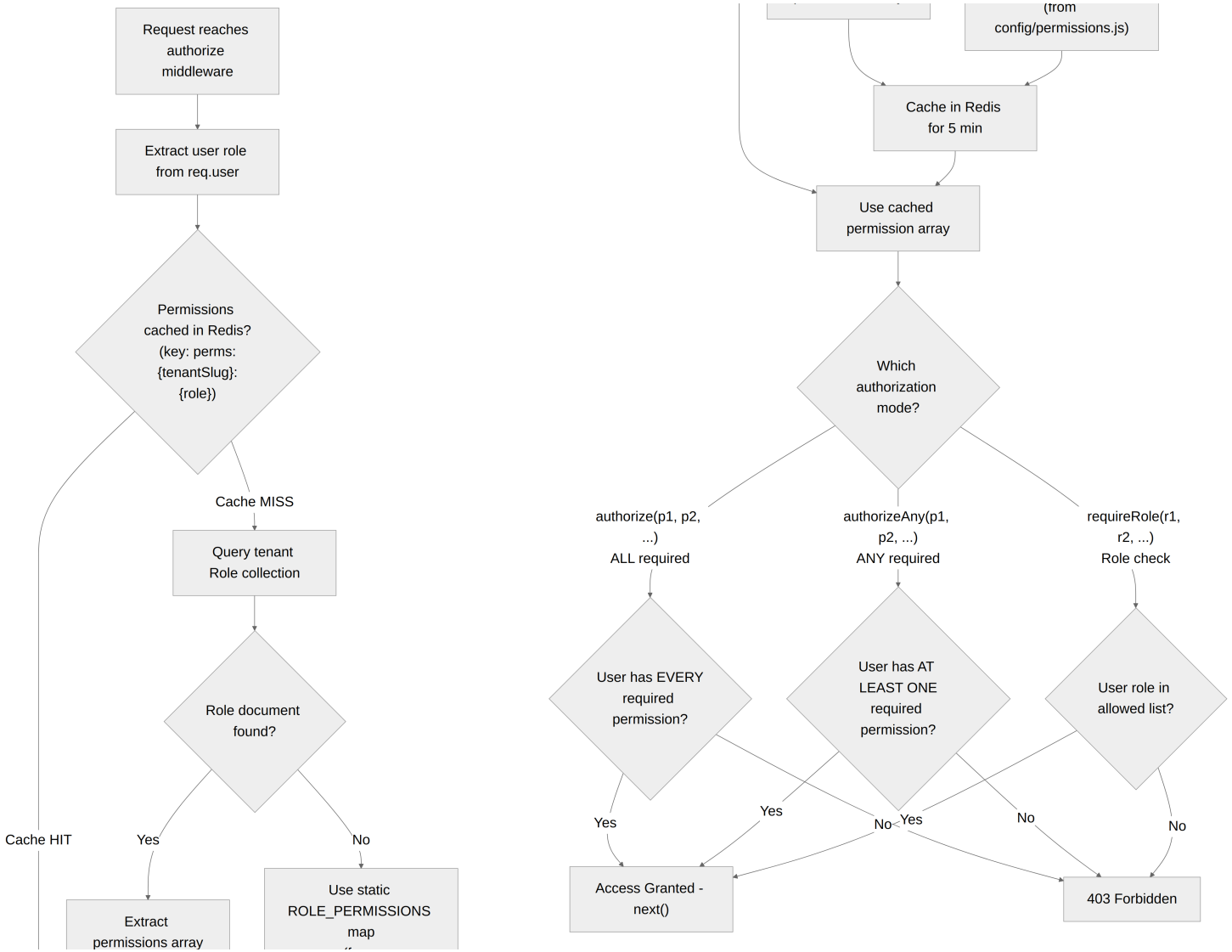


Diagram 18: Permission Check Flow

Permission Key Format: `module:action`

Examples: `patient:read`, `patient:read_own`, `appointment:create`, `invoice:void`, `cash_register:reconcile`

Self-Scope Enforcement:

When a user has only the `*_OWN` variant of a permission (e.g., `patient:read_own` but not `patient:read`), service-layer logic restricts queries to the user's own data:

- Patients: filter by `user: req.user.userId`
- Appointments: filter by `doctor: req.user.userId` (for doctors)
- Invoices: filter by linked patient's user ID
- Prescriptions: filter by patient ownership

14.3 Role Hierarchy

The 14 system roles are organized in a hierarchy that governs role creation and management:

```
Level 1 (Highest): product_owner
Level 2: super_admin
Level 3: admin
Level 4: manager, department_manager
Level 5: doctor, billing_officer, accountant, hr_payroll_manager, it_support
Level 6: receptionist, support_staff
Level 7: guardian
Level 8 (Lowest): patient
```

Hierarchy Rules: - Users cannot create roles at or above their own level - Users cannot modify roles at or above their own level - Role escalation attempts are blocked at the service layer - Self-deactivation is prevented

14.4 Subscription Enforcement

Location: `src/middleware/featureGuard.js`

Two middleware layers enforce subscription status and plan features:

`subscriptionActive()` — Global middleware on all tenant routes:

```
1. Extract tenantSlug from X-Tenant-ID header or JWT
2. Fetch Clinic from platform DB
3. Check clinic.status:
   IF suspended OR cancelled → 403 SUBSCRIPTION_BLOCKED
   IF active OR pending → next()
4. Skip check for auth, public, platform, monitoring routes
```

`featureGuard(featureKey)` — Per-route feature gating:

```
1. Fetch clinic.features from platform DB
2. Check features[featureKey]:
   IF false or undefined → 403 Feature not available on current plan
   IF true → next()
```

Feature keys checked: `teleHealthEnabled`, `advancedReports`, `customBranding`, `apiAccess`, `externalIntegrations`, `jobPortalEnabled`, `whiteLabeling`, `smsEnabled`

14.5 Security Headers & Rate Limiting

Helmet Configuration: - Content-Security-Policy headers - X-Frame-Options: DENY - X-Content-Type-Options: nosniff - Strict-Transport-Security (HSTS)

Rate Limiting: - Per-IP request limiting via `express-rate-limit` - Configurable window and max requests - Applied globally before route handlers

14.6 Data Encryption

Data Type	Algorithm	Key Source	Location
Tenant DB URIs	AES-256-GCM	ENCRYPTION_KEY env	tenantConnectionManager
Telemedicine room tokens	AES-256-GCM	TELEMEDICINE_SECRET env	roomToken.service
Telemedicine chat messages	AES-256-GCM	TELEMEDICINE_SECRET env	roomToken.service
Integration credentials	AES-256-CBC	ENCRYPTION_KEY env	encryption.service
User passwords	bcrypt (12 rounds)	—	User model pre-save hook
Razorpay webhooks	HMAC-SHA256	RAZORPAY_WEBHOOK_SECRET	payment.service

14.7 Audit & Compliance

The system maintains comprehensive audit trails:

Audit Type	Model	Purpose
Financial operations	FinancialAuditLog	Before/after state for payments, invoices, cash register
Patient record access	AccessLog	Who accessed which patient record, when, from where
Tenant operations	TenantAuditLog	Platform-level tenant lifecycle events
Inventory movements	InventoryTransaction	Stock in/out with reason and reference
Credit transactions	CreditTransaction	Credit wallet movements with running balance
Consent management	Consent	Patient consent grants and revocations with IP addresses
Compliance alerts	ComplianceAlert	Care task compliance violations

Chapter 15: Integration Points

15.1 Razorpay (Payment Gateway)

Purpose: Online payment processing for patient invoices and subscription billing.

Integration Points:

Feature	API Used	Location
Payment order creation	`razorpay.orders.create()`	payment.service
Payment verification	HMAC signature validation	payment.service
Subscription creation	`razorpay.subscriptions.create()`	subscription.service
Plan management	`razorpay.plans.create()`	subscription.service
Refund processing	`razorpay.payments.refund()`	payment.service
Webhook handling	POST `/payments/gateway/webhook`	payment.routes
Subscription webhooks	POST `/subscription/webhook`	subscription.routes

Webhook Events Handled: - `payment.captured` — Payment successful - `payment.failed` — Payment failed - `subscription.charged` — Subscription renewal - `subscription.updated` — Plan change - `subscription.ended` — Subscription ended - `refund.processed` — Refund completed

Security: - Webhook signature verification: `crypto.createHmac('sha256', secret).update(body).digest('hex` - Idempotent processing via `WebhookEvent` model (eventId uniqueness) - No sensitive data stored in client-side code

15.2 Google Gemini AI (Dental Analysis)

Purpose: AI-powered dental image analysis for cavity detection, plaque identification, and treatment recommendations.

Integration Points:

Feature	API Used	Location
Single image analysis	`generativeModel.generateContent()`	aiAnalysis.service
Multi-image analysis	`generativeModel.generateContent()`	aiAnalysis.service
Frame validation	`generativeModel.generateContent()`	aiAnalysis.service
Tooth detection	`generativeModel.generateContent()`	aiAnalysis.service

Model: `gemini-2.5-flash` (Google Generative AI SDK)

JSON Repair Pipeline (5-stage fallback):

```
Stage 1: Direct JSON.parse() of response
Stage 2: Extract JSON from markdown code blocks (```json...```)
Stage 3: Regex extraction of JSON object pattern
Stage 4: Token truncation repair (close unclosed brackets/braces)
Stage 5: Return raw text as summary with empty findings array
```

(Algorithm location: `src/services/aiAnalysis.service.js`)

Error Handling: - API timeout: 30 seconds - Rate limit backoff: exponential retry - Model unavailable: graceful degradation (scan marked as FAILED)

15.3 Firebase Cloud Messaging (Push Notifications)

Purpose: Mobile and web push notifications for appointments, emergencies, video calls, and care reminders.

Integration Points:

Feature	API Used	Location
User notification	`messaging.sendEachForMulticast()`	fcm.service
Appointment reminders	scheduled via BullMQ → FCM	reminder worker
Incoming call alerts	high-priority VoIP notification	video.service
Emergency broadcasts	broadcast to role group	emergency.service
Care summary push	care task notifications	doctorDashboard.service

Configuration: - Firebase Admin SDK initialized at boot (`src/config/firebase.js`) - Graceful initialization: non-fatal if Firebase not configured - Per-user FCM token array stored in User model - Auto-cleanup of invalid/expired tokens on send failure

15.4 Nodemailer (Email)

Purpose: Transactional email for password resets, appointment confirmations, invoice delivery, and job application notifications.

Integration via BullMQ: - Email jobs enqueued to `emailQueue` - Worker processes asynchronously - Retry with exponential backoff on failure - Templates for common notification types

15.5 AWS S3 (File Storage)

Purpose: Secure file storage for dental scan images, prescription scans, clinic logos, patient documents, and attachments.

Integration Points: - `uploadFile(buffer, key, mimeType)` — Uploads to S3 bucket - `getSignedUrl(key)` — Generates time-limited download URL - `deleteFile(key)` — Removes file from S3 - Size limits: 10MB per dental scan image, 5MB per clinic logo, 50MB per legacy upload

Fallback: Local filesystem storage (`/uploads/`) when S3 is not configured.

15.6 External Dental Imaging Systems (RioView / Romexis)

Purpose: Sync patient imaging data from third-party dental PACS systems.

Integration Architecture:

```
Adapter Pattern:
  getAdapter(integrationType) → returns type-specific sync adapter

Sync Flow:
  1. Fetch encrypted credentials from IntegrationCredential
  2. Route to appropriate adapter
```

```
  3. Adapter fetches data from external system
  4. Data mapper transforms external schema → BestoClinic schema
  5. Deduplication by externalId + externalSource
  6. Upsert into tenant database
  7. Create SyncLog entry

Scheduling:
  BullMQ repeatable job: `cron:{clinicSlug}:{integrationType}`
  Interval: 5-1440 minutes (configurable per integration)
```

Supported Types: `rioview`, `romexis`

15.7 WebSocket (Socket.IO)

Purpose: Real-time bidirectional communication for queue updates, appointment changes, video signaling, emergency broadcasts, and notifications.

Server Configuration (`src/websocket/index.js`): - CORS-enabled Socket.IO server - JWT authentication on connection - Auto-join rooms: `user:{userId}`, `tenant:{slug}`, `tenant:{slug}:{role}s`

Event Categories:

Category	Events	Purpose
Video	`video:offer`, `video:answer`, `video:ice-candidate`, `video:peer-joined`, `video:peer-left`, `video:call-ended`, `video:media-state`, `video:incoming-call`	WebRTC signaling relay
Chat	`chat:message`, `chat:delivered`, `chat:read`, `chat:typing`	In-session messaging
Emergency	`emergency:new`, `emergency:accepted`, `emergency:declined`, `emergency:cancelled`, `emergency:ended`	Emergency triage
Presence	`presence:change`, `presence:heartbeat`	Doctor online status
Notifications	`notification:new`, `notification:unreadCount`, `notification:ack`	Real-time alerts
Appointments	`appointment:update`, `availability:invalidated`	Schedule changes
Queue	`queue:update`	Patient queue changes
Consultation	`consultation:started`, `consultation:completed`	Consultation lifecycle

Client Implementation (`website/src/lib/socket.js`): - Singleton socket instance with auto-reconnect (10 attempts) - Custom subscriber pattern (`_makeSub`) for event delegation - Separate buses: `videoRelay`, `callLife`, `chatBus`, `emergencyBus`

15.8 BullMQ (Job Queues)

Purpose: Asynchronous job processing for tasks that should not block API responses.

Queue Configuration:

Queue Name	Jobs	Worker Location
`dentalScanQueue`	analyze-dental-scan	dentalScanWorker.js
`emailQueue`	send-email, send-bulk	emailWorker.js
`appointmentReminderQueue`	send-reminder	reminderWorker.js
`syncQueue`	data-sync (repeatable)	syncWorker.js
`invoiceQueue`	auto-generate-invoice	invoiceWorker.js
`careReminderQueue`	send-care-reminder	careReminderWorker.js
`subscriptionQueue`	process-webhook	subscriptionWorker.js
`notificationQueue`	batch-push	notificationWorker.js

Worker Lifecycle: - Workers started during boot sequence (Step 4) - Graceful shutdown: workers drained before process exit - Concurrency: configurable per queue - Retry: exponential backoff with configurable max attempts - Dead letter queue for permanently failed jobs

---

Part V — DevOps & Infrastructure

Chapter 16: Deployment Architecture

16.1 Container Architecture

Location: `Dockerfile`, `docker-compose.yml`

BestoClinic uses a multi-stage Docker build with security-hardened configuration:

Dockerfile Specification:

```
Base Image: node:18-alpine
Build Strategy: Multi-stage (builder → production)
User: Non-root (appuser:1001)
Health Check: wget http://localhost:5000/api/v1/health (30s interval)
Exposed Port: 5000
Entry Point: node server.js
```

Key Security Measures: - Non-root user execution (UID 1001) - Alpine base image (minimal attack surface) - Production-only dependencies (`npm ci --omit=dev`) - Health check for container orchestrator awareness

16.2 Service Topology

Docker Compose Services:

Service	Image	Port	Health Check	Purpose
`api`	Custom build	5000	HTTP GET /api/v1/health	Application server
`redis`	redis:7-alpine	6379	`redis-cli ping`	Session store, cache, job queues
`mongo`	mongo:7	27017	`mongosh --eval db.runCommand('ping')`	Platform database

Volume Mounts: - `mongo-data` — Persistent MongoDB storage - `redis-data` — Persistent Redis storage (AOF) - `./uploads` — Local file uploads (dev mode)

Network: All services communicate on the default Docker bridge network.

16.3 Boot Sequence

The server follows a strict ordered boot sequence with fail-fast behavior:

```
Step 1: Environment validation (src/config/env.js)
        → Validates all required env vars present
        → Fails immediately if missing critical config

Step 2: Connect Platform MongoDB
        → Establishes connection to shared platform database
        → Registers platform models

Step 3: Connect Redis
```

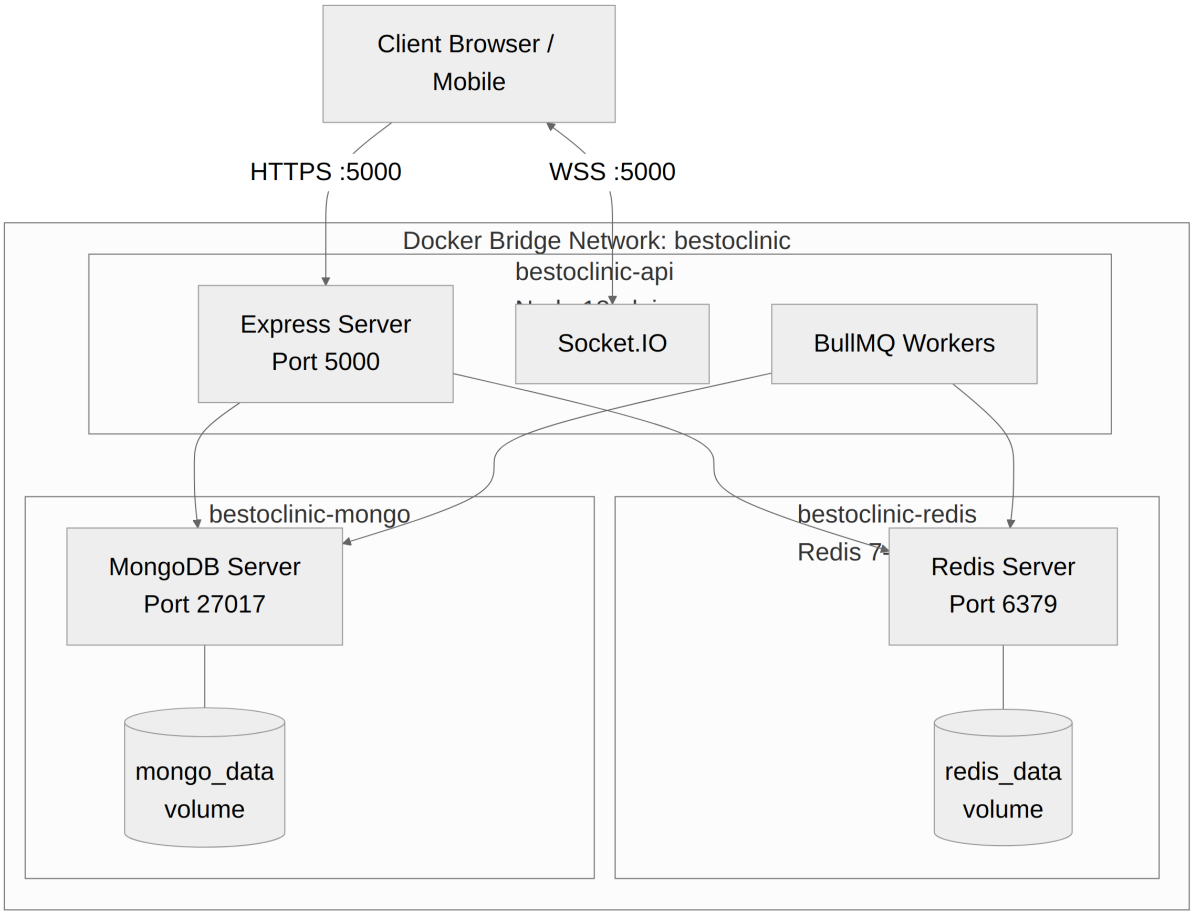


Figure 14: Diagram 14: Deployment Topology

```
        → Establishes Redis connection for sessions, cache, queues

Step 3b: Ensure upload directories exist
        → Creates: uploads/, uploads/logos, uploads/prescription-scans,
          uploads/xrays, uploads/avatars, uploads/documents
        → Non-fatal (S3 mode doesn't need local dirs)

Step 3c: Firebase Admin SDK initialization
        → Graceful if not configured

Step 4: Start BullMQ workers
        → All job queue workers initialized
        → Await readiness before accepting requests

Step 5: Create Express app + HTTP server
        → Mount middleware, routes, error handlers

Step 6: Initialize WebSocket (Socket.IO)
        → Attach to HTTP server

Step 7: Start listening on PORT (default: 5000)

Step 8: Schedule idle connection pruning (every 10 min)
```

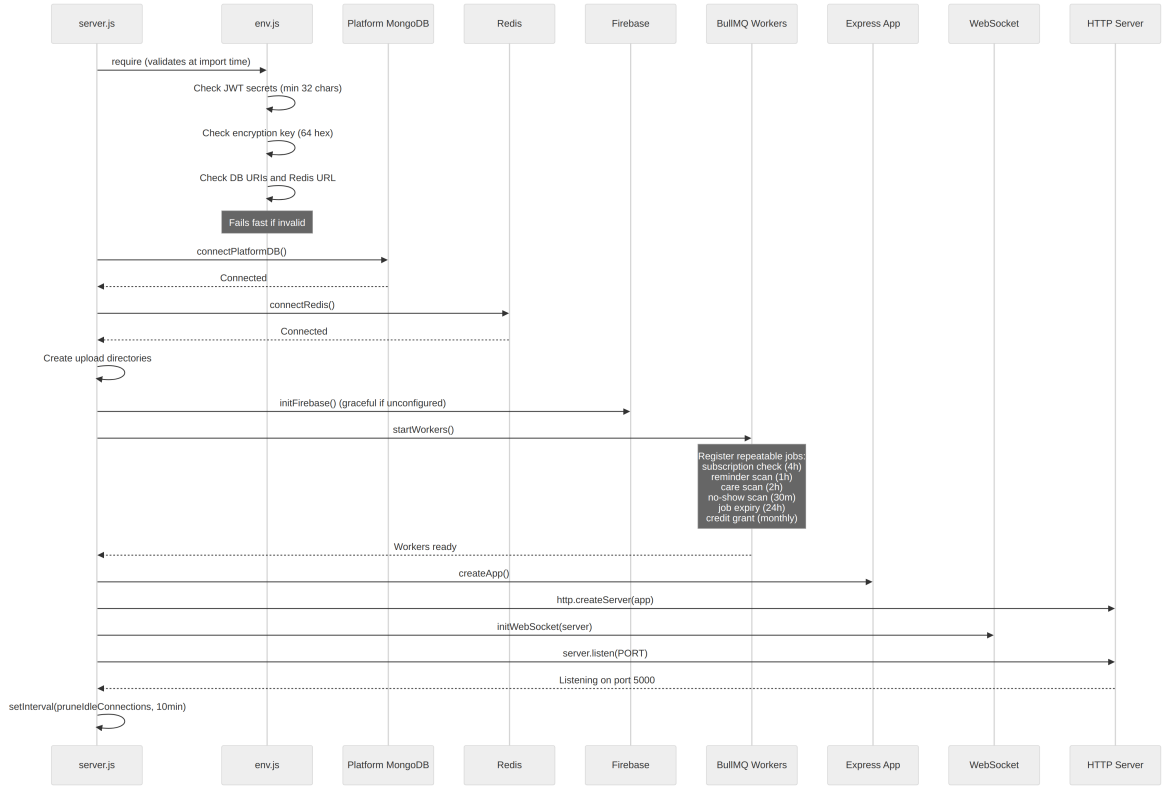


Figure 15: Diagram 15: Boot Sequence

16.4 Graceful Shutdown

Shutdown Timeout: 28 seconds (force exit if exceeded)

```
1. Stop accepting new HTTP connections (server.close())
2. Stop BullMQ workers (drain active jobs)
```

```
3. Close all tenant database connections
4. Disconnect platform MongoDB
5. Disconnect Redis
6. Log "Shutdown complete" and exit(0)

Triggers: SIGTERM, SIGINT, uncaughtException
```

16.5 Environment Configuration

Required Environment Variables:

Variable	Purpose	Example
`NODE_ENV`	Environment mode	production
`PORT`	Server port	5000
`PLATFORM_MONGO_URI`	Platform MongoDB connection	mongodb://...
`REDIS_URL`	Redis connection	redis://...
`JWT_SECRET`	Access token signing key	(random 256-bit)
`JWT_REFRESH_SECRET`	Refresh token signing key	(random 256-bit)
`JWT_PLATFORM_SECRET`	Platform token signing key	(random 256-bit)
`ENCRYPTION_KEY`	AES-256 key for data encryption	(random 256-bit)
`TELEMEDICINE_SECRET`	Video session encryption key	(random 256-bit)
`RAZORPAY_KEY_ID`	Razorpay API key	rzp_...
`RAZORPAY_KEY_SECRET`	Razorpay API secret	(secret)
`RAZORPAY_WEBHOOK_SECRET`	Razorpay webhook HMAC secret	(secret)
`GEMINI_API_KEY`	Google Generative AI key	AI...
`AWS_ACCESS_KEY_ID`	S3 access key	AKIA...
`AWS_SECRET_ACCESS_KEY`	S3 secret key	(secret)
`AWS_S3_BUCKET`	S3 bucket name	bestoclinic-uploads
`AWS_REGION`	AWS region	ap-south-1
`SMTP_HOST`	Email SMTP host	smtp.gmail.com
`SMTP_PORT`	Email SMTP port	587
`SMTP_USER`	Email username	noreply@bestoclinic.com
`SMTP_PASS`	Email password	(secret)
`FIREBASE_PROJECT_ID`	Firebase project	bestoclinic-xxx
`FIREBASE_CLIENT_EMAIL`	Firebase service account	firebase-adminsdk@...
`FIREBASE_PRIVATE_KEY`	Firebase private key	(PEM key)
`FRONTEND_URL`	Frontend base URL	https://bestoclinic.com

Chapter 17: System Monitoring & Observability

17.1 Health Check Endpoint

Endpoint: `GET /api/v1/health` (no authentication required)

Response:

```
{
  "success": true,
  "message": "BestoClinic API is running",
  "data": {
    "uptime": 86423.456,
    "timestamp": "2025-05-25T10:30:00.000Z",
    "environment": "production"
  }
}
```

Used by Docker health checks, load balancers, and monitoring systems.

17.2 Logging

Library: Winston (via `src/utils/logger.js`)

Log Levels: error, warn, info, http, debug

Output Targets: - Console (development): colorized, human-readable - File (production): JSON format for log aggregation - Structured fields: timestamp, level, message, metadata

HTTP Access Logging: Morgan middleware in `combined` format

17.3 Error Tracking

Centralized Error Handler (`src/middleware/errorHandler.js`): - Catches all unhandled errors from route handlers - Formats consistent error responses - Logs error details with stack traces - Distinguishes operational errors (ApiError) from programming errors

Process-Level Error Handling: - `unhandledRejection` — Logged, process continues - `uncaughtException` — Logged, triggers graceful shutdown

17.4 Platform Monitoring

Location: `src/modules/productOwner/` (platform admin dashboard)

The platform admin dashboard provides system-level monitoring:

- **Tenant Health:** Status of all clinic tenants (active, suspended, provisioning)
- **Subscription Metrics:** MRR, churn rate, trial conversions
- **API Latency:** Response time trends
- **Error Rates:** 4xx/5xx response counts
- **Resource Usage:** Storage per tenant, active connections

17.5 Connection Pool Monitoring

The tenant connection manager provides observability into database connection pools:

- **Active Connections:** Number of currently connected tenant databases
- **Pool Utilization:** Active vs. maximum pool size
- **Idle Pruning:** Connections closed during 10-minute prune cycles
- **Connection Errors:** Failed connection attempts logged with tenant context

---

Chapter 18: Scalability Considerations

18.1 Horizontal Scaling Strategy

Application Layer: - Stateless Express servers (no in-process session state) - JWT-based authentication eliminates server affinity - Redis-backed sessions and cache enable multi-instance deployment - WebSocket with Redis adapter for cross-instance event distribution

Database Layer: - Database-per-tenant enables per-tenant scaling - MongoDB replica sets for read scaling and high availability - Tenant connection pool with LRU eviction manages connection count - Idle connection pruning prevents resource exhaustion

Job Processing: - BullMQ workers can run on separate processes/machines - Job distribution handled by Redis (BullMQ's backing store) - Concurrency tunable per queue and per worker instance

18.2 Caching Strategy

Cache Layer	Storage	TTL	Purpose
Tenant DB URIs	Redis	1 hour	Avoid repeated decryption
Analytics queries	In-memory	5 minutes	Reduce DB aggregation load
Branch monitor	In-memory	15-30 seconds	Real-time-ish dashboards
Notification unread count	In-memory	30 seconds	Reduce count queries
Queue current state	In-memory	15 seconds	Live queue display
Role permissions	In-memory	5 minutes	Permission lookups
Feature flags	In-memory	Varies	Feature gate checks
Token blacklist	Redis	Token remaining life	Revoked token tracking

18.3 Database Indexing Strategy

All models are indexed for their primary query patterns:

High-Traffic Indexes: - `appointments: { doctor: 1, scheduledAt: 1 }` — Doctor schedule lookups - `appointments: { patient: 1, scheduledAt: -1 }` — Patient history - `appointments: { status: 1 }` — Status-based filtering - `patients: { email: 1 }` (unique) — Login/search - `invoices: { patient: 1, status: 1 }` — Billing queries - `notifications: { recipient: 1, isRead: 1, createdAt: -1 }` — Notification list - `queueentries: { branch: 1, status: 1, createdAt: -1 }` — Live queue

Soft-Delete Pattern: All models with soft-delete have `{ deletedAt: 1 }` index for efficient filtering.

TTL Indexes: - `guestscans: { expiresAt: 1 }` — 30-day auto-cleanup - `prescriptionscans: { expiresAt: 1 }` — 30-day auto-cleanup

18.4 Performance Patterns

Atomic Operations: - Invoice payment updates use `$inc` and `$push` (no read-modify-write) - Cash register close uses CAS (Compare-And-Set) via `findOneAndUpdate` - Ledger entries use MongoDB transactions for balance consistency - Queue token generation uses retry loop for race condition handling

Parallel Query Execution: - Dashboard aggregations use `Promise.all()` for concurrent MongoDB queries - Analytics endpoints compute multiple metrics in parallel - Global search uses `Promise.allSettled()` for partial failure tolerance

Denormalization: - Branch data denormalized to FeaturedDoctor records (avoids joins) - Clinic logo backfilled to JobPosting records - Invoice totals pre-calculated and stored (not computed on read)

18.5 Connection Limits

Resource	Limit	Configuration
Tenant DB connections	50 (configurable)	MAX_POOL_SIZE
HTTP body size	50MB	express.json({ limit })
File upload size	10MB (dental), 5MB (logo), 50MB (CSV)	Per-route multer config
Rate limit	Configurable per IP	express-rate-limit
WebSocket reconnect	10 attempts	Client-side config
Shutdown timeout	28 seconds	SHUTDOWN_TIMEOUT_MS

Part VI — Future Considerations

Chapter 19: Planned Enhancements

19.1 Near-Term Roadmap

Based on the codebase architecture and current feature set, the following enhancements are natural extensions:

1. **React Query / SWR Integration** — Replace manual `useEffect` data fetching with proper data fetching library for automatic caching, revalidation, and optimistic updates. The current pattern requires manual loading/error state management in every component.
2. **HTTP-Only Secure Cookies** — Migrate from localStorage-based JWT storage to HTTP-only secure cookies. The current localStorage approach is vulnerable to XSS token theft.
3. **WebSocket Redis Adapter** — Add Socket.IO Redis adapter (`@socket.io/redis-adapter`) to enable WebSocket event distribution across multiple server instances. Currently, WebSocket events only reach clients connected to the same server instance.
4. **Structured Logging** — Enhance Winston logger with correlation IDs (request tracing), tenant context in every log entry, and integration with log aggregation services (ELK, Datadog, CloudWatch).
5. **API Documentation** — Generate OpenAPI/Swagger documentation from route definitions and Joi validation schemas. The codebase has consistent patterns that could enable auto-generation.
6. **E2E Testing Suite** — Add comprehensive end-to-end tests for critical workflows (patient booking, consultation, billing, payment). The codebase currently lacks automated test coverage.
7. **Webhook Retry Queue** — Add exponential backoff retry for outgoing webhooks and notification delivery failures, with dead letter queue and manual retry dashboard.

19.2 Medium-Term Enhancements

8. **Multi-Language Support (i18n)** — Internationalization for both frontend UI and notification templates. The codebase currently has English-only strings.
9. **Advanced Reporting Engine** — Custom report builder allowing clinics to create ad-hoc queries and visualizations from their data, with scheduled report delivery.
10. **Audit Log Dashboard** — User-facing audit trail viewer with filters, search, and export. Currently audit logs are stored but not exposed through a dedicated UI.
11. **Appointment Waitlist** — Allow patients to join a waitlist for fully-booked time slots with automatic notification when a slot opens.
12. **Advanced Queue Features** — Estimated wait time calculation, SMS queue position updates, and queue analytics.

Chapter 20: Technical Debt & Recommendations

20.1 Identified Technical Debt

Area	Issue	Impact	Recommendation
Frontend State	Manual `useEffect` data fetching without caching library	Duplicate requests, stale data, complex loading states	Adopt React Query or SWR
Token Storage	JWT stored in localStorage	XSS vulnerability can steal tokens	Migrate to HTTP-only secure cookies
Test Coverage	No automated tests visible in codebase	Regression risk on changes	Add unit, integration, and E2E tests
WebSocket Scaling	No Redis adapter for Socket.IO	Multi-instance deployments won't share events	Add `@socket.io/redis-adapter`
Error Boundaries	Limited React error boundaries in frontend	Unhandled component errors crash entire app	Add error boundaries at route level
Type Safety	JavaScript without TypeScript	Runtime type errors, harder refactoring	Consider gradual TypeScript migration
API Versioning	Single version (`/api/v1`) with no migration path	Breaking changes affect all clients	Design v2 strategy for future
Database Migrations	No formal migration tool	Schema changes require manual scripts	Adopt migration tool (migrate-mongo)
Secret Rotation	No automated key rotation mechanism	Compromised keys require manual rotation	Implement key rotation with vault
Monitoring	Basic health check only	Limited production visibility	Add APM (Application Performance Monitoring)

20.2 Architecture Recommendations

1. **Service Layer Consistency** — Some service files (e.g., `patientPortal.service.js` at 82KB) are excessively large. Consider splitting into sub-services (e.g., `portalBooking.service.js`, `portalPayment.service.js`) to improve maintainability.
2. **Event-Driven Decoupling** — The consultation completion cascade (inventory, prescription, care tasks, invoicing, notifications) is currently procedural. Consider extracting into an event-driven pattern where `consultation:completed` triggers independent handlers, im-

proving testability and resilience.

3. **Database Connection Pooling** — The per-tenant connection model works well for isolation but may face scaling challenges beyond 50-100 tenants. Consider connection multiplexing or a connection broker pattern for high-tenant-count deployments.
4. **Frontend Component Library** — Extract shared design tokens and components (cards, buttons, modals, tables) into a proper design system package. The current approach of inline utility class strings (`cardCls`, `btnPrimary`) works but doesn't scale well.
5. **Rate Limiting Granularity** — Current rate limiting is per-IP only. Consider per-tenant and per-user rate limiting to prevent abuse within multi-tenant contexts.
6. **Idempotency Keys** — Extend idempotency key patterns (currently only on Razorpay webhooks) to all mutating API endpoints. This prevents duplicate operations from client retries or network issues.

20.3 Security Recommendations

1. **Content Security Policy** — Tighten CSP headers to restrict script sources, preventing XSS attack vectors.
2. **Input Sanitization** — While Joi validation handles structure, consider adding DOMPurify or similar for user-generated content fields (notes, descriptions) that might be rendered in the frontend.
3. **Rate Limiting on Auth** — Add stricter rate limiting on login and password reset endpoints to prevent brute-force attacks.
4. **Dependency Auditing** — Implement automated `npm audit` in CI/CD pipeline to catch known vulnerabilities in dependencies.
5. **Database Encryption at Rest** — Enable MongoDB encryption at rest for sensitive medical data (HIPAA/DISHA compliance).

Appendices

Appendix A: Glossary

Term	Definition
Tenant	A clinic organization with its own isolated database and user base
Platform	The shared BestoClinic SaaS infrastructure and admin layer
Slug	URL-safe unique identifier for a tenant (e.g., `demo-dental-clinic`)
RBAC	Role-Based Access Control — permission system based on user roles
SOAP Notes	Medical documentation format: Subjective, Objective, Assessment, Plan
FDI Notation	International tooth numbering system (1-48) used in dental charts
HSN Code	Harmonized System Nomenclature code for GST tax classification
GST	Goods and Services Tax (Indian tax system: CGST + SGST or IGST)
CGST/SGST	Central/State GST — applied for intrastate supply (50/50 split)
IGST	Integrated GST — applied for interstate supply (full rate)
BullMQ	Redis-backed job queue library for Node.js
Socket.IO	Real-time bidirectional event-based communication library
WebRTC	Web Real-Time Communication protocol for peer-to-peer video/audio
ICE Candidate	Interactive Connectivity Establishment — WebRTC NAT traversal
SDP	Session Description Protocol — WebRTC media negotiation format
TURN/STUN	NAT traversal servers for WebRTC connectivity
AES-256-GCM	Advanced Encryption Standard with Galois/Counter Mode
JWT	JSON Web Token — compact claims-based authentication token
LRU	Least Recently Used — cache eviction strategy
CAS	Compare-And-Set — atomic update pattern preventing race conditions
TTL	Time To Live — expiration duration for cached data
ICS	iCalendar format (RFC 5545) for calendar data interchange
FCM	Firebase Cloud Messaging — push notification service
PACS	Picture Archiving and Communication System — medical imaging
HMAC	Hash-based Message Authentication Code — webhook signature verification

Appendix B: Sample API Request/Response Payloads

B.1 Create Appointment

Request:

```
POST /api/v1/appointments HTTP/1.1
Authorization: Bearer eyJhbGciOiJIUzI1NiIs...
X-Tenant-ID: demo-dental-clinic
Content-Type: application/json

{
  "patient": "665a1b2c3d4e5f6a7b8c9d0e",
  "doctor": "665a1b2c3d4e5f6a7b8c9d1f",
  "scheduledAt": "2025-06-15T10:30:00.000Z",
  "duration": 30,
  "type": "consultation",
  "branch": "665a1b2c3d4e5f6a7b8c9d2a",
  "notes": "First visit — tooth pain upper left",
  "source": "receptionist"
}
```

Response (201 Created):

```
{
  "success": true,
  "message": "Appointment created successfully",
  "data": {
    "_id": "665b2c3d4e5f6a7b8c9d0e1f",
    "patient": {
      "_id": "665a1b2c3d4e5f6a7b8c9d0e",
      "name": "Rajesh Kumar",
      "patientId": "PAT-0042"
    },
    "doctor": {
      "_id": "665a1b2c3d4e5f6a7b8c9d1f",
      "name": "Dr. Priya Sharma",
      "specializations": ["General Dentistry"]
    },
    "scheduledAt": "2025-06-15T10:30:00.000Z",
    "duration": 30,
    "type": "consultation",
    "status": "confirmed",
    "reminders": [
      { "minutesBefore": 1440, "jobId": "rem_665b..." },
      { "minutesBefore": 60, "jobId": "rem_665c..." }
    ],
    "createdAt": "2025-06-10T14:22:33.456Z"
  }
}
```

B.2 Complete Consultation

Request:

```
PUT /api/v1/doctor/consultations/665c3d4e5f6a7b8c9d0e1f2a/complete HTTP/1.1
Authorization: Bearer eyJhbGciOiJIUzI1NiIs...
X-Tenant-ID: demo-dental-clinic
```

```
Content-Type: application/json

{
  "diagnosis": "Dental caries – upper left second premolar (#25)",
  "treatment": "Composite filling performed under local anesthesia",
  "prescription": [
    {
      "name": "Amoxicillin 500mg",
      "dosage": "500mg",
      "frequency": "3 times daily",
      "duration": "5 days",
      "instructions": "After meals"
    },
    {
      "name": "Ibuprofen 400mg",
      "dosage": "400mg",
      "frequency": "As needed",
      "duration": "3 days",
      "instructions": "For pain relief, max 3 per day"
    }
  ],
  "materialsUsed": [
    { "item": "665d4e5f6a7b8c9d0e1f2a3b", "itemName": "Composite Resin A2", "quantity": 1, "costPerUnit": 450 },
    { "item": "665d4e5f6a7b8c9d0e1f2a3c", "itemName": "Local Anesthetic Cartridge", "quantity": 2, "costPerUnit": 8
  ],
  "careTasks": [
    {
      "title": "Avoid hard foods",
      "taskType": "post_care",
      "frequency": "once_daily",
      "enabled": true,
      "reminderEnabled": true
    }
  ],
  "teeth": [25],
  "followUpDate": "2025-06-29T10:00:00.000Z",
  "followUpInstructions": "Review filling and check for sensitivity",
  "signature": {
    "dataUrl": "...",
    "signedAt": "2025-06-15T11:15:00.000Z",
    "signedBy": "665a1b2c3d4e5f6a7b8c9d1f",
    "ipAddress": "192.168.1.100"
  }
}
```

Response (200 OK):

```
{
  "success": true,
  "message": "Consultation completed successfully",
  "data": {
    "_id": "665c3d4e5f6a7b8c9d0e1f2a",
    "status": "completed",
    "completedAt": "2025-06-15T11:15:00.000Z",
    "diagnosis": "Dental caries – upper left second premolar (#25)",
    "treatment": "Composite filling performed under local anesthesia",
    "prescription": { "_id": "665e5f6a7b8c...", "status": "active", "medications": [...] },
    "materialsUsed": [...],
    "followUpDate": "2025-06-29T10:00:00.000Z"
  }
}
```

B.3 Paginated Patient List

Request:

```
GET /api/v1/patients?page=1&limit=20&search=kumar&gender=male HTTP/1.1
Authorization: Bearer eyJhbGciOiJIUzI1NiIs...
X-Tenant-ID: demo-dental-clinic
```

Response (200 OK):


```
{
  "success": true,
  "message": "Patients fetched",
  "data": [
    {
      "_id": "665a1b2c3d4e5f6a7b8c9d0e",
      "patientId": "PAT-0042",
      "name": "Rajesh Kumar",
      "email": "rajesh.kumar@email.com",
      "phone": "+919876543210",
      "dateOfBirth": "1985-03-15",
      "gender": "male",
      "primaryDoctor": { "_id": "...", "name": "Dr. Priya Sharma" },
      "isActive": true
    }
  ],
  "meta": {
    "page": 1,
    "limit": 20,
    "total": 47,
    "totalPages": 3,
    "hasNextPage": true,
    "hasPrevPage": false
  }
}
```

Appendix C: Sample Webhook Payloads

C.1 Razorpay Payment Captured

```
{
  "entity": "event",
  "account_id": "acc_XXXXXXXXXXXXX",
  "event": "payment.captured",
  "contains": ["payment"],
  "payload": {
    "payment": {
      "entity": {
        "id": "pay_XXXXXXXXXXXXX",
        "amount": 150000,
        "currency": "INR",
        "status": "captured",
        "order_id": "order_XXXXXXXXXXXXX",
        "method": "upi",
        "email": "patient@email.com",
        "contact": "+919876543210",
        "notes": {
          "invoiceId": "665f6a7b8c9d0e1f2a3b4c5d",
          "tenantSlug": "demo-dental-clinic",
          "patientId": "665a1b2c3d4e5f6a7b8c9d0e"
        }
      }
    }
  },
  "created_at": 1718450000
}
```

C.2 Razorpay Subscription Charged

```
{
  "entity": "event",
  "event": "subscription.charged",
  "payload": {
    "subscription": {
      "entity": {
        "id": "sub_XXXXXXXXXXXXX",
        "plan_id": "plan_XXXXXXXXXXXXX",
        "status": "active",
        "current_start": 1718450000,
        "current_end": 1721128400,
        "charge_at": 1721128400,
        "notes": {
          "clinicSlug": "demo-dental-clinic"
        }
      }
    },
    "payment": {
      "entity": {
        "id": "pay_YYYYYYYYYYYYY",
        "amount": 499900,
        "currency": "INR",
```

```
        "status": "captured"
      }
    }
  }
}
```

Appendix D: Configuration Reference

D.1 Subscription Plans

Plan	Slug	Monthly	Yearly	Trial Days	Max Doctors	Max Staff	Max Branches	Storage
Free	`free`	₹0	₹0	0	1	2	1	100 MB
Starter	`starter`	₹999	₹9,990	14	3	10	2	1 GB
Professional	`professional`	₹2,499	₹24,990	14	10	50	5	10 GB
Enterprise	`enterprise`	₹4,999	₹49,990	14	Unlimited	Unlimited	Unlimited	100 GB

D.2 Feature Matrix

Feature	Free	Starter	Professional	Enterprise
SMS Notifications	—	Yes	Yes	Yes
Telehealth	—	—	Yes	Yes
Custom Branding	—	—	Yes	Yes
API Access	—	—	—	Yes
Advanced Reports	—	—	Yes	Yes
White Labeling	—	—	—	Yes
Job Portal	—	—	Yes	Yes
Priority Support	—	—	—	Yes
External Integrations	—	—	—	Yes

D.3 Role-Permission Matrix (Summary)

Permission Category	super_admin	admin	manager	doctor	receptionist	billing_officer	patient
User Management	Full	Full	Read	—	—	—	—
Patient Records	Full	Full	Full	Read Own	Read	—	Read Own
Appointments	Full	Full	Full	Read Own	Full	Read	Read Own
Invoices	Full	Full	Full	Read	Create	Full	Read Own
Payments	Full	Full	Full	—	Create	Full	Read Own
Cash Register	Full	Full	Full	—	Open/Close	Full	—
Prescriptions	Full	Full	Read	Full	Read	—	Read Own

Permission Category	super_admin	admin	manager	doctor	receptionist	billing_officer	patient
Queue	Full	Full	Full	Read	Full	—	—
Analytics	Full	Full	Full	Dashboard	—	Revenue	—
Clinic Settings	Full	Full	Read	—	—	—	—
Staff Management	Full	Full	Read	—	—	—	—
Inventory	Full	Full	Full	Read	—	—	—
Insurance	Full	Full	Full	Read	—	Full	Read Own

End of Technical Report

Document Information:

Field	Value
Document Title	BestoClinic — Comprehensive Technical Project Report
Version	1.0.0
Project Version	1.0.0
Total Chapters	20
Total Appendices	4
Diagrams Embedded	18 rendered diagrams
Generated	May 2025